

대분류/20
정보통신

중분류/01
정보기술

소분류/02
정보기술개발

세분류/02
응용SW엔지니어링

능력단위/15

NCS학습모듈

프로그래밍 언어 활용

LM2001020215_15v3



교육부

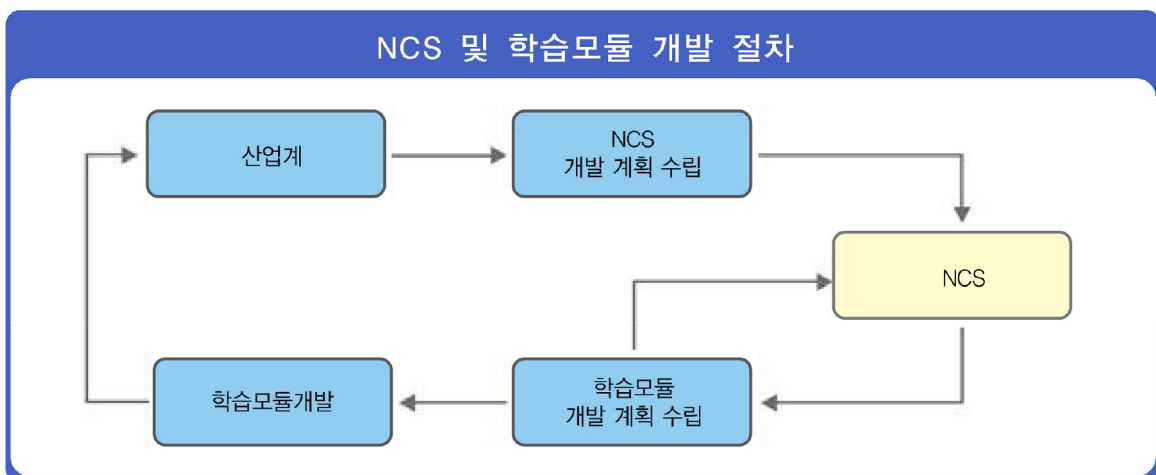
NCS 학습모듈은 교육훈련기관에서 출처를 명시하고 교육적 목적으로 활용할 수 있습니다. 다만 NCS 학습모듈에는 국가(교육부)가 저작권 일체를 보유하지 않은 저작물들(출처가 표기되어 있는 도표, 사진, 삽화, 도면 등)이 포함되어 있으므로 이러한 저작물들의 변형, 복제, 공연, 배포, 공중 송신 등과 이러한 저작물들을 활용한 2차 저작물의 생성을 위해서는 반드시 원작자의 동의를 받아야 합니다.

NCS 학습모듈의 이해

※ 본 학습모듈은 「NCS 국가직무능력표준」 사이트(<http://www.ncs.go.kr>) 에서 확인 및 다운로드 할 수 있습니다.

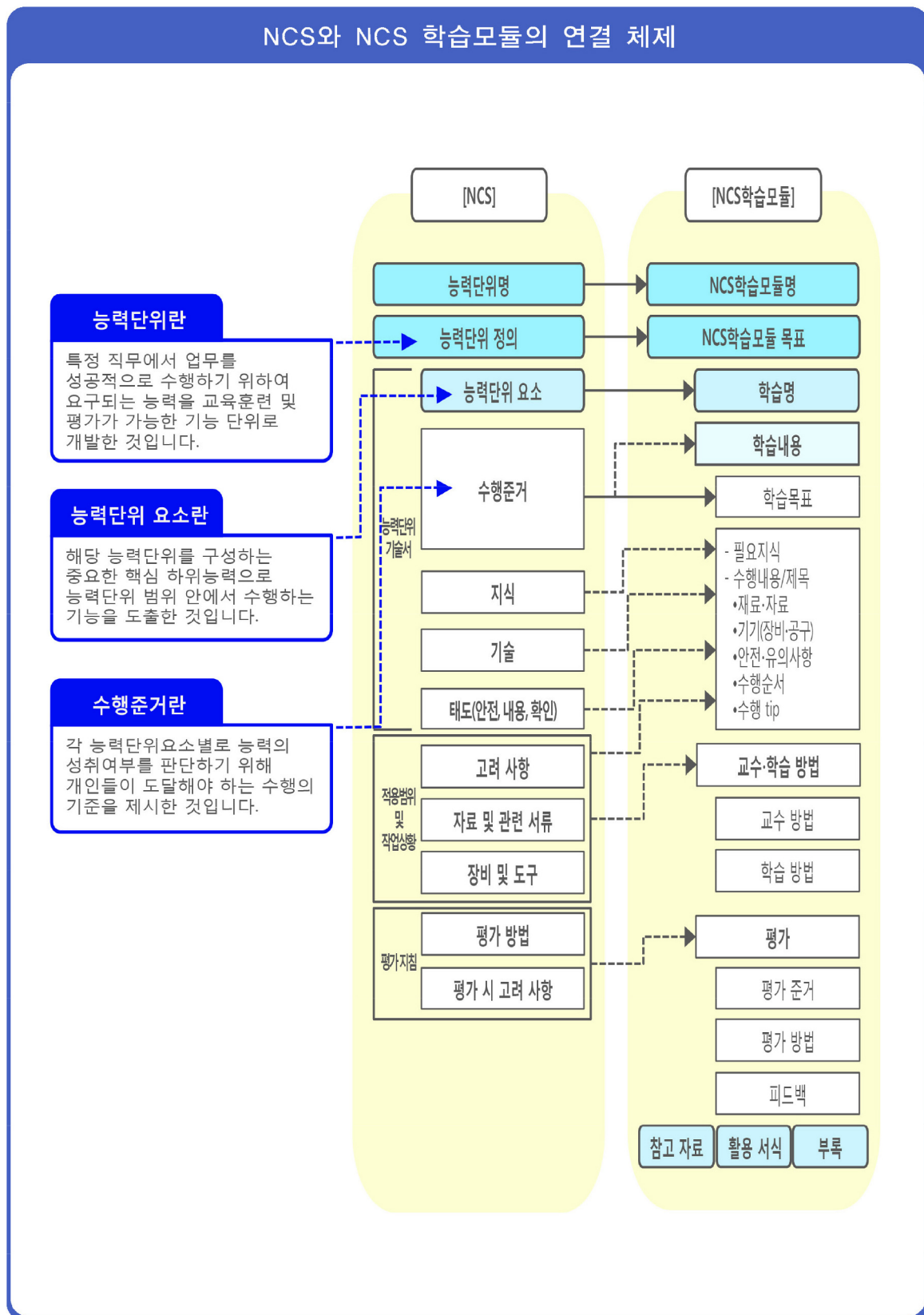
(1) NCS 학습모듈이란?

- 국가직무능력표준(NCS: National Competency Standards)이란 산업현장에서 직무를 수행하기 위해 요구되는 지식·기술·소양 등의 내용을 국가가 산업부문별·수준별로 체계화한 것으로 산업현장의 직무를 성공적으로 수행하기 위해 필요한 능력(지식, 기술, 태도)을 국가적 차원에서 표준화한 것을 의미합니다.
- 국가직무능력표준(이하 NCS)이 현장의 ‘직무 요구서’라고 한다면, NCS 학습모듈은 NCS의 능력단위를 교육훈련에서 학습할 수 있도록 구성한 ‘교수·학습 자료’입니다. NCS 학습모듈은 구체적인 직무를 학습할 수 있도록 이론 및 실습과 관련된 내용을 상세하게 제시하고 있습니다.



- NCS 학습모듈은 다음과 같은 특징을 가지고 있습니다.
- 첫째, NCS 학습모듈은 산업계에서 요구하는 직무능력을 교육훈련 현장에 활용할 수 있도록 성취목표와 학습의 방향을 명확히 제시하는 가이드라인의 역할을 합니다.
- 둘째, NCS 학습모듈은 특성화고, 마이스터고, 전문대학, 4년제 대학교의 교육기관 및 훈련기관, 직장교육기관 등에서 표준교재로 활용할 수 있으며 교육과정 개편 시에도 유용하게 참고할 수 있습니다.

- NCS와 NCS 학습모듈 간의 연결 체제를 살펴보면 아래 그림과 같습니다.



(2) NCS 학습모듈의 체계

- NCS 학습모듈은 1.학습모듈의 위치, 2.학습모듈의 개요, 3.학습모듈의 내용 체계, 4.참고 자료, 5.활용 서식/부록 으로 구성되어 있습니다.

1. NCS 학습모듈의 위치

- NCS 학습모듈의 위치는 NCS 분류 체계에서 해당 학습모듈이 어디에 위치하는지를 한 눈에 볼 수 있도록 그림으로 제시한 것입니다.

예시 : 이·미용 서비스 분야 중 네일미용 세분류

NCS-학습모듈의 위치

대분류	이용·숙박·여행·오락·스포츠
중분류	이·미용
소분류	아미용 서비스

세분류	능력단위	학습모듈명
헤어미용	네일 샵 위생 서비스	네일샵 위생서비스
피부미용	네일 화장물 제거	네일 화장물 제거
메이크업	네일 기본 관리	네일 기본관리
네일미용	네일 랩	네일 랩
이용	네일 팁	네일 팁
	젤 네일	젤 네일
	아크릴릭 네일	아크릴 네일
	평면 네일아트	평면 네일아트
	융합 네일아트	융합 네일아트
	네일 샵 운영관리	네일샵 운영관리

학습모듈은

NCS 능력단위 1개당 1개의 학습모듈 개발을 원칙으로 합니다. 그러나 필요에 따라 고용 단위 및 교과단위를 고려하여 능력단위 몇 개를 묶어서 1개의 학습모듈로 개발할 수 있으며, NCS 능력단위 1개를 여러 개의 학습모듈로 나누어 개발할 수도 있습니다.

2. NCS 학습모듈의 개요

구 성

- NCS 학습모듈 개요는 학습모듈이 포함하고 있는 내용을 개략적으로 설명한 것으로서 **학습모듈의 목표**, **선수 학습**, **학습모듈의 내용 체계**, **핵심 용어**로 구성되어 있습니다.

학습모듈의 목표	해당 NCS 능력단위의 정의를 토대로 학습목표를 작성한 것입니다.
선수 학습	해당 학습모듈에 대한 효과적인 교수·학습을 위하여 사전에 이수해야 하는 학습모듈, 학습 내용, 관련 교과목 등을 기술한 것입니다.
학습모듈의 내용 체계	해당 NCS 능력단위요소가 학습모듈에서 구조화된 방식을 제시한 것입니다.
핵심 용어	해당 학습모듈의 학습 내용, 수행 내용, 설비·기자재 등 가운데 핵심적인 용어를 제시한 것입니다.

활 용 안 내

예시 : 네일미용 세분류의 ‘네일 기본관리’ 학습모듈

네일 기본관리 학습모듈의 개요

학습모듈의 목표

고객의 네일 보호와 미적 요구 충족을 위하여 효과적인 네일 관리로 프리에지 형태 만들기, 큐티를 정리하기, 컬러링하기, 보습제 도포하기, 마무리를 할 수 있다.

선수학습

네일습 위성서비스(LM1201010401_14V2)

학습모듈의 내용체계

학습	학습 내용	NCS 능력단위 요소	
		코드번호	요소 명칭
1. 프리에지 형태 만들기	1-1. 네일 파일에 대한 이해와 활용 1-2. 프리에지 형태 파일링	1201010403_12v2.1	프리에지 모양 만들기
2. 큐티를 정리하기	2-1. 네일 기본관리 매뉴얼 이해 2-2. 큐티를 관리	1201010403_14v2.2	큐티를 정리하기
3. 컬러링하기	3-1. 컬러링 매뉴얼 이해 3-2. 컬러링 방법 선정과 작업 3-3. 쉘 컬러링 작업	1201010403_14v2.3	컬러링
4. 보습제 도포하기	4-1. 보습제 선정과 도포 4-2. 각질제거	1201010403_14v2.4	보습제 바르기
5. 네일 기본관리 마무리하기	5-1. 유분기 제거 5-2. 네일 기본관리 마무리와 정리	1201010403_14v2.5	마무리하기

핵심 용어

프리에지, 니퍼, 루셔, 플러시, 네일 파일, 스웨이형, 스웨이 오프형, 라운드형, 오발형, 포인트형

학습모듈의 목표는

학습자가 해당 학습모듈을 통해 성취해야 할 목표를 제시한 것으로, 교수자는 학습자가 학습모듈의 전체적인 내용흐름을 파악할 수 있도록 지도하는 것이 필요합니다.

선수 학습은

교수자나 학습자가 해당 모듈을 교수 또는 학습하기 이전에 이수해야 할 학습내용, 교과목, 핵심 단어 등을 표기한 것입니다. 따라서 교수자는 학습자가 개별 학습, 자기 주도 학습, 방과 후 활동 등 다양한 방법을 통해 이수할 수 있도록 지도하는 것이 필요합니다.

핵심 용어는

학습모듈을 통해 학습되고 평가되어야 할 주요 용어입니다. 또한 당해 모듈 또는 타 모듈에서도 핵심 용어를 사용하여 학습내용을 구성할 수 있으며, 「NCS 국가 직무능력표준」 사이트(www.ncs.go.kr)에서 색인(찾아보기) 중 하나를 이용할 수 있습니다.

3. NCS 학습모듈의 내용 체계

구 성

● NCS 학습모듈의 내용은 크게 **학습**, **학습 내용**, **교수·학습 방법**, **평가**로 구성되어 있습니다.

학습	해당 NCS 능력단위요소 명칭을 사용하여 제시한 것입니다. 학습은 크게 학습 내용, 교수·학습 방법, 평가로 구성되며 해당 NCS 능력단위의 능력단위 요소별 지식, 기술, 태도 등을 토대로 학습 내용을 제시한 것입니다.
학습 내용	학습 내용은 학습 목표, 필요 지식, 수행 내용으로 구성하였으며, 수행 내용은 재료·자료, 기기(장비·공구), 안전·유의 사항, 수행 순서, 수행 tip으로 구성한 것입니다. 학습모듈의 학습 내용은 업무의 표준화된 프로세스에 기반을 두고 실제 산업현장에서 이루어지는 업무활동을 다양한 방식으로 반영한 것입니다.
교수·학습 방법	학습 목표를 성취하기 위한 교수자와 학습자 간, 학습자와 학습자 간의 상호작용이 활발하게 일어날 수 있도록 교수자의 활동 및 교수 전략, 학습자의 활동을 제시한 것입니다.
평가	평가는 해당 학습모듈의 학습 정도를 확인할 수 있는 평가 준거, 평가 방법, 평가 결과의 피드백 방법을 제시한 것입니다.

활 용 안 내

예시 : 네일미용 세분류의 ‘네일 기본관리’ 학습모듈의 내용

학습 1	프리에지 형태 만들기(LM1201010403_14v2.1)
학습 2	큐티를 정리하기(LM1201010403_14v2.2)
학습 3	컬러링하기(LM1201010403_14v2.3)
학습 4	보습제 도포하기(LM1201010403_14v2.4)
학습 5	네일 기본관리 마무리하기(LM1201010403_14v2.5)

학습은

해당 NCS 능력단위요소 명칭을 사용하여 제시하였습니다. 학습은 일반교과의 '대단원'에 해당되며, 모듈을 구성하는 가장 큰 단위가 됩니다. 또한 완성된 직무를 수행하기 위한 가장 기본적인 단위로 사용할 수 있습니다.

학습내용은

요소 별 수행준거를 기준으로 제시하였습니다. 일반교과의 '중단원'에 해당합니다.

학습목표는

모듈 내의 학습내용을 이수했을 때 학습자가 보여줄 수 있는 행동수준을 의미합니다. 따라서 일반 수업시간의 과목목표로 활용할 수 있습니다.

필요지식은

해당 NCS의 지식을 토대로 해당 학습에 대한 이해와 성과를 높이기 위해 알아야 할 주요 지식을 제시하였습니다. 필요지식은 수행에 꼭 필요한 핵심 내용을 위주로 제시하여 교수자의 역할이 매우 중요하며, 이후 수행순서 내용과 연계하여 교수·학습으로 진행할 수 있습니다.

3-1. 컬러링 매뉴얼 이해

학습목표

- 고객의 요구에 따라 네일 폴리시 색상과 침착을 먹기 위한 베이스코트를 아주 얇게 도포할 수 있다.
- 작업 매뉴얼에 따라 네일 폴리시를 일찍 없이 균일하게 도포할 수 있다.
- 작업 매뉴얼에 따라 네일 폴리시 도포 후 컬러 보호와 광택 부여를 위한 톱코트를 바를 수 있다.

필요 지식 /

□ 컬러링 매뉴얼

컬러링 작업 전, 아세톤 또는 네일 폴리시 리무버를 사용하여 손톱표면과 큐티를 주변, 손톱 밑 부분까지 깨끗하게 유분기를 제거해야 한다. 컬러링의 순서는 Base coating 1회 → Polishing 2회 → 컬러수정 → Top coating 1회 → 최종수정의 순서로 한다. 베이스코트는 착색을 방지하고 발림성 향상을 위해 가장 먼저 도포하며 컬러링의 마지막에 컬러의 유지와 광택을 위해 톱코트를 도포한다. 네일 보강제(Nail Strengthner)를 바를 시에는 베이스코트를 도포하기 전에 사용한다.

수행 내용 / 컬러링 매뉴얼 실습하기

재료·자료

- 컬러링 관련 네일 미용 자료들
- 정리바구니, 베이스코트, 네일 폴라시, 톱코트, 오렌지우드스티, 탈지면, 폴라시러무버, 디스펜서 등

기기(장비·공구)

- 컴퓨터, 빔 프로젝터, 스크린 등

안전·유의사항

- 컬러링 재료들의 냄새를 직접적으로 맡지 않도록 유의한다.
- 컬러링 제품들이 대부분 유리병에 들어 있기 때문에 깨지지 않도록 각별히 조심한다.
- 컬러링 제품들은 상온에 마르기 때문에 개봉 후 뚜껑을 잘 닫도록 한다.

수행 순서

Ⅰ 네일 폴라시를 빠르게 잡는다.

1. 손바닥에 네일 폴라시를 놓고 약지 소지를 이용하여 네일 폴라시를 잡는다.
2. 폴라시를 왼 손의 엄지와 검지로 고객의 작업손가락을 잡는다.
3. 폴라시를 왼 손의 중지 손가락을 굳게 펴서 발침대가 되도록 한다.
4. 반대편 손으로 네일 폴라시의 두건을 열고 소지 손가락을 펴서 네일 폴라시를 왼 중지 손가락 위에 발쳐놓는다.
5. 다양한 형태의 폴라시를 잡아본다.

수행 tip

- 편식이 많이 섞인 네일 폴라시의 경우는 붓의 각도를 높이 세워 빠르게 브러시 작업을 해야 붓 자국이 나지 않는다.
- 컬러링은 기본 2회 정도이나 컬러에 따른 도포량과 컬러감에 따라 1~3회 사이로 증감할 수 있다.

수행 내용은

모듈에 제시한 것 중 기술(Skill)을 습득하기 위한 실습 과제로 활용할 수 있습니다.

재료·자료는

수행 내용을 수행하는데 필요한 재료 및 준비물로 실습 시 필요 준비물로 활용할 수 있습니다.

기기(장비·공구)는

수행 내용을 수행하는데 필요한 기본적인 장비 및 도구를 제시하였습니다. 제시된 기기 외에도 수행에 필요한 다양한 도구나 장비를 활용할 수 있습니다.

안전·유의사항은

수행 내용을 수행하는데 안전상 주의해야 할 점 및 유의사항을 제시하였습니다. 수행 시 유념해야 하며, NCS의 고려사항도 추가적으로 활용할 수 있습니다.

수행 순서는

실습과제의 진행 순서로 활용할 수 있습니다.

수행 tip은

수행 내용에서 수행의 수월성을 높일 수 있는 아이디어를 제시하였습니다. 따라서 수행tip은 지도상의 안전 및 유의사항 외에 전반적으로 적용되는 주의점 및 수행과제 목적에 대한 보충설명, 추가사항 등으로 활용할 수 있습니다.

학습3 교수·학습 방법

교수·학습 방법은

학습목표를 성취하는데 필요한 교수 방법과 학습 방법을 제시하였습니다.

교수 방법

- 컬러링 제품의 성분과 컬러별 질도의 차이, 베이스코트와 톱코트의 역할, 폴라시 잡는 방법, 큐어링 시간 등의 내용을 화면 자료와 함께 설명한다.
- 서식지를 활용하여 네일 컬러링 방법을 그림으로 그려 보게 한 뒤, 다양한 컬러링의 매뉴얼을 그려서 숙지하도록 한다.
- 셀 컬러링 시 주의사항을 계속 숙지시키도록 하며, 큐어링 시간에 대해 작성하도록 한다.

교수 방법은

해당 학습활동에 필요한 학습내용, 학습내용과 관련된 학습 자료명, 자료 형태, 수행내용의 진행 방식 등에 대하여 제시하였습니다. 또한 학습자의 수업참여도를 제고하기 위한 방법 및 수업진행상 유의사항 등도 제시하였습니다. 선수학습이 필요한 학습을 학습자가 숙지하였는지 교수자가 확인하는 과정으로 활용할 수도 있습니다.

학습 방법

- 컬러링을 위한 재료의 필요성과 사용방법을 숙지하고 컬러링 매뉴얼 과정에 맞추어 작업 내용을 이해한다.
- 컬러링의 다양성에 대한 용어를 숙지하고 진행과정에 맞추어 내용을 작업한다.
- 셀 컬러링 시 적합한 큐어링 시간을 선택해서 큐어링 해본다.

학습 방법은

해당 학습활동에 필요한 학습자의 자기주도적 학습 방법을 제시하였습니다. 또한 학습자가 숙달해야 할 실기능력과 학습과정에서 주의해야 할 사항 등으로 제시하였습니다. 학습자가 학습을 이수하기 전에 반드시 숙지해야 할 기본 지식을 학습하였는지 스스로 확인하는 과정으로 활용할 수 있습니다.

학습3 평가

평가는

해당 NCS 능력단위 평가방법과 평가 시 고려 사항을 준용하여 작성하였습니다. 교수자 및 학습자가 평가항목 별 성취수준을 확인하는데 활용할 수 있습니다.

평가 준거

- 평가자는 학습자가 학습 목표를 성공적으로 달성하였는지를 평가해야 한다.
- 평가자는 다음 사항을 평가해야 한다.

학습내용	학습 목표	성취수준		
		상	중	하
킬러링 매뉴얼 이해	- 고객의 요구에 따라 네일 폴리시 색상의 칠착을 하기 위한 베이스코트를 아주 얇게 도포할 수 있다.			
	- 작업 매뉴얼에 따라 네일 폴리시를 얼룩 없이 균일하게 도포할 수 있다.			
	- 작업 매뉴얼에 따라 네일 폴리시 도포 후 킬러 보호와 광택 부여를 위한 톱코트를 바를 수 있다.			

평가 준거는

학습자가 해당 학습을 어느 정도 성취하였는지를 평가하기 위한 기준을 제시하고 있습니다. 학습목표와 연계하여 단위수업 시간에 평가항목 별 성취수준을 평가하는데 활용할 수 있습니다.

평가 방법은

평가 방법

- 작업장 평가

학습내용	평가 항목	성취수준		
		상	중	하
킬러링 매뉴얼 이해	- 고객의 요구에 따라 네일 폴리시 색상의 칠착을 하기 위한 베이스코트를 아주 얇게 도포할 수 있다.			
	- 작업 매뉴얼에 따라 네일 폴리시를 얼룩 없이 균일하게 도포할 수 있다.			
	- 작업 매뉴얼에 따라 네일 폴리시 도포 후 킬러 보호와 광택 부여를 위한 톱코트를 바를 수 있다.			

NCS 능력단위의 평가방법을 준용하였으며, 평가 준거에 따른 평가방법을 2개 이상 제시하였습니다. 평가방법으로는 포트폴리오, 문제해결 시나리오, 서술형 시험, 논술형 시험, 사례연구, 평가자 체크리스트, 작업장 평가 등이 있으며, NCS의 능력단위 요소 별 수행 수준을 평가하는데 가장 적절한 방법을 선정하여 활용할 수 있습니다.

피드백은

피드백

1. 작업장 평가
 - 작업 결과물을 확인하여 수정사항을 제시하고 수정 부분을 인지하도록 한다.

평가 후에 학습자들에게 평가 결과를 피드백하여 부족한 부분을 알려주고, 학습 결과가 미진한 경우, 해당 부분을 다시 학습하여 학습목표를 달성하는 데 활용할 수 있습니다.

4. 참고 자료

참고자료

참고자료는

- 김미원(2011). 『Nail Study』. 서울: 사)한국네일저서서비스협회.
- 민봉경(2015). 『미용사(네일)평가』. 서울: 예문사.
- 박은주(2014). 『네일미용』. 서울: 정담미디어.

해당 학습모듈의 필요지식에 대한 출처와 인용한 참고 자료 및 사이트를 제시하였습니다.

5. 활용 서식/부록

활용서식

활용서식은

평가 서식, 실습시트 등 교수학습 시 활용 가능한 다양한 서식들로 구성하였습니다. 과제 진행에서 평가에 이르기까지 필요한 서식을 해당 학습모듈의 특성에 맞춰 개발하거나 기존의 양식을 활용하여 제시하였습니다.

프리에지 형태 실습지

1. 프리에지 형태의 이해

모 양	이 름	특 징
	() Square nail	- 강한 느낌의 사각형태 - 네일의 양끝 모서리 부분이 90° 사각의 형태이다. () - 발톱의 형태 활용 - 내인성 발톱의 보정시에 적용

부록

부록은

활용서식 이외에 교수학습과정에서 참고할 수 있는 자료가 있는 경우 제시하였습니다.

네일 기본관리 도구와 재료 목록

목록	비고	준비
위생가운	흰색	작업자 착용
위생 마스크	흰색	작업자 착용
보호안경	투명한 렌즈 (안경으로 대체 가능)	작업자 착용
재료정리함	재질, 색상 무관	작업대

[NCS-학습מוד의 위치]

대분류	정보 통신	
중분류	정보 기술	
소분류	정보 기술 개발	

세분류

SW 아키텍처	능력단위	학습מוד명
응용SW 엔지니어링	요구 사항 확인	요구 사항 확인
임베디드 SW 엔지니어링	데이터 입출력 구현	데이터 입출력 구현
DB 엔지니어링	통합 구현	통합 구현
NW 엔지니어링	정보 시스템 이행	정보 시스템 이행
보안 엔지니어링	제품 소프트웨어 패키징	제품 소프트웨어 패키징
UI/UX 엔지니어링	서버 프로그램 구현	서버 프로그램 구현
시스템SW 엔지니어링	인터페이스 구현	인터페이스 구현
빅데이터 플랫폼 구축	애플리케이션 배포	애플리케이션 배포
핀테크 엔지니어링	프로그래밍 언어 활용	프로그래밍 언어 활용
데이터 아키텍트	응용SW 기초 기술 활용	응용SW 기초 기술 활용
	애플리케이션 리팩토링	애플리케이션 리팩토링
	인터페이스 설계	인터페이스 설계
	애플리케이션 요구 사항 분석	애플리케이션 요구 사항 분석

기능 모델링	기능 모델링
애플리케이션 설계	애플리케이션 설계
정적 모델 설계	정적 모델 설계
동적 모델 설계	동적 모델 설계
화면 설계	화면 설계
화면 구현	화면 구현
애플리케이션 테스트 관리	애플리케이션 테스트 관리
애플리케이션 테스트 수행	애플리케이션 테스트 수행
소프트웨어 공학 활용	소프트웨어 공학 활용
소프트웨어 개발 방법론 활용	소프트웨어 개발 방법론 활용

차 례

학습모듈의 개요	1
학습 1. 기본 문법 활용하기	
1-1. 변수 활용	3
1-2. 연산자와 명령문 활용	16
1-3. 사용자 정의 자료형 활용	27
• 교수 · 학습 방법	33
• 평가	34
학습 2. 언어 특성 활용하기	
2-1. 언어 특성 활용	36
• 교수 · 학습 방법	61
• 평가	62
학습 3. 라이브러리 활용하기	
3-1. 라이브러리 활용	64
• 교수 · 학습 방법	75
• 평가	76
참고 자료	78

프로그래밍 언어 활용 학습מוד의 개요

학습מוד의 목표

응용소프트웨어 개발에 사용되는 프로그래밍 언어의 기초 문법을 적용하고 언어의 특징과 라이브러리를 활용하여 기본 응용소프트웨어를 구현할 수 있다.

선수학습

프로그래밍 기초, 자료 구조, 데이터 통신 체계, 컴퓨터 시스템 구조, 요구 사항 분석

학습מוד의 내용 체계

학습	학습 내용	NCS 능력단위 요소	
		코드 번호	요소 명칭
	1-1. 변수 활용		
1. 기본 문법 활용하기	1-2. 연산자와 명령문 활용	2001020215_15v3.1	기본 문법 활용하기
	1-3. 사용자 정의 자료형 활용		
2. 언어 특성 활용하기	2-1. 언어 특성 활용	2001020215_15v3.2	언어 특성 활용하기
3. 라이브러리 활용하기	3-1. 라이브러리 활용	2001020215_15v3.3	라이브러리 활용하기

핵심 용어

변수, 데이터 타입, 연산자, 조건문, 반복문, 사용자 정의 자료형, 추상화, 프로그래밍 최적화, 언어 특성, 라이브러리

학습 1 기본 문법 활용하기

학습 2 언어 특성 활용하기

학습 3 라이브러리 활용하기

1-1. 변수 활용

학습 목표

- 응용소프트웨어 개발에 필요한 프로그래밍 언어의 데이터 타입을 적용하여 변수를 사용할 수 있다.

필요 지식 /

① 프로그래밍 언어 활용의 개요

프로그래밍 언어란 컴퓨터 시스템을 동작시키기 위한 프로그램 작성 언어를 말한다. 프로그램은 다소 단순해 보이는 명령어들의 조합으로 구성되는데, 이러한 조합들은 비트(Bit)라고 불리는 0과 1의 값으로 작성되거나 변환되어 컴퓨터가 이해할 수 있도록 한다.

1. 비트(Bit)

비트는 컴퓨터를 이해하기 위한 가장 기본적인 용어로, Binary Digit의 약칭이다. 0과 1로만 구성된 이진법(ex. 101001)을 이용하며, 컴퓨터 이용 시 흔히 접할 수 있는 1바이트(Byte)는 8비트(Bit)를 의미한다.

<표 1-1> 비트(Bit)와 바이트(Byte)의 관계

구분	설명
바이트	1Byte(B) = 8bit, 1bit = 0.125B
킬로바이트	1kilobyte(KB) = 8,192bit, 1bit = 0.000122KB
메가바이트	1megabyte(MB) = 8,388,608bit, 1bit = 1.1921e-7MB
기가바이트	1Gigabyte(GB) = 8.5899e+9bit, 1bit = 1.1642e-10GB
테라바이트	1Terabyte(TB) = 8.7961e+12bit, 1bit = 1.1369e-13TB
페타바이트	1Petabyte(PB) = 9.0072e+15bit, 1bit = 1.1102e-16PB
엑사바이트	1Exabyte(EB) = 9.2234e+18bit, 1bit = 1.0842e-19EB

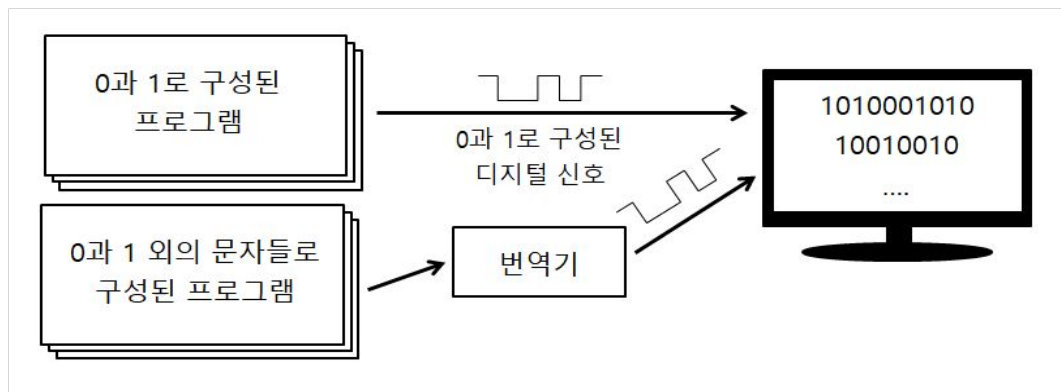
0~9까지의 10진수를 2진법으로 표현하면 [그림 1-1]과 같다.

0 ₁₀ : 0000 ₂	1 ₁₀ : 0001 ₂	2 ₁₀ : 0010 ₂	3 ₁₀ : 0011 ₂	4 ₁₀ : 0100 ₂
5 ₁₀ : 0101 ₂	6 ₁₀ : 0110 ₂	7 ₁₀ : 0111 ₂	8 ₁₀ : 1000 ₂	9 ₁₀ : 1001 ₂

[그림 1-1] 0~9에 대한 10진수의 2진수 표현

2. 컴퓨터에 명령 전달 방법

컴퓨터에 명령을 전달할 때에는 비트(Bit)로 전달이 된다. 비트의 0과 1을 컴퓨터가 이해할 수 있는 방식으로 변환하여 전달하게 되는데, 그 방법으로는 전기적 신호를 이용하거나 광 신호를 이용하는 방식 등이 있다. 예를 들어 0은 0V(볼트), 1은 5V(볼트)의 전기적 신호에 매칭시켜 컴퓨터가 이해할 수 있도록 할 수 있다. 그 밖에 불을 깜빡이는 횟수에 차이를 두어 컴퓨터가 신호를 받아들일 수 있도록 하는 방식 등이 있다.



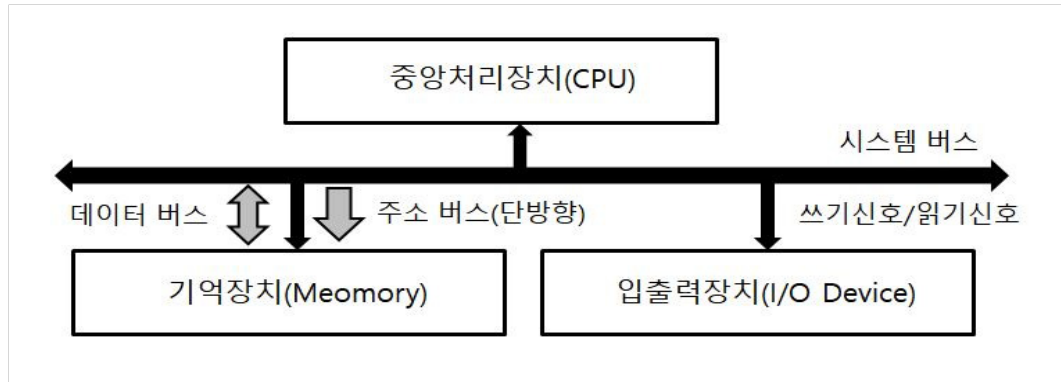
[그림 1-2] 컴퓨터에 명령 전달을 위한 신호 변환 과정

어떠한 프로그래밍 언어를 사용하느냐에 따라 비트(Bit) 변환 없이 프로그램 자체를 0과 1로만 작성할 수도 있으나, 이 경우에는 어느 정도의 한계가 있다. 따라서 보다 쉽게 작성하여 컴퓨터에 전달하고, 더욱 복잡한 프로그램을 구현하기 위해 프로그래밍 언어는 점진적으로 발전하게 되었다.

3. 컴퓨터 시스템 구조

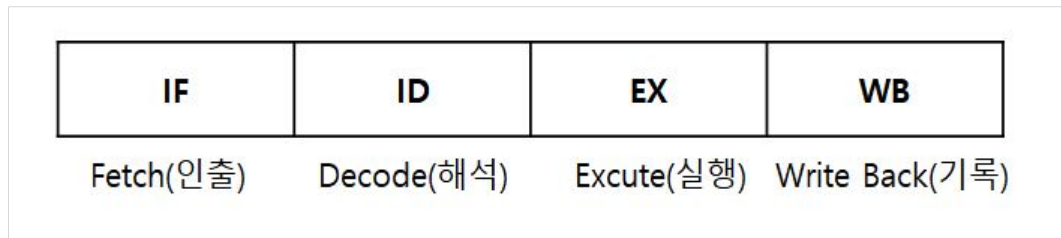
컴퓨터는 전달받은 0과 1의 값들을 정해진 순서대로 실행하며 그 과정에서 처리(Processing), 저장(Store) 등을 수행한다.

프로그램 실행 및 데이터 처리는 중앙 처리 장치(CPU)에서 수행하고, 저장은 기억 장치(Memory)에서 수행하며, 중앙 처리 장치(CPU)로부터 명령을 받아 데이터를 입력 또는 출력하는 일은 입출력 장치(I/O Device)에서 수행한다.



[그림 1-3] 컴퓨터 시스템 구조

또 중앙 처리 장치(CPU)에서는 목적에 따라 연산 등을 수행하는데, 연산을 위한 명령어가 실행되고 기록되기 위해서는 일반적으로 인출, 해석, 실행, 기록의 단계를 거쳐야 한다.

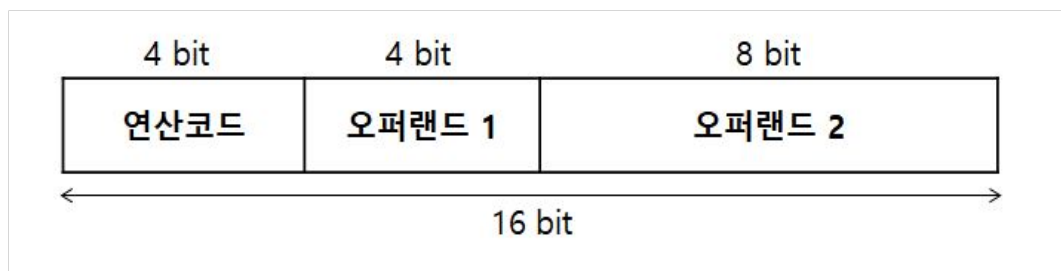


[그림 1-4] 명령어 실행 단계

4. 명령어 형식

비트(Bit)는 그 개수만큼 많은 정보를 전달할 수 있다. 용도에 따라 필드(Field)로 나눌 수 있으며, 필드의 수, 배치 방법, 각 필드에 포함되는 0과 1의 개수를 정의한 것을 명령어 형식이라고 한다.

명령어 형식에 따라 정보의 양이 달라지기 때문에 용도에 따라 적절하게 구성하여야 한다.



[그림 1-5] 명령어 형식(예시)

(1) 연산 코드: 수행될 연산을 지정(LOAD, ADD 등)

(2) 오퍼랜드: 연산을 수행하는 데 필요한 데이터 또는 데이터의 주소

5. 프로그램의 구성 요소

프로그램은 크게 자료 구조와 알고리즘으로 구성된다.

(1) 자료 구조

자료 구조는 컴퓨터에 데이터를 삽입, 삭제, 수정하게 해 주는 논리적인 공간 구조를 의미하며, 자료의 형태에 따라 단순 구조, 선형 구조, 비선형 구조, 파일 구조로 분류할 수 있다.

<표 1-2> 자료의 형태에 따른 자료 구조 분류

유형	설명
단순 구조(Simple)	프로그래밍 언어에서 제공하는 기본 데이터 타입이다. (ex. int, float, double, char 등)
선형 구조(Liner)	자료들 사이의 선후 관계가 일대일인 구조이다. (ex. 연결 자료 구조, 순차 자료 구조, 스택, 큐, 덱 등)
비선형 구조 (Non-Liner)	자료들 사이의 선후 관계가 계층 또는 그물 형태를 가지는 구조이다. (ex. 트리, 그래프 등)
파일 구조(File)	보조 기억 장치에 데이터값이 실제로 기록되는 자료 구조이다. (ex. 순차 파일, 색인 파일 등)

(2) 알고리즘

알고리즘은 넓은 의미에서 자료 구조와 함께 프로그램을 구성하는 요소를 의미하며, 좁은 의미에서는 어떤 문제에 대한 답을 찾는 해법을 의미한다. 이 알고리즘은 기본적인 명령어를 통해서도 작성 가능한데, <표 1-3>과 같은 특성을 만족하여야 한다.

<표 1-3> 알고리즘의 5가지 특성

유형	설명
입력	외부로부터 입력되는 자료가 0개 이상이어야 한다.
출력	출력되는 결과가 1개 이상이어야 한다.
명확성	각 명령어의 의미가 명확하여야 한다.
유한성	정해진 단계를 지나면 종료되어야 한다.
유효성	모든 명령은 실행이 가능한 연산들이어야 한다.

알고리즘의 표현은 자연어, 순서도, 의사 코드, 프로그래밍 언어를 이용하는 방법이 있으며, 따라서 프로그래밍 언어가 아니더라도 알고리즘의 표현은 가능하다.

② 용어 정의

프로그래밍을 위한 대표적인 용어로 변수, 바인딩, 데이터 타입 등이 있으며, 데이터 타입을 변수에 적용하기 위해서는 다음 용어에 대한 이해가 필요하다.

1. 변수: 어떤 값을 주기억 장치에 기억하기 위해서 사용하는 공간을 의미한다.
2. 식별자: 프로그램의 구성 요소를 구별하기 위한 기준으로, 변수명이 식별자에 속한다.

3. 바인딩: 변수와 변수에 관련된 속성을 연결하는 과정으로, 정적 바인딩과 동적 바인딩으로 구분된다.
 - (1) 정적 바인딩: 프로그램 실행 시간 전에 속성을 연결하는 방식이다.
 - (2) 동적 바인딩: 프로그램 실행 시간에 속성을 연결하는 방식이다.
4. 선언: 변수에 이름, 데이터 타입 등의 속성을 부여하는 작업으로, 명시적 선언과 묵시적 선언으로 구분된다.
 - (1) 명시적 선언: 선언문을 이용하여 변수 이름을 나열하고 속성을 부여하는 방식이다.
 - (2) 묵시적 선언: 별도의 선언문 없이 디폴트 규칙에 의해 속성이 부여되는 방식이다.
5. 영역: 이름이 사용되는 범위를 의미하며, 정적 영역과 동적 영역으로 구분된다.
 - (1) 정적 영역: 변수를 찾을 때 구조에 기반하는 방식이다.
 - (2) 동적 영역: 변수를 찾을 때 구조보다는 순서에 기반하는 방식이다.
6. 할당: 변수에 메모리 공간을 바인딩하는 작업을 의미한다.
7. 데이터 타입: 변수가 가질 수 있는 속성값의 길이 및 성질을 의미한다.
8. 연산자: 데이터 처리를 위해 연산을 표현하는 기호로 +, - 등과 같은 연산자를 포함한다.
9. 명령문: 프로그램을 구성하는 문장으로, 지시 사항을 처리하는 단위를 의미한다.

수행 내용 / 변수 활용하기

재료 · 자료

- 요구 사항 관련 문서
- 개발 범위 관련 문서
- 제약 사항 관련 문서
- 개발 표준 관련 문서

기기(장비 · 공구)

- 전산 장비: 인터넷, 컴퓨터, 프린터, 복사기, 빔 프로젝터
- 지원 도구: 문서 작성 도구, 문서 발표 도구, 소프트웨어 개발 도구

안전 · 유의 사항

- 계획서를 작성하여 일정에 맞게 학습 및 실습이 가능하도록 준비한다.
- 변수와 데이터 타입의 의미를 명확히 파악한 후 요구 사항을 반영한다.
- 요구 사항을 도출할 때에 해당 기능을 과포장하거나 축소하지 않도록 유의한다.
- 주관적인 판단보다는 표준 지침 등을 참고하여 작성한다.

수행 순서

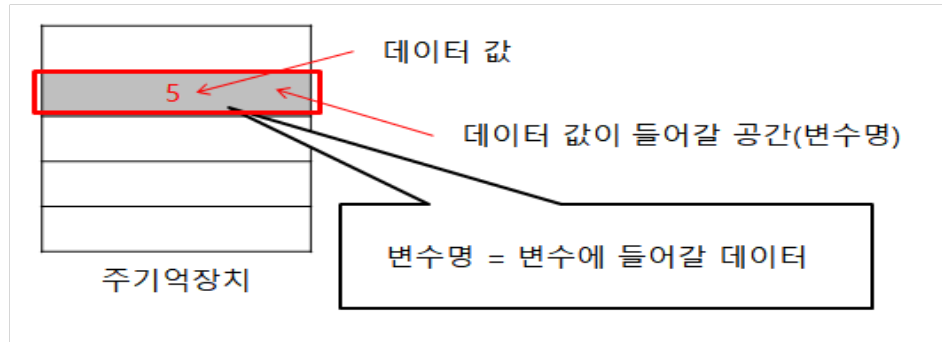
① 응용소프트웨어 개발에 필요한 프로그래밍 언어의 데이터 타입을 파악한다.

1. 프로그래밍 언어에서 사용되는 변수와 데이터 타입의 의미를 파악하고, 데이터 타입에는 어떠한 것들이 있는지 파악한다.

프로그래밍 언어 활용을 위해서는 변수와 데이터 타입에 대하여 먼저 알아야 한다. 데이터 타입의 종류에 따라 그에 적합한 변수를 사용할 수 있으므로 변수 및 데이터 타입의 의미와 데이터 타입의 유형에는 어떠한 것들이 있는지 파악한다.

(1) 변수의 의미를 파악한다.

변수는 저장하고자 하는 어떠한 값이 있을 때, 그 값을 주기억 장치에 기억하기 위한 공간을 의미한다. [그림 1-6]의 변수의 개념도를 참고하여 변수의 의미와 역할을 파악한다.



[그림 1-6] 변수의 개념도

(2) 데이터 타입의 의미를 파악한다.

데이터 타입은 변수에 들어갈 값의 특성을 구분하여 그 특성에 맞게 저장할 수 있도록 하는 데이터 속성값의 길이 및 성질을 의미한다. 사전에 정의된 데이터 타입에 맞는 값이 들어갈 수 있도록 데이터 타입이 가지는 의미를 파악한다.

[참고] 변수와 데이터 타입

변수는 어떤 값을 주기억 장치에 기억하기 위해서 사용하는 공간이고, 데이터 타입은 변수가 가질 수 있는 속성값의 길이 및 성질이다.

(3) 데이터 타입의 유형을 파악한다.

프로그래밍 언어에 따라 데이터 타입의 유형을 구분하는 기준은 차이가 있다. 하지만 일반적으로 <표 1-4>와 같이 불린 타입, 문자 타입, 문자열 타입, 정수 타입, 부동 소수점 타입, 배열 타입의 유형을 가지므로 제시된 유형을 파악한다.

<표 1-4> 데이터 타입의 유형

유형	설명
불린 타입	조건이 참인지 거짓인지 판단하고자 할 때 사용한다.
문자 타입	문자 하나(ex. 'A')를 저장하고자 할 때 사용한다.
문자열 타입	나열된 여러 개의 문자(ex. 'CAR')를 저장하고자 할 때 사용한다.
정수 타입	정숫값을 저장하고자 할 때 사용한다.
부동 소수점 타입	소수점을 포함하는 실숫값을 저장하고자 할 때 사용한다.
배열 타입	여러 데이터를 하나로 묶어서 저장하고자 할 때 사용한다.

2. 데이터 타입을 변수에 적용하기 위한 작성 규칙을 확인한다.

변수는 프로그램 소스코드의 공유, 유지 관리, 표준화 등을 위해 일정한 규칙에 따라 작성된다. 작성 규칙을 확인하여 데이터 타입을 변수에 적용할 수 있도록 사전에 준비한다.

(1) 변수 작성 규칙과 관련된 자료를 수집한다.

개발 표준 정의서, 개발자 가이드, 디자인 정의서 등 변수 작성 규칙을 확인할 수 있는 관련 자료들을 수집한다.

‘파이썬’은 <표 1-5>와 같이 변수를 설정하기 위한 규칙이 있으므로, 이를 참고하여 각 프로그래밍 언어에서 사용하는 전반적인 변수 작성 규칙을 파악한다.

<표 1-5> 파이썬의 변수 설정 규칙

구분	설명
사용 가능 문자	영문 대문자/소문자, 숫자, 밑줄('_')의 사용이 가능하다(대소문자 구분).
변수 사용 규칙	첫 자리에는 숫자를 사용할 수 없다. 변수 이름의 중간에는 공백을 사용할 수 없다. 데이터값의 의미나 역할을 표현할 수 있다.
변수 의미 부여	파이썬에서 이미 사용되고 있는 예약어의 경우에는 변수로 사용할 수 없다(ex. True, False, None, def, not, or, from, except 등).

(2) 수집된 개발 표준 관련 문서로부터 변수 작성 규칙을 확인한다.

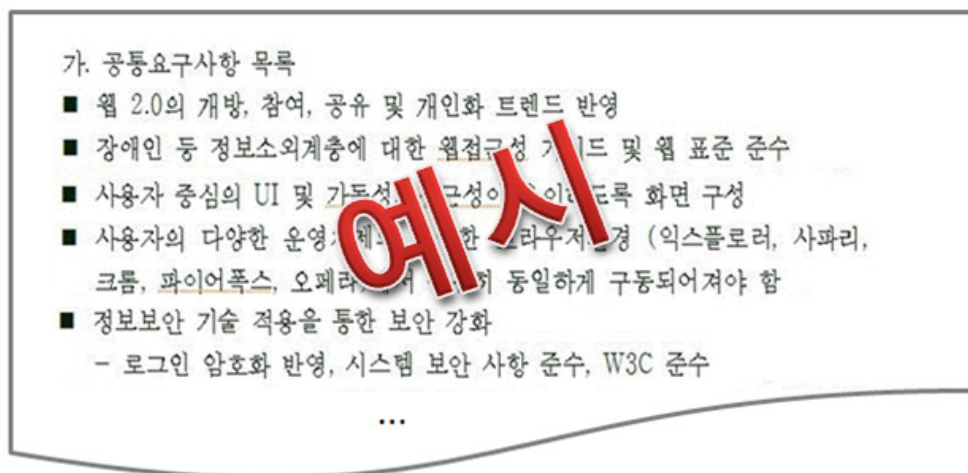
수집된 자료로부터 변수 작성 규칙을 확인하고, 만약 관련 자료가 없거나 수집된 자료에서 제공하는 정보가 부족한 경우에는 변수명이 변수값의 특징을 잘 반영할 수 있도록 변수명을 직접 설정한다.

② 데이터 타입을 적용하여 변수를 사용한다.

1. 데이터 타입의 판별을 위해 응용소프트웨어의 기능 요건을 명확히 식별한다.

(1) 요구 사항이 명시된 관련 자료를 수집한다.

제안 요청서, 제안서, 인터뷰 결과서 등 요구 사항을 확인할 수 있는 관련 자료들을 수집한다. 예를 들어 공공 정보화 사업 수행 시에는 [그림 1-7]과 같은 제안 요청서에 요구 사항이 명시되어 있으며, 이 외에 제안서, 인터뷰 결과서 등을 통해 요구 사항을 확인할 수 있다. 이를 참고하여 요구 사항 관련 자료를 수집한다.



[그림 1-7] 제안 요청서의 일반적인 공통 요구 사항 예시

(2) 수집된 자료로부터 개발하여야 할 기능 요건을 명확히 파악한다.

요구 사항 확인을 위해 수집된 자료로부터 개발하여야 하는 기능 요건을 파악한다. 이때 성능, 보안 등과 같은 비기능적인 요건들도 만족할 수 있도록 비기능 요구 사항에 대해서도 확인한다.

PER-001	
요구사항 명	시스템 평균 응답 시간
요구사항 내용	○ 데이터 입력/수정/삭제 및 통계 조회 등의 소량 데이터 처리 기능은 3초 이내 응답할 수 있도록 개발 ○ GIS 이미지 등 대량 데이터 처리 기능은 10초 이내에 응답
PER-002	
요구사항 명	시스템 오류 응답
요구사항 내용	○ 사용자 입력오류나 시스템 오류발생시 3초 이내에 오류 메시지 안내
PER-003	
요구사항 명	동시 사용자 접속 수
요구사항 내용	○ 동시 사용자 접속 시, 성능이 저하되지 않아야 함 ○ 동시 사용자 수는 500명 이상

[그림 1-8] 비기능 요건들의 예시

(3) 환경적, 기술적, 비즈니스적인 제약 사항이 있는지 확인하여 개발 범위를 확정한다.

사전에 개발 범위를 확정하였거나 협의가 완료된 경우라 하더라도 개발 시점과 환경 변화 등에 따라 제약 사항이 발생할 수 있다. 따라서 제약이 되는 환경 요소, 기술 요소, 비즈니스 요소를 파악하여 개발 범위를 확정한다.

예를 들어, 공공 정보화 사업 수행 시에는 과업 대비표라는 산출물이 있으며, 이는 제안 요청서의 요건에 대한 기술 협상 결과를 [그림 1-9]와 같이 기재하게 되므로, 그림을 참고하여 확정된 개발 범위를 명시한다.

OOO 시스템						서비스시스템명	
본서						작성일자	
요구사항ID	제안요청서		기술제안서		사업수합계획서		
	설명	페이지	설명	페이지	설명	페이지	기술협상
SFR-001	- MOOC 회원 가입 및 로그인 - 회원 가입시 SNS 연계 로그인 제공 - MOOC 강의통합검색 - 교육철학, 최근 교육이력, MOOC 수업 사례 기록 - MOOC 강의 통합기록 및 자료 관리 - 수업활동 자료 일괄 표시 (과제, 팀프로젝트, 토론 등) - MOOC 강의 학습성과 자료 - 학습성과 자료 일괄 표시 (과제, 중간/기말시험, 퀴즈, 성적 등)	13	- 회원 가입시 SNS 연계 로그인 제공 - 교육철학, 최근 교육이력, MOOC 수업 사례 기록 - 수업활동 자료 일괄 표시 (과제, 팀프로젝트, 토론 등) - 학습성과 자료 일괄 표시 (과제, 중간/기말시험, 퀴즈, 성적 등)	56	- MOOC 회원 가입 및 로그인 - 회원 가입시 SNS 연계 로그인 제공 - MOOC 강의통합검색 - 교육철학, 최근 교육이력, MOOC 수업 사례 기록 - MOOC 강의 통합기록 및 자료 관리 - 수업활동 자료 일괄 표시 (과제, 팀프로젝트, 토론 등) - MOOC 강의 학습성과 자료 - 학습성과 자료 일괄 표시 (과제, 중간/기말시험, 퀴즈, 성적 등)	58	로그인 시, SSL 적용
SFR-002	- 합원대 - 강의 일정 학습활동 열람대 표시 (주/월/일별 표시) - 강의과목 목록 - 교수자의 현재 강의과목 및 강의시간 표시 - 대시보드 가능 제공 - 현재 진행 중인 강의/과제/토론/팀프로젝트 정보 표시	13	- 합원대 - 강의 일정 학습활동 열람대 표시 (주/월/일별 표시) - 교수자의 현재 강의과목 및 강의시간 표시 - 현재 진행 중인 강의/과제/토론/팀프로젝트 정보 표시 - 현재 공지사항 목록 표시, 사용자 위젯 표시	57	- 강의 일정 학습활동 열람대 표시 (주/월/일별 표시) - 교수자의 현재 강의과목 및 강의시간 표시 - 현재 진행 중인 강의/과제/토론/팀프로젝트 정보 표시 - 현재 공지사항 목록 표시, 사용자 위젯 표시	59	
SFR-003	- 교수자가 공지사항에 대한 위젯 사항을 확인 가능 - 주요 공지사항은 목록 상단에 표시 - 공지사항 합일 기능제공 (사이즈 선택 조절)	13	- 교수자가 공지사항에 대한 위젯 사항을 확인 가능 - 주요 공지사항은 목록 상단에 표시 - 공지사항 합일 기능제공 (사이즈 선택 조절)	58	- MOOC 회원 가입 및 로그인 - 회원 가입시 SNS 연계 로그인 제공 - MOOC 강의통합검색 - 교육철학, 최근 교육이력, MOOC 수업 사례 기록	60	

...

[그림 1-9] 과업 대비표의 예시

[참고] 과업 대비표

과업 대비표는 제안 요청서 내용에 대하여 제안서, 사업 수행 계획서의 해당 페이지를 록하고 협상 결과에 따른 요구 사항 수용 여부 등을 명시한 문서를 말한다.

(4) 계획된 개발 범위 내에서 구현 가능한 응용소프트웨어의 기능을 정의한다.

확정된 개발 범위에 맞게 응용소프트웨어의 기능을 문서화한다. 예를 들어, 공공 정보화 사업 수행 시에는 요구 사항 정의서 또는 요구 사항 명세서라는 산출물을 통해 기능을 정의할 수 있으며, [그림 1-10]을 참고하여 요구 사항에 대한 내용을 기록한다.

시스템명		OOO 시스템			서브시스템명				
단계명		분석			작성일자		버전		
구분	요구사항ID	요구사항명	요구사항 설명	상세 요구사항 ID	상세 요구사항명	중요도	우선순위	제약사항	요구사항
기능	SFR-001	온라인 공개강좌 시스템 (LMS) - 공통사항	공통사항 기능요구사항을 정의	SFR-001-001	MOOC 회원 가입 및 로그인	상	중		제안요청 기술제안.
				SFR-001-002	MOOC 강의통합계획	상	중		제안요청 기술제안.
				SFR-001-003	MOOC 강의 통합기록 및 자료 관리	상	중		제안요청 기술제안.
	SFR-002	온라인 공개강좌 시스템 (교수자 기능) 요구사항	교수자 메인페이지 기능요구사항을 정의	SFR-002-01	캘린더	중	중		제안요청 기술제안서 별첨
				SFR-002-02	강의과목 목록	중	중		
				SFR-002-03	게시판관리	중	중		

...

[그림 1-10] 요구 사항 정의서의 예시

2. 기능 요건을 구현하기 위해 개발되어야 할 프로그램을 정의한다.

(1) 응용소프트웨어의 기능 요건으로부터 개발되어야 할 프로그램을 도출한다.

하나의 기능 요건이 여러 개의 프로그램으로 구성될 수도 있고, 여러 개의 기능 요건이 공통된 하나의 프로그램으로 구성될 수도 있기 때문에 기능 요건에 따라 개발되어야 할 프로그램을 도출하여 정의한다. 프로그램에 대한 정의는 [그림 1-11]을 참고하여 작성한다.

순번	업무명	프로그램 ID	프로그램명	프로그램 설명	프로세스ID
1	출장관리	PRGM-001	여비지급구분	출장관리 > 여비지급기준 구분	PROC-001
2	출장관리	PRGM-002	국내출장여비	출장관리 > 국내 출장여비 지급 기준관리	PROC-001
3	출장관리	PRGM-003	국외출장여비	출장관리 > 국외 출장여비 지급 기준관리	PROC-001
4	시간외근무	PRGM-007	근무예외자	시간외근무에 대한 예외자 관리	PROC-006
5	시간외근무	PRGM-009	근무수당	시간외근무에 대한 수당 지급 예외자 관리	PROC-007
6	교육관리	PRGM-018	교육이수 기준시간 관리	교육관리 > 연간상시학습 기준시간 관리	PROC-014
7	교육관리	PRGM-019	교육이수 인정시간 관리	교육관리 > 연간상시학습 인정시간 관리	PROC-014
...					

[그림 1-11] 프로그램 목록 예시

- (2) 프로그램이 필요로 하는 변수를 식별하여 변수 작성 규칙에 따라 지정한다.

프로그램에 포함되어야 할 변수가 어떤 유형이 있는지 식별하고, 변수 작성 규칙을 참고하여 변수명을 표준화한다.

변수명을 지정할 때에는 다음의 사항들을 고려하여 작성한다.

- (가) 변수명의 최대 길이를 확인한다.
- (나) 대소문자의 구별 여부를 확인한다.
- (다) 사용할 수 있는 문자의 집합을 확인한다.
- (라) 변수명으로 사용이 어려운 키워드를 포함하고 있지 않은지 확인한다.

[참고] 변수 적용 방법

변수는 프로그래밍 언어에서 '동일하다'라는 의미의 '='을 사용하여 데이터를 변수명에 저장한다(변수명 = 변수에 들어갈 데이터).

3. 변수가 가져야 하는 데이터 타입을 정의한다.

- (1) 변수가 가져야 하는 데이터가 참/거짓을 판별하는 값이어야 하는지 확인한다.

변수에 저장되어야 하는 값이 참(True), 거짓(False)과 같이 두 개의 값으로만 구별되는지 확인한다. 참/거짓을 판별하는 값만 가질 경우, 프로그래밍 언어에서 불린 타입(Boolean Type)으로 변수가 설정되도록 정의한다.

[참고] 불린 타입(Boolean Type)

불린 타입(Boolean Type)은 True와 False 값만을 가지며, 참/거짓을 판별할 때 사용하는 변수 타입이다.

- (2) 변수가 가져야 하는 데이터가 숫자인지, 문자인지 확인한다.

변수에 저장되어야 하는 값이 숫자인지, 문자인지 확인하여 잘못된 값이 변수에 저장되지 않도록 정의한다.

- (3) 변수가 숫자인 경우에는 정수인지, 소수점을 포함하는지 확인한다.

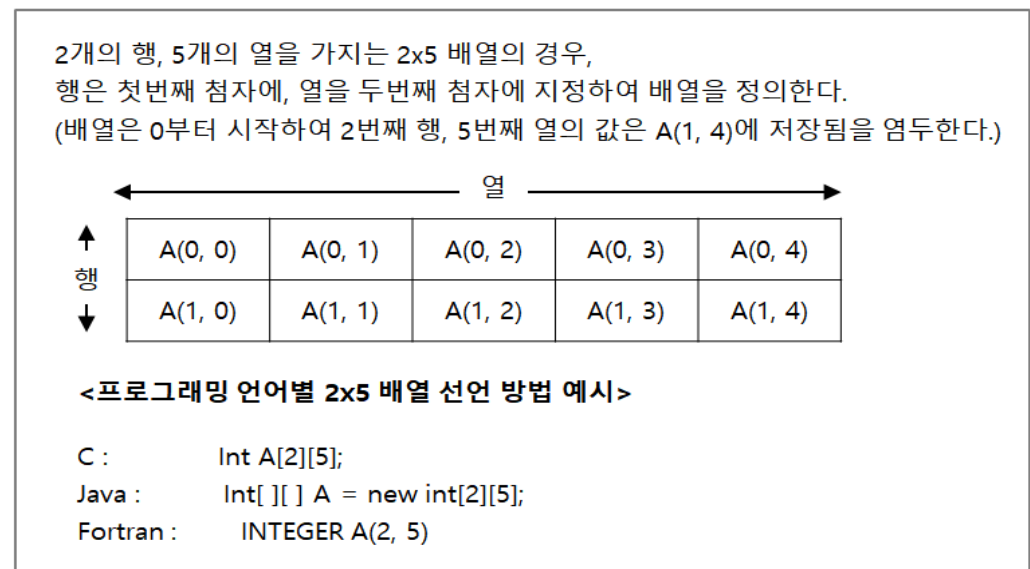
정숫값을 변수에 저장하는 경우를 정수(Integer) 타입, 소수점을 포함하는 실숫값을 저장하는 경우를 부동 소수점(Floating-point) 타입이라고 하며, 프로그래밍 언어마다 표현 방식에는 차이가 있으므로 언어별 방식을 참고하여 정의한다.

<표 1-6> Java와 C/C++의 정수 타입

구분	타입	크기	범위
Java	byte	1byte	-128~127
	short	2byte	-32,768~32,767
	int	4byte	-2,147,483,648~2,147,483,647
	long	8byte	-9,223,372,036,854,775,808 ~9,223,372,036,854,775,807
C/C++	short	2byte	-32,768~32,767
	unsigned short	2byte	0~65,535
	int	2byte(or 4byte)	-32,768~32,767 (or -2,147,483,648~2,147,483,647)
	unsigned int	2byte(or 4byte)	0~65,535 (or 0~4,294,967,295)
	long	4byte	-2,147,483,648~2,147,483,647
	unsigned long	4byte	0~4,294,967,295

(4) 동일한 데이터 타입의 데이터를 묶어서 배열 타입을 적용한다.

각각의 변수를 생성하지 않고, 동일한 데이터 타입의 데이터를 묶어서 하나의 변수에 저장하고 싶은 경우에는 배열 타입을 이용하여 저장한다.



[그림 1-12] 배열의 작성 방법

(5) 정의된 변수와 데이터 타입에 대한 설계상의 오류는 없는지 파악한다.

도출된 데이터 타입 및 변수가 주어진 요구 사항을 충분히 반영하고 있는지 검토를 수행한다. 요구 사항이 충분히 반영되지 않았거나 설계상의 오류가 발견된 경우에는 오류에 대한 수정 작업을 시행한다.

수행 tip

- 요구 사항에 따라 변수와 데이터 타입의 유형은 달라질 수 있다. 요구 사항이 명확히 식별되었는지 점검하는 절차를 가지도록 한다.

1-2. 연산자와 명령문 활용

학습 목표

- 프로그래밍 언어의 연산자와 명령문을 사용하여 애플리케이션에 필요한 기능을 정의하고 사용할 수 있다.

필요 지식 /

① 연산자

연산자는 프로그램 실행을 위해 연산을 표현하는 기호로, 프로그램 내에서는 +, -와 같은 연산자가 사용된다. 연산자에는 산술 연산자, 시프트 연산자, 관계 연산자, 논리 연산자 등이 있으며, 각 연산자는 다음과 같은 종류를 가진다.

1. 산술 연산자(Arithmetic Operator)

산술 연산자는 +, -와 같이 가장 일반적으로 사용되는 연산자이다.

<표 1-7> 산술 연산자의 종류

연산자	연산 내용
+	양쪽의 값을 더한다.
-	왼쪽 값에서 오른쪽 값을 뺀다.
*	두 개의 값을 곱한다.
/	왼쪽 값을 오른쪽 값으로 나눈다.
%	왼쪽 값을 오른쪽 값으로 나눈 나머지를 계산한다.

2. 시프트 연산자(Shift Operator)

시프트 연산자는 비트를 이동시키는 연산자이다.

<표 1-8> 시프트 연산자의 종류

연산자	연산 내용
<<	왼쪽 값을 오른쪽 값만큼 비트를 왼쪽으로 이동시킨다.
>>	왼쪽 값에 오른쪽 값만큼의 부호 비트를 채우면서 왼쪽으로 이동시킨다.

3. 관계 연산자(Relation Operator)

관계 연산자는 두 피연산자 사이의 크기를 비교하는 연산자이다.

<표 1-9> 관계 연산자의 종류

연산자	연산 내용
>	왼쪽에 있는 값이 오른쪽에 있는 값보다 크면 참(True)을 반환하고, 그렇지 않으면 거짓(False)을 반환한다.
<	왼쪽에 있는 값이 오른쪽에 있는 값보다 작으면 참(True)을 반환하고, 그렇지 않으면 거짓(False)을 반환한다.
>=	왼쪽에 있는 값이 오른쪽에 있는 값보다 크거나 같으면 참(True)을 반환하고, 그렇지 않으면 거짓(False)을 반환한다.
=<	왼쪽에 있는 값이 오른쪽에 있는 값보다 작거나 같으면 참(True)을 반환하고, 그렇지 않으면 거짓(False)을 반환한다.
==	왼쪽에 있는 값이 오른쪽에 있는 값과 같으면 참(True)을 반환하고, 그렇지 않으면 거짓(False)을 반환한다.
!=	왼쪽에 있는 값이 오른쪽에 있는 값과 다르면 참(True)을 반환하고, 그렇지 않으면 거짓(False)을 반환한다.

4. 논리 연산자(Logic Operator)

논리 연산자는 두 피연산자 사이의 논리적인 관계를 정의하는 연산자이다.

<표 1-10> 논리 연산자의 종류

연산자	연산 내용
&&	두 개의 논릿값이 모두 참이면 참(True)을 반환하고, 그렇지 않으면 거짓(False)을 반환한다.
	두 개의 논릿값 중 하나가 참이면 참(True)을 반환하고, 그렇지 않으면 거짓(False)을 반환한다.

5. 비트 연산자(Bit Operator)

비트 연산자는 0과 1의 각 자리에 대한 연산을 수행하며, 0 또는 1의 결괏값을 가진다.

<표 1-11> 비트 연산자의 종류

연산자	연산 내용
&	두 값을 비트로 연산하여 모두 참이면 참(True)을 반환하고, 그렇지 않으면 거짓(False)을 반환한다.
	두 값을 비트로 연산하여 하나가 참이면 참(True)을 반환하고, 그렇지 않으면 거짓(False)을 반환한다.
^	두 값을 비트로 연산하여 서로 다르면 참(True)을 반환하고, 그렇지 않으면 거짓(False)을 반환한다.

② 명령문

명령문은 프로그램을 구성하는 문장으로, 지시 사항을 처리하는 단위이다. 기본적인 문법들의 종류가 매우 많은 것은 아니며, 각 언어마다 유사한 문법 체계를 사용한다. 기본적인 명령어로는 조건문과 반복문이 있는데, 각각의 유형은 다음과 같다.

1. 조건문

조건문은 조건의 참, 거짓 여부에 따라 실행 경로를 달리하는 if문과 여러 경로 중에 하나를 선택하는 case문, switch문으로 구분한다.

if문의 경우, FORTRAN에서 처음 사용되었으며, FORTRAN 77을 비롯한 다른 프로그래밍 언어의 발전에 따라 대다수의 프로그래밍 언어에서 기본 명령문으로 사용하게 되었다.

<표 1-12> 조건문의 종류

종류	설명
if문	조건이 참인지 거짓인지에 따라 경로를 선택한다.
case/switch문	조건에 따라 여러 개의 선택 경로 중 하나를 취하고자 할 때 사용하며, 프로그래밍 언어에 따라 case 또는 switch를 명령문으로 사용한다.

if문의 경우에는 단순 if문, 선택 if문, 중첩 if문의 유형으로 구분할 수 있다. 단순 if문은 어떤 문장을 수행할지 여부를 결정할 때 사용하며, 선택 if문은 경로를 선택할 때 사용한다. 중첩 if문은 수행 경로가 2개가 넘어갈 때 사용하는데, else 구문을 추가하여 작성할 수 있다.

else 구문은 조건식이 거짓인 경우에 수행되는 구문으로, 프로그래밍 언어마다 차이가 있으나 else if와 같은 방식으로 분기를 추가할 수 있다.

```
if (num > 100) {  
    printf("100보다 큼니다.");  
} else if (num > 50) {  
    printf("50보다 큼니다.");  
} else if (num > 30) {  
    printf("30보다 큼니다.");  
} else {  
    printf("30보다 작거나 같습니다.");  
}
```

[그림 1-13] else를 이용한 if문 작성 방법 예시(C언어 예시)

case문은 다중 if문과는 다르다. if문은 산술 또는 논리적으로 비교가 가능하나 case는 조건이 동일한지의 여부만 확인한다.

```
switch(num) {
    case 1: printf("1입니다.");
    case 2: printf("2입니다.");
    case 3: printf("3입니다.");
    case 4: printf("4입니다.");
    default: printf("오류입니다.");
}
```

[그림 1-14] switch-case 작성 방법 예시(C언어 예시)

2. 반복문

반복문은 특정 부분을 조건이 만족할 때까지 실행하도록 하는 명령문으로 while문과 for문, do문이 있다. FORTRAN의 do문이 시초라고 볼 수 있으며, 최근에는 while문과 for문이 주로 사용된다.

<표 1-13> 반복문의 종류

종류	설명
while문	수식이 거짓이 될 때까지 해당 문장을 반복해서 실행한다.
for문	시작과 종료 조건을 지정하여 참인 동안에는 해당 문장을 반복해서 실행한다.
do문	초깃값, 최종값, 증감값을 지정하여 반복을 실행한다.

반복문을 사용할 때 특별한 조건이 없으면 무한 처리를 반복하게 된다. 이를 무한 루프라고 하며, 무한 루프를 방지하기 위해 break;와 continue;를 사용할 수 있는데, break;는 반복문을 중지하도록 하는 명령어이고, continue;는 다음 반복으로 넘어갈 수 있도록 하는 명령어이다.

```
>> while (True) :
    print ('A')
    print ('B')
    print ('C')
    break
    print ('D')
```

이 경우에는 D는 출력되지 않고, A, B, C 까지만 출력된다.

← break를 이용하여 반복을 중지한다.

[그림 1-15] break를 이용하여 반복문을 작성한 사례(파이썬 예시)

```
>> while (True) :
    print ('A')
    print ('B')
    print ('C')
    continue
    print ('D')
```

이 경우에는 A, B, C 출력 후, 다시 A, B, C 출력이 반복된다.

← continue를 이용하여 반복을 건너뛴다.

[그림 1-16] continue를 이용하여 반복문을 작성한 사례(파이썬 예시)

수행 내용 / 연산자와 명령문 활용하기

재료 · 자료

- 요구 사항 관련 문서
- 개발 범위 관련 문서
- 제약 사항 관련 문서
- 개발 표준 관련 문서

기기(장비 · 공구)

- 전산 장비: 인터넷, 컴퓨터, 프린터, 복사기, 빔 프로젝터
- 지원 도구: 문서 작성 도구, 문서 발표 도구, 소프트웨어 개발 도구

안전 · 유의 사항

- 연산자의 의미와 역할을 명확히 파악 후 적용할 수 있도록 유의한다.
- 연산자 우선순위를 파악하여 잘못된 연산이 되지 않도록 유의한다.
- 조건문과 반복문이 불필요하게 많이 포함되지 않도록 유의한다.
- 요구 사항을 명확히 파악한 후 연산자와 명령문을 적용할 수 있도록 유의한다.

수행 순서

① 프로그래밍 언어의 연산자와 명령문을 사용하여 애플리케이션에 필요한 기능을 정의하고 사용한다.

1. 프로그래밍 언어가 가지는 연산자와 명령문의 유형을 파악한다.

응용소프트웨어를 구현하기 위해서는 연산 및 명령어의 작성이 필요하므로 연산자와 명령어에는 어떠한 것들이 있는지 파악한다.

(1) 연산자의 유형을 파악한다.

프로그래밍 언어에서 사용되는 연산자의 유형 및 유형에 따른 종류에는 어떠한 것들이 있는지 파악한다.

(가) 괄호의 종류를 확인한다.

(나) 산술 연산자의 종류를 확인한다.

(다) 시프트 연산자의 종류를 확인한다.

(라) 관계 연산자의 종류를 확인한다.

- (마) 논리 연산자의 종류를 확인한다.
- (바) 비트 연산자의 종류를 확인한다.
- (사) 기타 프로그램에서 사용되는 연산자를 확인한다.

[참고] 연산자의 유형

연산자는 프로그램 실행을 위해 연산을 표현하는 기호로 크게 산술 연산자, 시프트 연산자, 관계 연산자, 논리 연산자, 비트 연산자 등의 종류로 나눌 수 있다.

연산자	연산 내용
(), []	괄호, 대괄호
++, --	증감 산술 연산자
*, /, %	곱셈, 나눗셈 산술 연산자
+, -	덧셈, 뺄셈 산술 연산자
<<, >>, >>>	시프트 연산자
>, <, >=, <=, ==, !=	관계 연산자
&&,	논리 연산자
&, , ^	비트 연산자

(2) 명령문의 유형을 파악한다.

프로그래밍 언어에서 명령문은 크게 조건문, 반복문의 유형으로 나눌 수 있다. 조건문은 if문, case문, switch문, 반복문은 for문, while문, do문의 종류를 가지고 있으므로 조건문과 반복문의 종류를 파악한다.

(가) 조건문의 종류를 파악한다.

조건문에는 if문, case문, switch문이 있으며, 조건에 따라 경로를 선택하도록 하는 역할을 수행한다. 조건문에는 어떠한 종류가 있는지 파악한다.

(나) 반복문의 유형을 파악한다.

반복문에는 for문, while문, do문이 있으며, 정해진 조건을 만족할 때까지 반복을 실행하도록 하는 역할을 수행한다. 반복문에는 어떠한 종류가 있는지 파악한다.

[참고] 명령문의 유형

명령문은 지시 사항을 처리하는 단위로, 다음과 같은 유형을 가진다.

	유형	세부 내용
조건문	if문	조건이 참인지 거짓인지에 따라 경로를 선택한다.
	case/switch문	조건에 따라 여러 개의 선택 경로 중 하나를 취하고자 할 때 사용하며, 프로그래밍 언어에 따라 case 또는 switch를 명령문으로 사용한다.
반복문	while문	수식이 거짓이 될 때까지 해당 문장을 반복해서 실행한다.
	for문	시작과 종료 조건을 지정하여 참인 동안에는 해당 문장을 반복해서 실행한다.
	do문	초깃값, 최종값, 증감값을 지정하여 반복을 실행한다.

2. 프로그램이 필요로 하는 연산 수식을 도출하여 표기법 및 연산 우선순위에 맞게 정의한다.

(1) 프로그램이 필요로 하는 연산자를 도출한다.

응용소프트웨어 프로그램의 구현을 위해 필요한 연산을 식별하고, 연산에 적용할 연산자를 도출한다.

(2) 연산 표기법을 정의한다.

연산 표기법에는 전위 표기법, 중위 표기법, 후위 표기법이 있으며, <표 1-14>를 참고하여 프로그래밍 목적에 맞게 적절한 표기법으로 정의한다.

<표 1-14> 수식을 표현하는 표기법의 종류

종류	설명	a + (b * c) 표현 예시
전위(Prefix) 표기법	연산자를 먼저 쓰고 피연산자를 뒤에 쓰는 표기법	+ a * b c
중위(Infix) 표기법	실생활에 사용되는 방식과 동일하게 쓰는 표기법	a + (b * c)
후위(Postfix) 표기법	피연산자를 먼저 쓰고 연산자를 맨 뒤에 쓰는 표기법	a b c * +

(3) 연산자의 우선순위를 참고하여 연산 수식을 완성한다.

연산 수식은 연산자, 피연산자, 괄호 등으로 구성되며, 연산자의 우선순위에 따라 연산이 수행되므로 프로그래밍 언어별 연산 우선순위를 참고하여 수식을 완성한다.

<표 1-15> Java의 연산자 우선순위

연산자	연산 내용	우선순위
(), []	괄호, 대괄호	1
!, ~, ++, --	부정, 증감 산술 연산자	2
*, /, %	곱셈, 나눗셈 산술 연산자	3
+, -	덧셈, 뺄셈 산술 연산자	4
<<, >>, >>>	시프트 연산자	5
<, <=, >, >=, ==, !=	관계 연산자	6
&, ^, , &&,	논리 연산자	7
?:	삼항 연산자	8
=, +=, -=, *=, /=, %=, <<=, >>=, &=, ^=, ~=	대입 연산자	9

<표 1-16> FORTRAN의 연산자 우선순위

연산자	연산 내용	우선순위
**	지수 연산자	1
*	곱셈 산술 연산자	2
+, -	덧셈, 뺄셈 산술 연산자	3
.LE., .EQ., .NE., .LT., .GT.	비교 연산자	4
.NOT.	부정 논리 연산자	5
.AND.	and 논리 연산자	6
.OR.	or 논리 연산자	7

<표 1-17> Ada의 연산자 우선순위

연산자	연산 내용	우선순위
*	지수 연산자	1
*, /, mod	곱셈, 나눗셈 산술 연산자	2
+, -	덧셈, 뺄셈 산술 연산자	3
=, <=, <, >	비교 연산자	4
and, or, xor	논리 연산자	5

<표 1-18> C언어의 연산자 우선순위

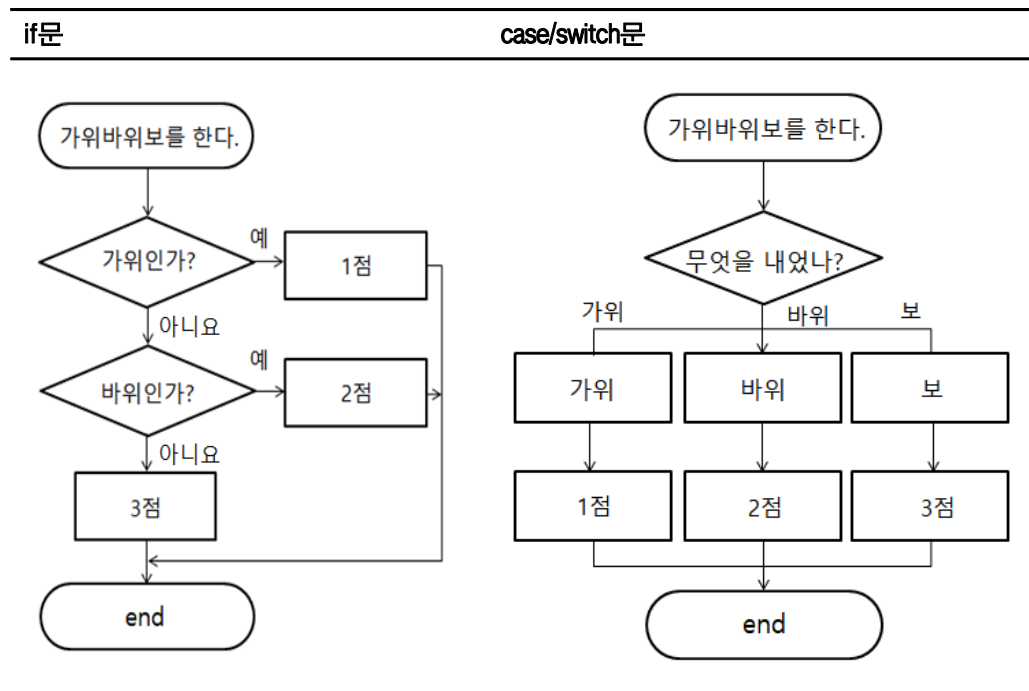
연산자	연산 내용	우선순위
(), []	괄호, 대괄호	1
!, ~, ++, --	부정, 증감 산술 연산자	2
*, /, %	곱셈, 나눗셈 산술 연산자	3
+, -	덧셈, 뺄셈 산술 연산자	4
<<, >>	시프트 연산자	5
<, <=, >, >=	관계 연산자	6
&, ^, !, &&,	논리 연산자	7
?:	삼항 연산자	8
=, +=, -=, *=, /=	대입 연산자	9

3. 조건문, 반복문을 이용하여 명령문을 작성한다.

(1) 연산 수식에 대한 제어의 필요 여부를 확인한다.

조건에 따라 경로를 선택해야 하는 분기가 필요한지 확인한다. 분기가 필요한 경우의 예시를 순서도로 살펴보면 <표 1-19>와 같다(가위 바위 보를 했을 때, 무엇을 내는지에 따라 점수를 부여하는 경우의 사례이다.).

〈표 1-19〉 분기가 필요한 경우의 순서도 예시



(2) 조건문을 적용한다.

분기가 필요한 부분에 대해 if문과 case문 등을 이용하여 조건문을 적용한다.

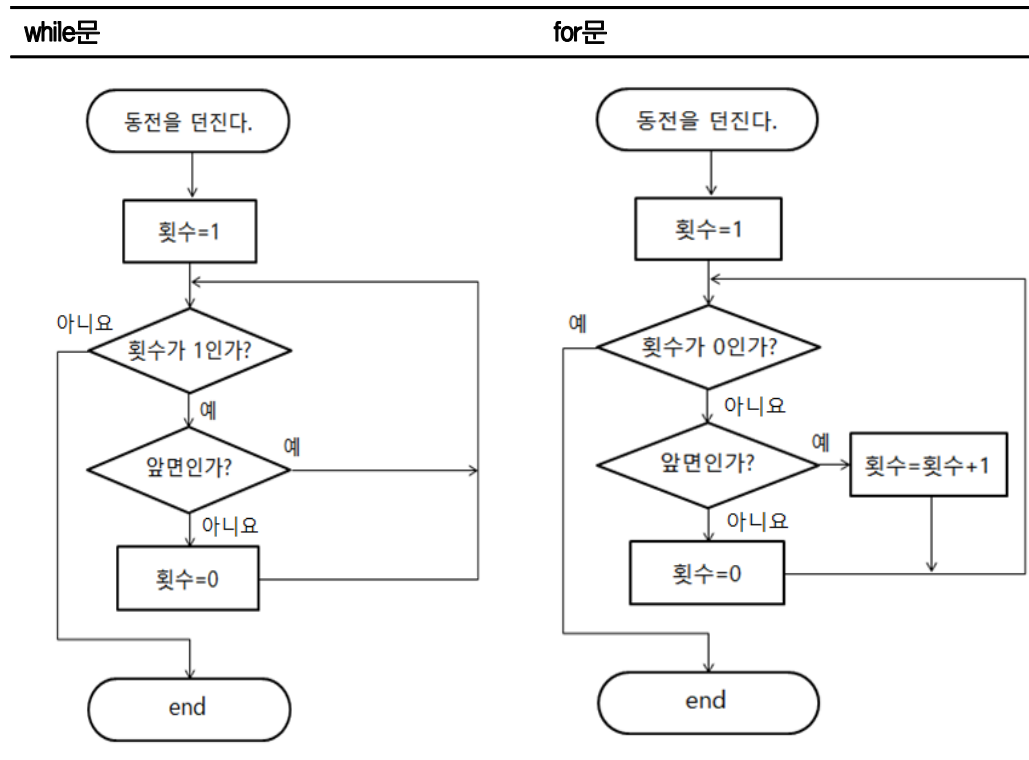
〈표 1-20〉 조건문의 작성 예시

종류	설명	C/C++/JAVA 작성 예시
if문	조건이 참인지 거짓인지에 따라 경로를 선택한다.	if (수식) { ... } else { ... }
case/switch문	조건에 따라 여러 개의 선택 경로 중 하나를 취하고자 할 때 사용하며, 프로그래밍 언어에 따라 case 또는 switch를 명령문으로 사용한다.	case (수식) of begin ... end

(3) 반복의 필요 여부를 확인한다.

연산 수식 및 분기에 대하여 반복이 필요한 부분이 있는지 확인한다. 반복이 필요한 경우의 예시를 순서도로 살펴보면 〈표 1-21〉과 같다(동전의 뒷면이 나올 때까지 계속 던지는 경우의 사례이다.).

<표 1-21> 반복이 필요한 경우의 순서도 예시



(4) 반복문을 적용한다.

반복이 필요한 부분에 대해 while문과 for문 등을 이용하여 반복문을 적용한다.

<표 1-22> 반복문의 작성 예시

종류	설명	C/C++/JAVA 작성 예시
while문	수식이 거짓이 될 때까지 해당 문장을 반복해서 실행한다.	<pre>while (수식) { ... }</pre>
for문	시작과 종료 조건을 지정하여 참인 동안에는 해당 문장을 반복해서 실행한다.	<pre>for (수식;수식;수식) ...</pre>

C/C++/JAVA의 경우에는 while문 형식에 복합문이 올 수도 있으며, do문을 이용하여 반복문장을 적어도 한 번은 수행하도록 강제할 수 있다.

[참고] break와 continue

반복문은 목적에 따라 break;와 continue;를 사용한다. break;가 반복문 문장 내에 있으면 반복문을 중지하고, continue;가 문장 내에 있으면 다음 반복 횟수로 넘어간다.

4. 적용된 조건문과 반복문이 원하는 값을 출력하는지 확인한다.

단순한 프로그램 개발 시에도 작성된 프로그램이 목적에 맞게 정확하게 동작하는지 확인이 필요하다. 조건문과 반복문 적용 후 원하는 값이 정확하게 출력되는지 확인한다.

수행 tip

- 반복문을 사용할 경우, 어떠한 반복문을 먼저 사용하는지에 따라 성능의 차이가 발생할 수 있으므로 성능을 고려하여 작성하도록 한다.

1-3. 사용자 정의 자료형 활용

학습 목표

- 프로그래밍 언어의 사용자 정의 자료형을 정의하고 애플리케이션에서 사용할 수 있다.

필요 지식 /

① 사용자 정의 자료형의 개요

C/C++, JAVA와 같은 프로그래밍 언어에서는 사용자가 직접 자료형을 만드는 것이 가능하다. 이렇게 직접 만드는 자료형을 사용자 정의 자료형이라고 하며, C++의 경우 열거체, 구조체, 공용체로 구분하여 사용자 정의 자료형의 작성이 가능하다.

1. 열거체

열거체는 괄호 안에 연속적인 값이 들어가는 자료형이다. 특정 값을 넣어 주지 않으면 1씩 늘어나기 때문에 상수 배열이라고 부르기도 한다.

```
1 enum
2 {
3     상수 멤버 1
4     상수 멤버 2
5     상수 멤버 3
6 };
```

[그림 1-17] C++을 이용한 열거체 구현 방법 예시

2. 구조체

구조체는 괄호 안에 멤버 변수를 사용하는 자료형이다. 내부에 멤버 변수 자료형을 마음대로 선언할 수 있으며, 멤버 함수의 작성도 가능하다.

```
1 struct
2 {
3     자료형 변수명 1;
4     자료형 변수명 2;
5     자료형 변수명 3;
6     반환자료형 함수명 ()
7     {
8     };
9 };
```

[그림 1-18] C++을 이용한 구조체 구현 방법 예시

3. 공용체

공용체는 구조체와 거의 유사하나 조금 더 범주가 크다고 볼 수 있다. 열거체나 구조체와 달리 공용체명을 작성하여야 하며, 구조체를 멤버로 사용할 수 있다.

```
1 union 공용체명
2 {
3     자료형 변수명 1;
4     자료형 변수명 2;
5     자료형 변수명 3;
6
7     struct
8     {
9     };
10 };
```

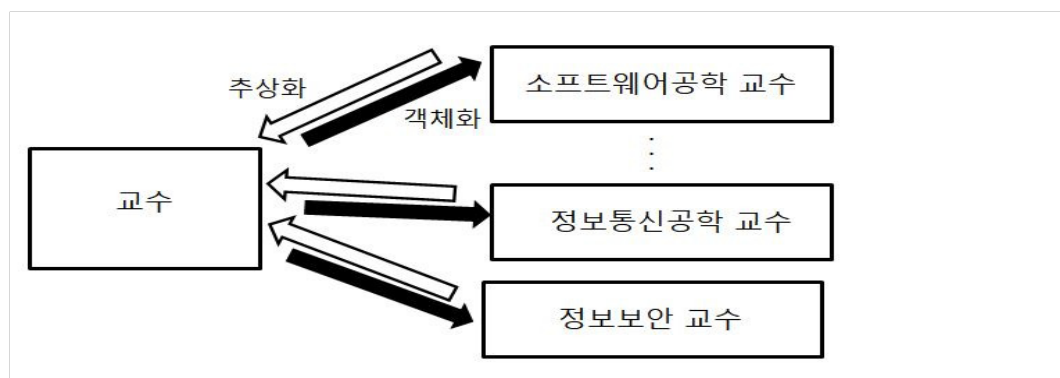
[그림 1-19] C++을 이용한 공용체 구현 방법 예시

② 추상화와 상속

사용자 정의 자료형은 추상화와 상속의 개념을 이용한다. 추상화와 상속에 대한 내용은 다음과 같다.

1. 추상화

추상화는 복잡한 문제의 본질을 이해하기 위해 세부 사항은 배제하고 중요한 부분을 중심으로 간략화하는 기법으로 기능 추상화, 자료 추상화, 제어 추상화로 구분할 수 있다.



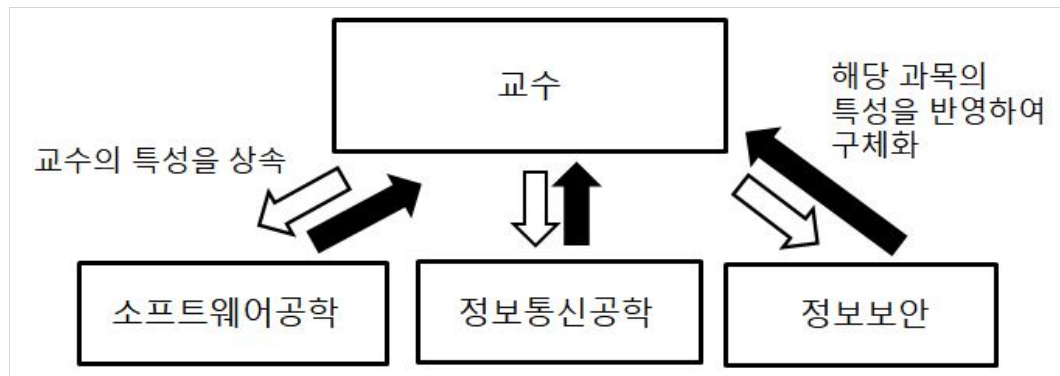
[그림 1-20] 추상화 개념도

- (1) 기능 추상화: 입력 자료를 출력 자료로 변환하는 과정을 추상화하는 방법이다.
- (2) 자료 추상화: 자료와 자료에 적용할 수 있는 오퍼레이션(Operation)을 함께 정의하는 방법이다.
- (3) 제어 추상화: 외부 이벤트에 대한 반응을 추상화하는 방법이다.

2. 상속과 구체화

상속이란, 상위 수준 그룹의 모든 특성을 하위 수준 그룹이 이어받아 재사용 또는 확장하는 특성을 의미하고, 구체화란 하위 수준 그룹이 상위 수준 그룹의 추상적인 부분을 구체화시키는 것을 의미한다.

상속은 단일 상속과 다중 상속이 있으며, 상위 수준의 그룹이 하나만 존재할 때 이를 단일 속성이라고 부른다.



[그림 1-21] 상속과 구체화 개념도

수행 내용 / 사용자 정의 자료형 활용하기

재료 · 자료

- 요구 사항 관련 문서
- 개발 범위 관련 문서
- 제약 사항 관련 문서
- 개발 표준 관련 문서

기기(장비 · 공구)

- 전산 장비: 인터넷, 컴퓨터, 프린터, 복사기, 빔 프로젝터
- 지원 도구: 문서 작성 도구, 문서 발표 도구, 소프트웨어 개발 도구

안전 · 유의 사항

- 계획서를 작성하여 일정에 맞게 학습 및 실습이 가능하도록 준비한다.
- 기본 명령문을 활용하여 특정 프로그래밍 언어에 종속되지 않도록 유의한다.
- 개인 정보가 포함된 자료를 취급할 경우에는 보안에 유의하여 이용한다.
- 주관적인 판단보다는 표준 지침 등을 참고하여 작성한다.

수행 순서

① 프로그래밍 언어의 사용자 정의 자료형을 정의하고 애플리케이션에서 사용한다.

1. 중복되는 변수와 연산, 명령문에 대하여 추상화 개념을 적용한다.

(1) 재활용이 가능한 변수와 조건문, 반복문을 식별한다.

응용소프트웨어 프로그램 구현을 위해 자주 사용되는 변수, 연산, 제어문, 반복문을 식별한다.

(2) 추상화 개념을 적용한다.

(가) 불필요한 것은 제외하고, 공통적으로 사용 가능한 큰 틀을 구상한다.

(나) 큰 틀에서 데이터가 저장되어야 하는 변수 부분과 실제 행위를 가능하게 하는 명령 부분을 분리하여 식별한다.

(다) 변수에 대한 데이터 타입을 정의하고, 상세한 부분은 다루지 않는 추상적인 형태로 구조화한다.

[참고] 추상화(Abstraction)

추상화(Abstraction)란 복잡한 문제로부터 본질을 이해하기 위하여 불필요한 세부 사항은 배제하고 중요한 부분을 중심으로 간략화하는 기법을 의미한다.

2. 변수와 연산자, 명령문 간의 연관 관계를 식별하여 상속과 구체화의 개념을 적용한다.

(1) 연관 관계를 식별한다.

(가) 추상화된 그룹 간의 관계를 식별한다.

(나) 추상화된 그룹과 추상화되지 않은 하위 그룹 간의 관계를 식별한다.

(다) 추상화되지 않은 하위 그룹 간의 관계를 식별한다.

(2) 상속과 구체화의 개념을 적용한다.

프로그래밍 언어에서 추상화된 그룹과 추상화되지 않은 하위 그룹 간에는 부모-자식의 계층적 관계가 존재하는데, 부모가 자식에게 특성을 물려주고, 자식이 구체적 기능을 구현하는 것으로써 이 개념을 변수와 명령문에 적용한다.

[참고] 상속과 구체화

상속이란 상위 레벨 그룹의 모든 특성을 하위 레벨 그룹이 이어받는 것을 의미하며, 구체화란 하위 레벨 그룹이 상위 레벨 그룹의 추상적인 부분을 구체화시키는 것을 의미한다.

3. 사용자 정의 자료형으로 정의한다.

(1) 추상화된 변수와 명령문을 묶어 하나의 형태로 정의한다.

추상화, 상속, 구체화의 개념이 적용된 변수(데이터)와 명령문(함수)을 모아 하나의 형태로 정의하는 방법에는 struct { } 또는 class { }를 이용한 방법이 있다. 이를 이용하여 사용자 정의 자료형으로 정의한다.

<표 1-23> struct { }와 class { }의 사용 예시

구분	작성 예시	예시 설명
C에서의 struct 사용 예시	<pre>struct animal { char no[10]; char name[20]; char bark[30]; ... }</pre>	struct를 이용하여 no, name, bark 등이 조합된 사용자 정의 자료형 생성
JAVA에서의 class 사용 예시	<pre>public class animal { public void bark() { ... } }</pre>	클래스를 이용하여 bark에 대한 상속 및 구체화를 구현

(2) 사용자 정의 자료형을 추가한다.

효율적인 응용소프트웨어를 구현하기 위해 지속적으로 사용자 정의 자료형을 추가한다.

수행 tip

- 효율성, 운용성, 유지 보수성이 뛰어난 응용소프트웨어를 개발하기 위해서는 사용자 정의 자료형을 적절히 이용한다.

학습 1 교수 · 학습 방법

교수 방법

- 변수의 의미와 컴퓨터 시스템의 입출력에 대한 전반적인 사항을 설명한다.
- 데이터 타입의 의미와 역할을 설명하고, 변수에 적용하기 위한 작성 규칙을 사례를 들어 설명한다.
- 변수의 식별을 위해 요구 사항에 대한 수집, 분석, 정의, 검증 방법을 설명한다.
- 데이터 타입의 각 유형에 대해 어떠한 상황에 써야 하는지 예시를 들어 설명한다.
- 프로그래밍을 위한 연산자와 명령문의 유형을 설명한다.
- 추상화 및 상속에 대한 개념을 설명하고, 사용자 정의 자료형의 예시를 들어 설명한다.

학습 방법

- 변수의 의미와 컴퓨터 시스템의 입출력에 대한 전반적인 사항을 파악한다.
- 데이터 타입의 의미와 역할을 파악하고, 변수에 적용하기 위한 작성 규칙을 실습한다.
- 요구 사항에 대한 수집, 분석, 정의, 검증 방법 및 절차를 파악한다.
- 데이터 타입의 각 유형을 어떠한 상황에 써야 하는지 파악한다.
- 연산자와 명령문의 유형을 파악한다.
- 사용자 정의 자료형의 설계를 실습한다.

학습 1 평 가

평가 준거

- 평가자는 학습자가 학습 목표를 성공적으로 달성하였는지를 평가해야 한다.
- 평가자는 다음 사항을 평가해야 한다.

학습 내용	학습 목표	성취수준		
		상	중	하
변수 활용	- 응용소프트웨어 개발에 필요한 프로그래밍 언어의 데이터 타입을 적용하여 변수를 사용할 수 있다.			
연산자와 명령문 활용	- 프로그래밍 언어의 연산자와 명령문을 사용하여 애플리케이션에 필요한 기능을 정의하고 사용할 수 있다.			
사용자 정의 자료형 활용	- 프로그래밍 언어의 사용자 정의 자료형을 정의하고 애플리케이션에서 사용할 수 있다.			

평가 방법

- 문제 해결 시나리오

학습 내용	평가 항목	성취수준		
		상	중	하
변수 활용	- 변수에 대한 데이터 타입을 도출할 수 있는 능력			
연산자와 명령문 활용	- 조건문과 반복문을 적용할 수 있는 능력			
사용자 정의 자료형 활용	- 사용자 정의 자료형을 목적에 맞게 적용할 수 있는 능력			

• 평가자 체크리스트

학습 내용	평가 항목	성취수준		
		상	중	하
변수 활용	- 요구 사항을 분석하여 변수에 데이터 타입을 적용할 수 있는 능력			
연산자와 명령문 활용	- 연산자와 명령문의 사용 목적 및 활용 방법을 파악하고 있는 능력			
사용자 정의 자료형 활용	- 사용자 정의 자료형을 정의할 수 있는 능력			

피드백

1. 문제 해결 시나리오

- ‘변수 활용’ 학습 내용에서 제시된 데이터 타입 및 변수의 적용이 가능한지 평가하고, 올바른 대처 방법과 수정이 필요한 대처 내용을 알려 준다.
- ‘연산자와 명령문 활용’ 학습 내용에서 제시된 조건문, 반복문의 작성이 가능한지 평가하고, 효과적인 작성 방법과 보완 내용을 설명한다.
- ‘사용자 정의 자료형 활용’ 학습 내용에서 제시된 사용자 정의 자료형의 설계가 가능한지 평가하고, 개선점에 대해 설명한다.
- 각 학습 내용에서 문제 해결 시나리오 결과가 ‘상’인 경우 다른 학습자와 모범 사례를 공유하고 설명할 수 있도록 안내한다.
- 각 학습 내용에서 문제 해결 시나리오 결과가 ‘중’인 경우 부족한 부분을 보완할 수 있도록 설명한다.
- 각 학습 내용에서 문제 해결 시나리오 결과가 ‘하’인 경우 모범 사례를 참조하여 다시 학습할 수 있도록 유도한다.

2. 평가자 체크리스트

- ‘변수 활용’은 프로그래밍 언어 특성을 반영하기 위한 기본적인 공통 사항이므로 용어에 대한 지식을 숙지할 수 있도록 안내한다.
- ‘연산자와 명령문 활용’은 프로그래밍 언어가 공통적으로 가지는 연산과 명령어에 대한 사항이므로 사용되는 기본적인 내용에 대해 숙지할 수 있도록 안내한다.
- ‘사용자 정의 자료형 활용’은 프로그래밍을 보다 효율적으로 개발할 수 있는 방법이므로 효과적인 설계가 가능하도록 안내한다.
- 각 학습 내용에서 평가자 체크리스트 결과가 ‘상’인 경우, 다른 학습자와 모범 사례를 공유하고 설명할 수 있도록 안내한다.
- 각 학습 내용에서 평가자 체크리스트 결과가 ‘중’인 경우 부족한 부분을 보완할 수 있도록 설명한다.
- 각 학습 내용에서 평가자 체크리스트 결과가 ‘하’인 경우 모범 사례를 참조하여 다시 학습할 수 있도록 유도한다.

학습 1	기본 문법 활용하기
학습 2	언어 특성 활용하기
학습 3	라이브러리 활용하기

2-1. 언어 특성 활용

학습 목표

- 프로그래밍 언어별 특성을 파악하고 설명할 수 있다.
- 파악된 프로그래밍 언어의 특성을 적용하여 애플리케이션을 구현할 수 있다.
- 애플리케이션을 최적화하기 위해 프로그래밍 언어의 특성을 활용할 수 있다.

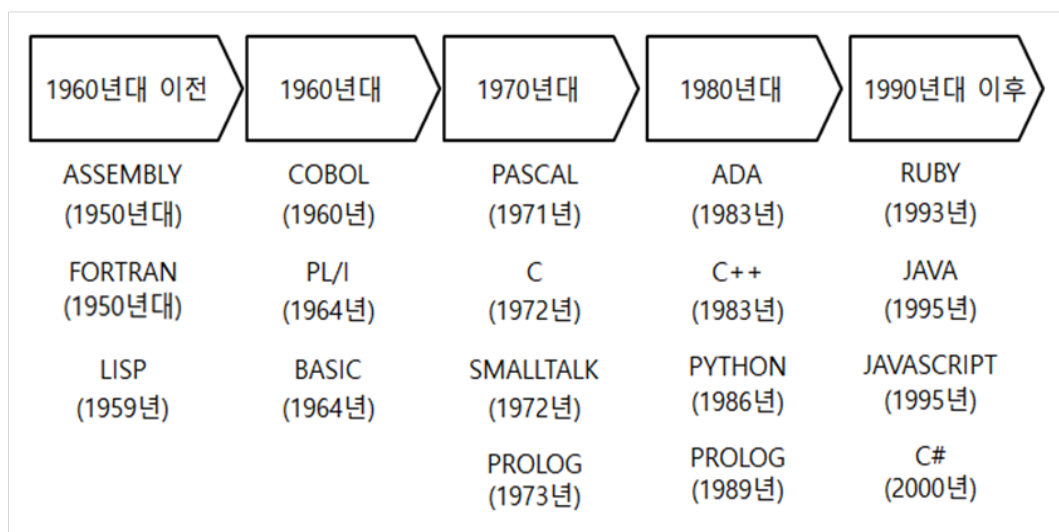
필요 지식 /

① 프로그래밍 언어의 언어별 특성

프로그래밍 언어는 컴퓨터 시스템의 역사와 함께하고 있으며 프로그래밍 언어가 개발된 시대적인 패러다임에 따라 유사한 특성을 가지고 있다. 프로그래밍 언어의 발전 과정과 각 언어별 특성은 다음과 같다.

1. 프로그래밍 언어의 발전 과정

프로그래밍 언어는 1940년대 컴퓨터의 발전과 함께 발전하였다. 세계 최초의 프로그램은 내장 방식의 프로그램으로, 이후에 현재까지 지속적으로 발전하고 있다.



[그림 2-1] 프로그래밍 언어의 발전 과정

2. 프로그래밍 언어별 특성

프로그래밍 언어는 각각의 특성들을 가지고 있으며, 주요 내용은 다음과 같다.

(1) FORTRAN

1954년에 초기 버전이 개발된 언어로 시스템 의존적이고, 프로그램 작성을 위해서는 컴퓨터 시스템 관련 지식이 많이 필요하다는 특징을 가지고 있다. 1977년 FORTRAN 77이 등장하였으나 그 이후에 등장한 언어와 비교하여 자료 구조가 제한적이었다. 재귀 함수, 구조체, 포인터 등은 제공되지 않았으며, FORTRAN 90이 등장하고 나서야 포함되었다.

(2) COBOL

1960년에 미국 국방성에 의해 개발되었다. 단순한 입출력 구현 시에도 많은 형식적인 문장이 필요하며, 비교적 프로그램 크기가 크고 구문이 복잡하다. 순차적 방식의 언어로, 웹 응용 프로그램과 쉽게 통합할 수 있다는 점이 유용하며, 자료 구조의 선언 부분과 프로그램의 실행 부분을 분리하였다는 특징이 있다. FORTRAN과 마찬가지로 대문자로만 작성된다.

(3) PASCAL

1971년 개발되어 1980년대 말까지 많이 쓰인 언어이다. 잘 짜인 구조와 간결성으로 인해 프로그래밍 언어로서 성공하였으나, 분리 컴파일과 문자열의 적절한 처리 등을 제공하지는 못하였다. 사용자 정의 추상화 기능은 제공하나 정보 은닉 기능이 없어 현대의 프로그래밍 기법을 적용하기에는 다소 부족하다는 단점을 가지고 있다.

(4) C

1972년에 개발된 언어로, UNIX 운영체제 구현에 사용되는 언어이다. 범용 언어로 개발되었으나 문법의 간결성, 효율적 실행, 효과적인 포인터 타입 제공이라는 특징으로 인해 가장 많이 사용되는 시스템 프로그래밍 언어가 되었다.

(5) C++

C 언어를 발전시킨 언어로 클래스, 상속 등을 제공하는 객체 지향 프로그래밍 언어이다. 대형 프로젝트 수행 시 모듈별 분리가 가능하여 개발과 유지 관리에 적합하다.

(6) JAVA

C++에 비해 단순하고 분산 환경 및 보안성을 지원한다. JAVA 언어는 컴파일을 거쳐야 하며, 컴파일을 통해 생성된 class 파일을 가상 머신을 통해 실행하여야 한다.

(7) JAVASCRIPT

1995년에 개발된 객체 지향 스크립트 언어로 웹 페이지 동작을 구현할 수 있다. 빠른 시간에 코드를 완성할 수 있고 확장성이 좋으며 배우기 쉽다는 장점이 있으나, 보안이나 성능이 다른 언어에 비하여 부족하다는 단점이 있다.

(8) PERL

텍스트 처리에 주안점을 두고 개발된 인터프리터 언어로 CGI용으로 많이 사용된다. 변수를 명시적으로 선언할 필요가 없고, 모든 변수가 지정되지 않았을 경우에는 기본 초깃값을 가진다.

(9) PYTHON

배우기 쉽고 이식성이 좋은 언어로 최근 트렌드와 잘 맞는다, 다양한 함수들도 많이 제공되어 스타트업과 글로벌 기업에서도 많이 사용한다. PERL 언어처럼 인터프리터 언어이면서 객체 지향 언어, 스크립트 언어이다.

(10) C#

2000년에 .NET 환경에 맞춰 설계된 언어이다. C와 C++의 발전된 형태로, VISUAL BASIC과 같이 사용자 인터페이스를 쉽게 만드는 컴포넌트 기능을 제공하기도 한다. .NET 환경에서 실행되기 때문에 .NET 환경이 설치되어야 하고, C# 컴파일러를 필요로 한다.

(11) GOLANG

2009년 Google에서 만든 언어로 짧게 GO라고도 부른다. C 언어와 직접적인 연관을 가지며, 내장 라이브러리가 많이 지원된다. 대체로 C 언어의 문법과 유사하나 if, for, switch를 포함한 제어 구조를 가지며, 하드웨어 사양이 낮더라도 빠른 컴파일이 가능하다는 특징을 가진다.

(12) DART

JAVASCRIPT와 JAVA의 영향을 받아 개발되었으며, 객체 지향적인 언어이다. 백그라운드에서 작동한다는 점에서 JAVASCRIPT와 차이를 가지고 있으며, JAVASCRIPT와 유사하나 단순화하였다. 별도의 라이브러리 설치 없이 HTML 페이지를 수정할 수 있다는 특징을 가진다.

(13) CEYLON

JAVA에 기반을 둔 언어로 모듈성을 주요 특징으로 가진다. 코드를 패키지와 모듈로 정리하여 가상 머신에서 컴파일을 수행하며, CEYLON Herd라는 저장소에서 모듈을 발행한다.

② 프로그래밍 언어의 유형 분류

프로그래밍 언어는 개발 편의성에 따라 저급 언어와 고급 언어로, 실행하는 방식에 따라 명령형, 함수형, 논리형, 객체 지향형 언어로, 구현 기법에 따라 컴파일 방식의 언어, 인터프리터 방식의 언어, 혼합형 언어로 분류할 수 있다.

1. 개발 편의성에 따른 분류

- (1) 저급 언어: 기계가 이해할 수 있도록 만들어진 언어로 기계어, 어셈블리어가 저급 언어에 속한다.
- (2) 고급 언어: 개발자가 소스코드를 작성할 때 쉽게 이해할 수 있도록 작성된 언어로 C, C++, JAVA 등이 고급 언어에 속한다.

2. 실행하는 방식에 따른 분류

- (1) 명령형 언어: 컴퓨터에 저장된 명령어들이 순차적으로 실행되는 프로그래밍 방식으로 절차형 언어라고도 불리며, FORTRAN, COBOL, PASCAL, C 등이 명령형 언어에 속한다.
- (2) 함수형 언어: 수학적 수식과 같은 함수들로 프로그램을 구성하여 호출하는 방식으로 LISP 등의 프로그래밍 언어가 함수형 언어에 속한다.
- (3) 논리형 언어: 규칙에 대한 활성화 조건이 만족되면 연관된 규칙이 실행되는 구조로, 추론과 관계 규칙에 의해 원하는 결과를 얻어 내는 방식이다. PROLOG 등이 논리형 언어에 속한다.
- (4) 객체 지향 언어: 객체 간의 메시지 통신을 이용하여 프로그래밍 하는 방식으로, JAVA와 C++ 등이 객체 지향 언어에 속한다.

3. 구현 기법에 따른 분류

- (1) 컴파일 방식의 언어: 고급 언어를 기계어로 번역하는 방식의 언어를 의미하며, FORTRAN, PASCAL, C, C++ 등이 컴파일 방식의 언어에 속한다. 컴파일 방식은 실행에 필요한 정보가 컴파일 시간에 계산되어 실행 속도가 높다는 특징을 가지고 있다.
- (2) 인터프리터 방식의 언어: 고급 언어 명령문을 하나씩 번역하고 실행하는 방식의 언어를 의미하며, BASIC, PROLOG, LISP, SNOBOL 등이 인터프리터 방식의 언어에 속한다. 프로그램 실행 시에 계산된다는 특징을 가지고 있다.
- (3) 혼합형 방식의 언어: 고급 언어를 컴파일 하여 중간 언어로 변환한 후, 인터프리터에 의해 번역을 실행하는 방식의 언어를 의미하며 JAVA가 이에 속한다.

③ 컴파일러와 인터프리터

고급 언어로 작성된 프로그램의 경우에는 컴퓨터 시스템이 바로 이해할 수 없기 때문에 번역을 필요로 하며, 컴파일러 또는 인터프리터를 이용해 번역을 수행한다.

- (1) 컴파일러(Compiler): FORTRAN, C 등과 같은 고급 언어를 기계어로 번역하는 도구로, 컴파일 방식의 언어는 모두 컴파일러를 필요로 한다. 컴파일러는 프로그램의 한 종류로 볼 수 있다.
- (2) 인터프리터(Interpreter): 프로그램 문장을 하나씩 번역하고 실행할 수 있도록 하는 프로그램이다. 컴파일 과정이 없기 때문에 개발하는 과정에서 사용하면 유용하며, 인터프리터 방식의 언어는 모두 인터프리터를 필요로 한다. 실행 속도가 느리고 메모리 사용이 비효율적이라는 단점이 있다.

④ 객체 지향 프로그래밍

프로그래밍은 크게 절차 지향 프로그래밍과 객체 지향 프로그래밍으로 구분할 수 있다. 이는 프로그래밍 언어 구조에 따라 구분한 것으로, 객체 지향 프로그래밍의 개념이 절차 지향 프로그래밍보다 뒤늦게 등장하였으나, 절차 지향 프로그래밍을 알아야 객체 지향 프로그래밍을 알 수 있는 관계는 아니다.

1. 절차 지향 프로그래밍과 객체 지향 프로그래밍

절차 지향 프로그래밍은 객체라는 개념이 등장하기 이전에 모듈, 변수, 함수를 사용하여 개발하는 방법이다. 프로그램을 순차적으로 수행시키는 방법으로 자료 구조와 명령 중심으로 프로그램을 구성한다.

객체 지향 프로그래밍의 경우에는 객체와 객체 간의 통신을 통해 프로그램이 구현된다. 객체와 속성, 메소드로 구성되는데, 절차 지향에서의 모듈은 객체에 대응되고, 변수는 속성에 대응되며, 함수는 메소드에 대응된다.

2. 객체 지향 프로그래밍의 구성 요소

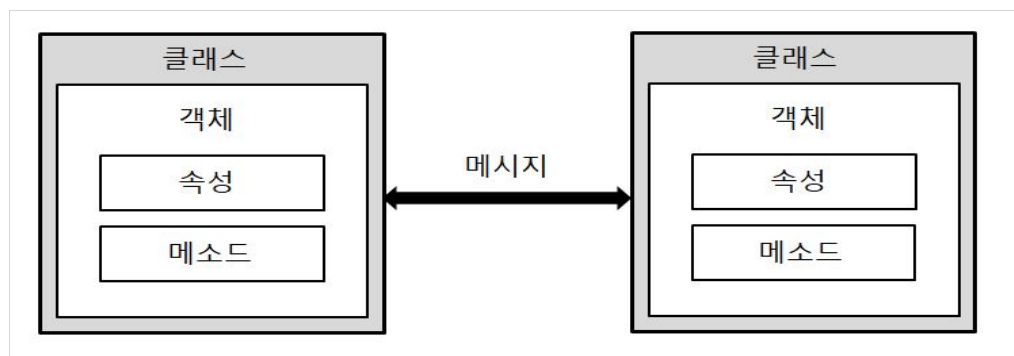
객체 지향 프로그래밍은 객체, 클래스, 메시지를 이용하여 개발하는 방식으로, 각 구성 요소에 대한 내용은 다음과 같다.

- (1) 객체(Object): 개체, 속성, 메소드로 구성된 클래스의 인스턴스를 의미한다.
- (2) 클래스(Class): 객체의 타입을 정의하고 객체를 생성하는 틀을 의미한다.
- (3) 메시지(Message): 객체 간의 통신을 의미한다.

3. 객체의 구성 요소

객체는 현실 세계에서 개체(Entity)를 데이터 속성(Attribute)과 메소드(Method)를 결합시킨 형태로 표현한 것을 의미하며, 각 구성 요소에 대한 내용은 다음과 같다.

- (1) 개체(Entity): 현실 세계에 보이는 본질을 의미한다.
- (2) 속성(Attribute): 자료 저장소 역할을 하며, 절차 지향 프로그래밍의 변수와 대응된다.
- (3) 메소드(Method): 호출 단위를 의미하며, 절차 지향 프로그래밍의 함수와 대응된다.



[그림 2-2] 객체 지향 프로그래밍과 객체의 개념도

4. 클래스(Class)

클래스는 객체 지향 프로그래밍에서 객체를 표현하는 추상 데이터 타입으로 객체를 생성하는 틀이다. 기존 언어에서 지원하던 사용자 정의 자료형 및 서브프로그램의 개념을 발전시켜 JAVA, C++ 등에서 클래스를 통해 추상화된 자료형을 제공한다.

수행 내용 / 언어 특성 활용하기

재료 · 자료

- 요구 사항 관련 문서
- 개발 표준 관련 문서
- 기능 설계 관련 문서
- 제약 사항 관련 문서

기기(장비 · 공구)

- 전산 장비: 인터넷, 컴퓨터, 프린터, 복사기, 빔 프로젝터
- 지원 도구: 문서 작성 도구, 문서 발표 도구, 소프트웨어 개발 도구

안전 · 유의 사항

- 계획서를 작성하여 일정에 맞게 학습 및 실습이 가능하도록 준비한다.
- 언어가 가지는 특성을 명확히 이해하여 반영할 수 있도록 사전에 숙지한다.
- 프로그램 개선을 진행할 때에는 이전 작업물에 대해 백업을 받을 수 있도록 한다.
- 작업물의 버전이 관리될 수 있도록 버전관리에 유의한다.

수행 순서

① 프로그래밍 언어별 특성을 파악하고 설명한다.

1. 프로그래밍 언어의 특성이 반영된 분류 체계를 파악한다.

(1) 프로그래밍 언어의 발전 과정을 파악한다.

프로그래밍 언어는 시간이 흐르면서 발전하였고, 시기에 따라 특징을 가지고 있으므로 등장한 시기의 프로그래밍 언어를 파악한다.

<표 2-1> 프로그래밍 언어의 발전 과정

시기	주요 프로그래밍 언어
1960년 이전	ASSEMBLY, FORTRAN, LISP
1960년대	COBOL, PL/I, BASIC
1970년대	PASCAL, C, SMALLTALK, PROLOG
1980년대	ADA, C++, PERL, PYTHON
1990년대	RUBY, JAVA, JAVASCRIPT, PHP, VISUAL BASIC
2000년 이후	C#, SCALA, GO, CLOJURE, CEYLON, KOTLIN, DART 등

(2) 프로그래밍 언어의 분류 체계를 파악한다.

프로그래밍 언어의 개발 편의성, 실행 방식, 구현 기법에 따른 분류 체계를 파악한다.

(가) 저급 언어와 고급 언어의 특징을 파악한다.

개발자가 소스코드를 작성할 때 얼마나 쉽게 작성할 수 있는지에 따라 저급 언어와 고급 언어로 분류될 수 있다. 개발 편의성 분류 기준에 따른 특성을 파악한다.

<표 2-2> 개발 편의성에 따른 프로그래밍 언어의 분류 체계

구분	특징
저급 언어	기계가 이해할 수 있도록 만들어진 언어이다. 추상화 수준이 낮고, 프로그램 작성이 어렵다. 다만 저급 언어에 능숙한 개발자는 매우 효율적으로 프로그램을 작성할 수 있다. 현재는 거의 사용되지 않는다.
고급 언어	개발자가 소스코드를 작성할 때 쉽게 이해할 수 있도록 작성된 언어이다. 번역하는 과정이 필요하며, 컴파일러 또는 인터프리터를 이용한다.

(나) 실행 방식에 따른 분류 체계를 파악한다.

실행 방식에 따라 명령형, 함수형, 논리형, 객체 지향형으로 구분할 수 있으며, 각 분류 기준에 대한 특징을 파악한다.

<표 2-3> 실행 방식에 따른 프로그래밍 언어의 분류 체계

구분	특징
명령형 언어	컴퓨터에 저장된 명령어들이 순차적으로 실행되는 프로그래밍 방식으로 절차형 언어라고도 한다.
함수형 언어	수학적 수식과 같은 함수들로 프로그램을 구성하여 호출하는 방식의 언어이다.
논리형 언어	규칙에 대한 활성화 조건이 만족되면 연관된 규칙이 실행되는 구조로, 추론과 관계 규칙에 의해 원하는 결과를 얻는다.
객체 지향형 언어	객체 간의 메시지 통신을 이용하여 프로그래밍하는 방식으로 JAVA와 C++이 객체 지향 언어에 속한다.

(다) 구현 기법에 따른 분류 체계를 파악한다.

구현 기법에 따라 컴파일 방식의 언어, 인터프리터 방식의 언어로 구분할 수 있으며, 각 분류 기준에 대한 특징을 파악한다.

<표 2-4> 구현 기법에 따른 프로그래밍 언어의 분류 체계

구분	특징
컴파일 방식의 언어	고급 언어를 기계어로 번역한다. (실행에 필요한 정보가 컴파일 시간에 계산되어 실행 속도가 높다.)
인터프리터 방식의 언어	고급 언어 명령문을 하나씩 번역하고 실행한다. (프로그램 실행 시 계산한다.)
혼합형 방식의 언어	고급 언어를 컴파일하여 중간 언어로 변환한 후 인터프리터에 의해 번역을 실행한다.

2. 각 분류 체계별 프로그래밍 언어의 종류와 특성을 파악한다.

각 분류 체계가 포함하는 언어의 종류를 파악하고, 각 언어가 가지는 특성을 파악한다.

(1) 분류 방식에 따른 언어 종류를 파악한다.

(가) 저급 언어와 고급 언어에 해당하는 언어의 종류를 파악한다.

개발 편의성에 따라 저급 언어와 고급 언어로 분류된 프로그래밍 언어의 종류를 파악한다.

<표 2-5> 개발 편의성에 따라 분류된 프로그래밍 언어의 종류

구분	주요 언어
저급 언어	기계어, 어셈블리어
고급 언어	FORTTRAN, C, C++, JAVA, PYTHON, .NET 등 대다수 언어

(나) 실행 방식에 따라 분류된 언어의 종류를 파악한다.

실행 방식에 따라 명령형 언어, 함수형 언어, 논리형 언어, 객체 지향형 언어로 분류된 프로그래밍 언어의 종류를 파악한다.

<표 2-6> 실행 방식에 따라 분류된 프로그래밍 언어의 종류

구분	주요 언어
명령형 언어	FORTRAN, COBOL, PASCAL, C
함수형 언어	LISP, ML, SCHEME, ERLANG
논리형 언어	PROLOG
객체 지향형 언어	JAVA, C++, SMALLTALK, PYTHON

(다) 구현 기법에 따라 분류된 언어의 종류를 파악한다.

구현 기법에 따라 컴파일 방식, 인터프리터 방식으로 분류된 프로그래밍 언어의 종류를 파악한다.

<표 2-7> 구현 기법에 따라 분류된 프로그래밍 언어의 종류

구분	주요 언어
컴파일 방식의 언어	FORTRAN, PASCAL, C, C++
인터프리터 방식의 언어	BASIC, PROLOG, LISP, SNOBOL
혼합형 방식의 언어	JAVA

(2) 각 프로그래밍 언어의 특성을 파악한다.

개발 편의성, 실행 방식, 구현 기법에 따라 분류된 분류 체계를 참고하여 각 프로그래밍 언어의 특성을 파악한다.

[참고] 컴파일 방식과 인터프리터 방식

컴파일 방식은 고급 언어를 컴퓨터에서 바로 실행이 가능한 프로그램으로 변환하는 것을 의미하며, 이러한 변환 작업을 하는 프로그램을 컴파일러(Compiler)라고 한다.

인터프리터 방식은 고급 언어를 중간 형태의 프로그램으로 먼저 변환하는 것을 의미하며, 이러한 변환 작업을 하는 프로그램을 인터프리터(Interpreter)라고 한다.

3. 적용하고자 하는 프로그래밍 언어가 가지는 개별적 특성을 파악한다.

프로그래밍 언어는 개별적인 특성을 가지고 있다. 예를 들어 파이썬의 경우에는 단순하고 이식성이 좋다는 특성이 있고, 최근 트렌드인 인공지능, 데이터 분석 등에 많이 사용되는 언어이다. 적용하고자 하는 프로그래밍 언어가 가지는 개별적인 특성을 파악한다.

4. 프로그래밍 언어의 특성을 설명한다.

파악된 프로그래밍 언어의 특성을 설명하여 잘못 파악된 부분이 없는지 점검하고, 지식을 다른 사람들과 공유한다.

(1) 프로그래밍 언어의 분류 체계를 설명한다.

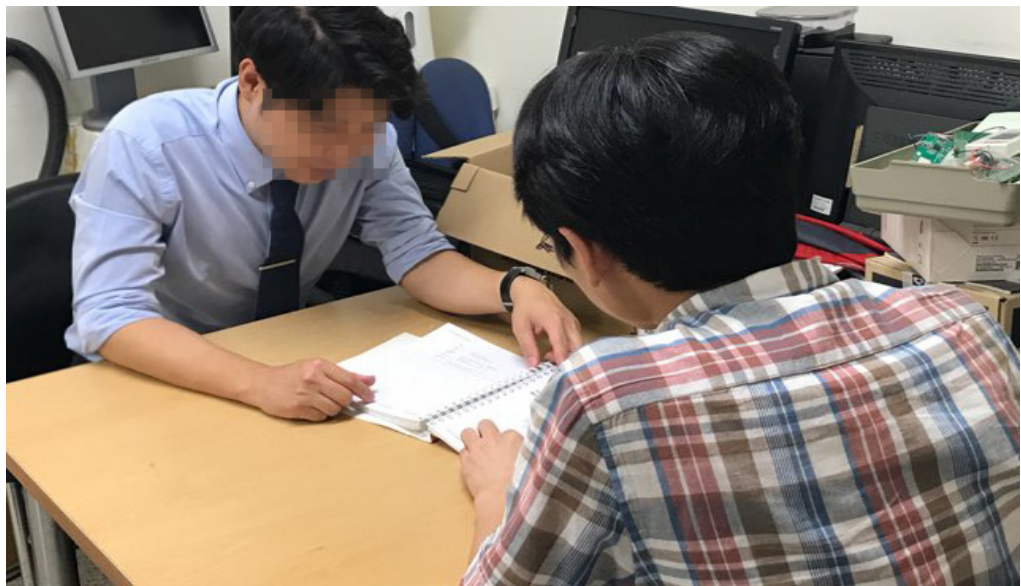
개발 편의성, 실행 방식, 구현 기법에 따른 분류 기준 및 분류 체계의 특징을 설명한다.

(2) 각 프로그래밍 언어의 특성을 설명한다.

프로그래밍 언어가 가지는 고유한 특성을 다른 사람에게 설명한다.

(3) 설명한 내용 중 잘못된 내용은 없는지 확인하고, 특성을 함께 공유함으로써 일관된 규칙을 정립한다.

설명을 통해 잘못 알고 있는 부분을 점검하고, 알고 있는 내용을 공유한다. 또 해당 특성에 따라 제약이 발생하는 부분이나 공통 설정이 필요한 부분을 검토하여 표준 체계를 가지고 갈 수 있도록 준비한다.



[그림 2-3] 프로그래밍 언어 특성을 설명하는 현장 사례

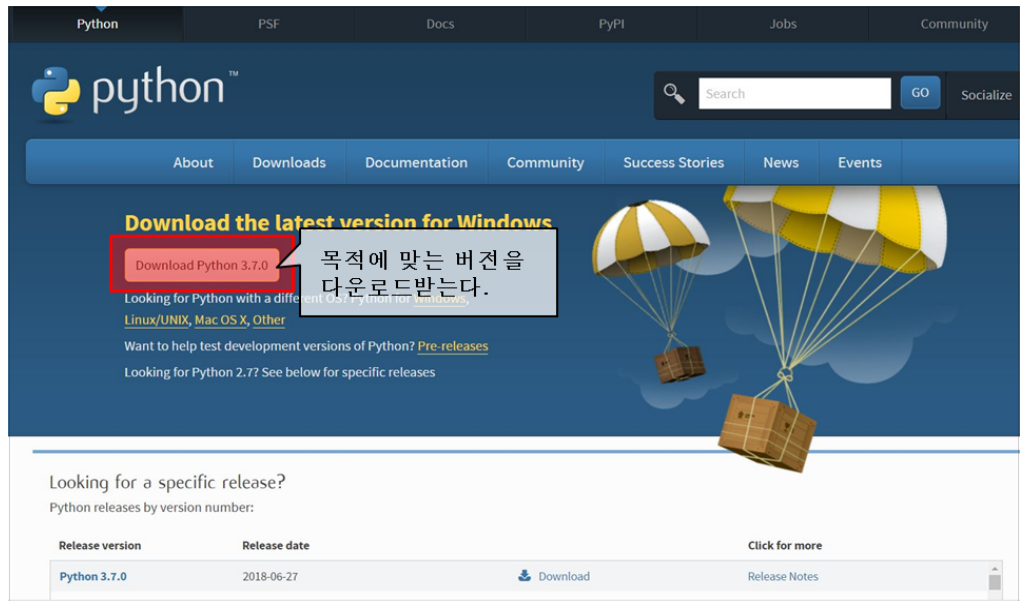
② 언어의 특성을 적용하여 애플리케이션을 구현한다.

1. 프로그래밍 언어를 이용하여 애플리케이션을 구현하기 위한 환경을 세팅한다.

프로그래밍 언어를 이용하기 위해서는 개발할 수 있는 환경이 마련되어야 하며, 일반적으로 개발환경 준비가 가능한 프로그래밍 언어 설치 파일과 개발 지원을 위한 개발 툴을 필요로 한다. 해당 프로그래밍 언어에 맞는 환경을 세팅한다.

(1) 프로그래밍 언어를 사용하기 위한 개발환경을 세팅한다.

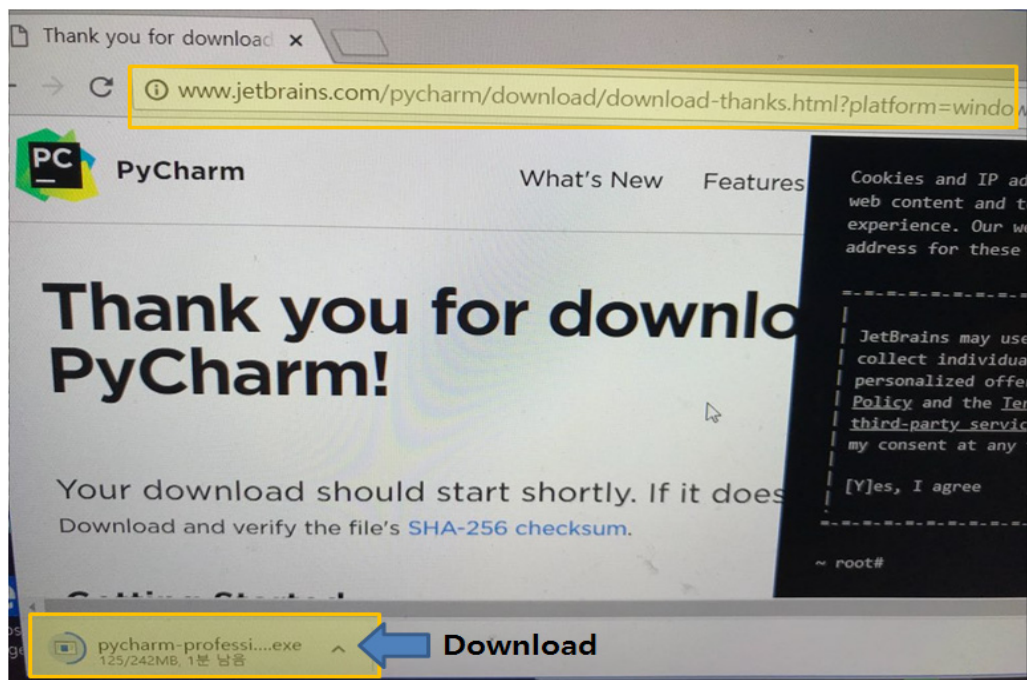
파이썬의 경우에는 컴파일러를 필요로 하며, <http://www.python.org>에서 무료 버전을 다운로드받아 설치할 수 있으므로 안전이 검증되고 목적에 부합한 버전을 다운로드 받는다.



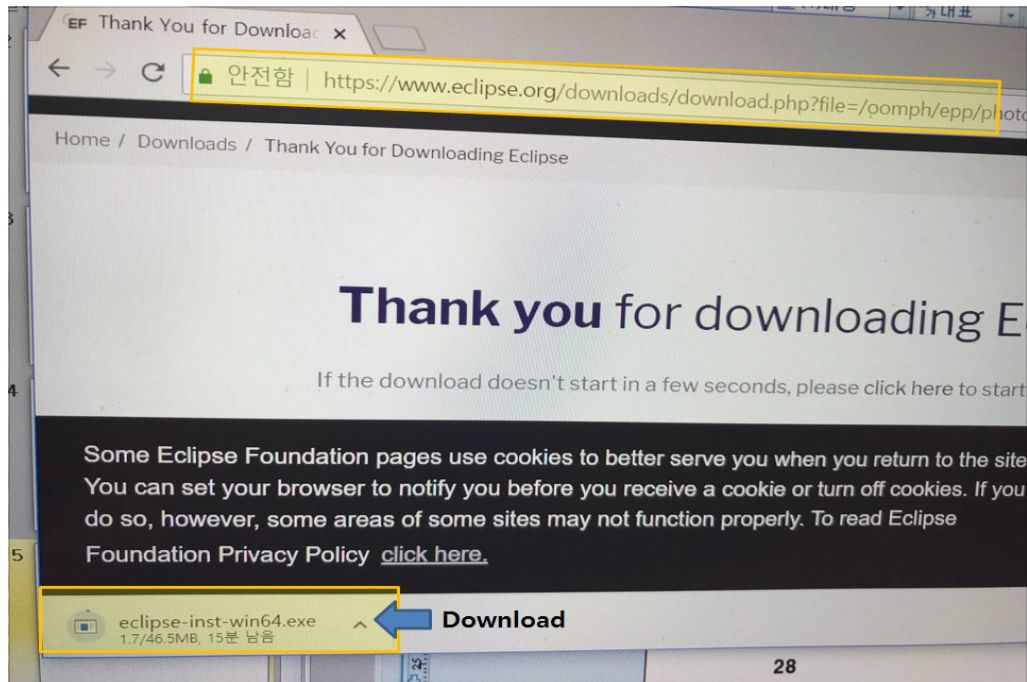
[그림 2-4] 파이썬 최신 파일 다운로드 예시

(2) 코드의 작성을 위한 개발 툴을 설치한다.

파이썬에서 주로 사용하는 통합 개발환경 프로그램은 PyCharm과 eclipse 등이 있으며, PyCharm은 <http://www.jetbrains.com/pycharm>에서, eclipse는 <https://www.eclipse.org>에서 개발자의 PC 환경에 맞는 버전을 선택하여 다운로드한 후, 설치를 진행한다.



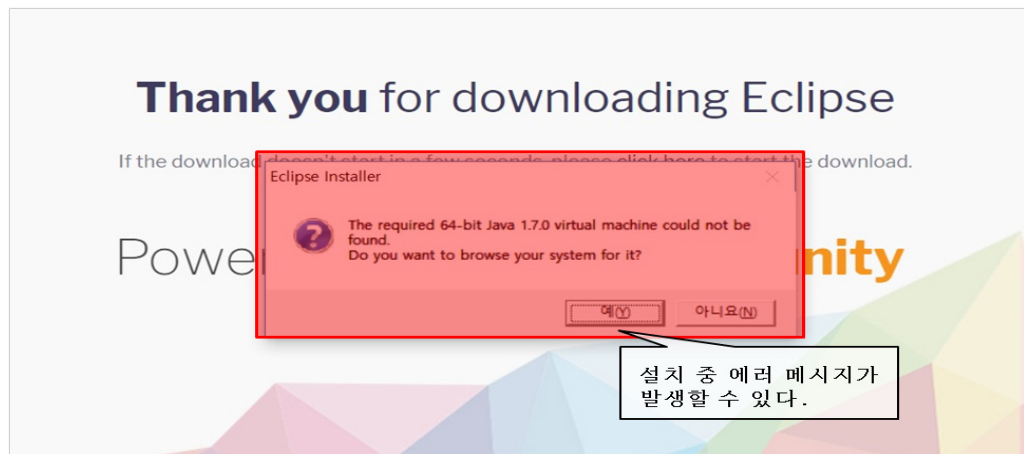
[그림 2-5] 통합 개발환경 PyCharm 설치 현장 사례



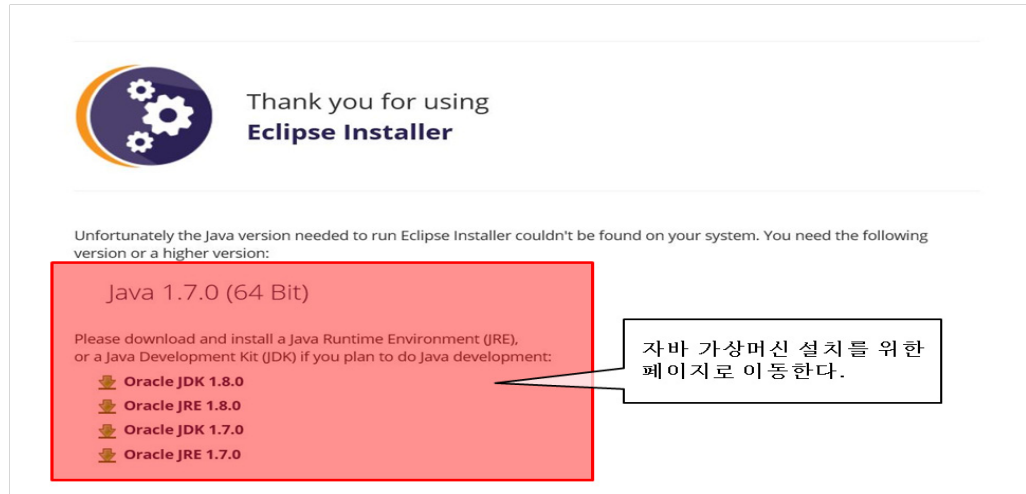
[그림 2-6] 통합 개발환경 eclipse 설치 현장 사례

(3) 자바 가상 머신이 필요한 경우 자바 가상 머신을 설치한다.

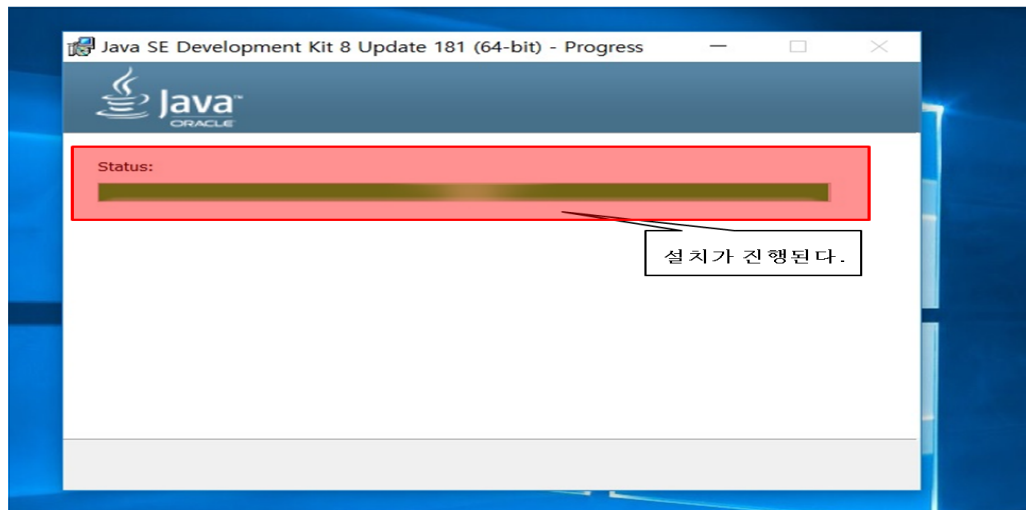
eclipse의 경우에는 자바 가상 머신이 설치되어 있지 않으면 [그림 2-7]과 같은 메시지가 발생하고, '예'를 선택하게 되면 자바 가상 머신을 설치할 수 있는 페이지로 이동하므로 PC 환경에 맞는 자바 가상 머신 파일을 다운로드받아 설치한다.



[그림 2-7] eclipse 설치 시의 에러 메시지 예시



[그림 2-8] 자바 가상 머신 설치를 위한 이동 사례



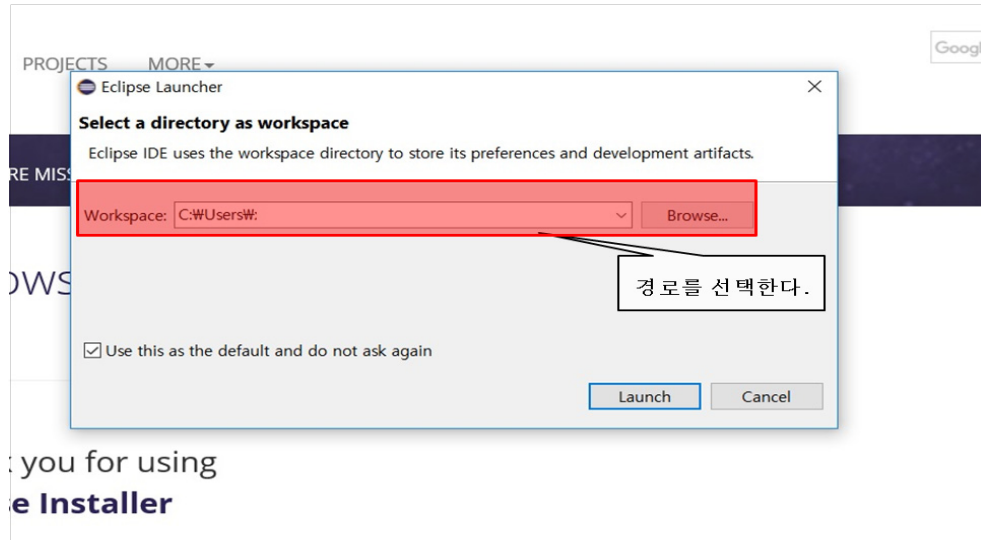
[그림 2-9] 자바 가상 머신 설치 진행 사례

(4) 도구를 사용할 수 있도록 설정을 변경한다.

파이썬에서 eclipse를 사용하고자 할 경우에는 별도의 설정이 필요하기 때문에 설정을 조정한다.

(가) Workspace의 경로를 조작한다.

Workspace는 프로그램을 구동하고자 하는 위치로, 해당 경로를 원하는 위치로 조작한다.

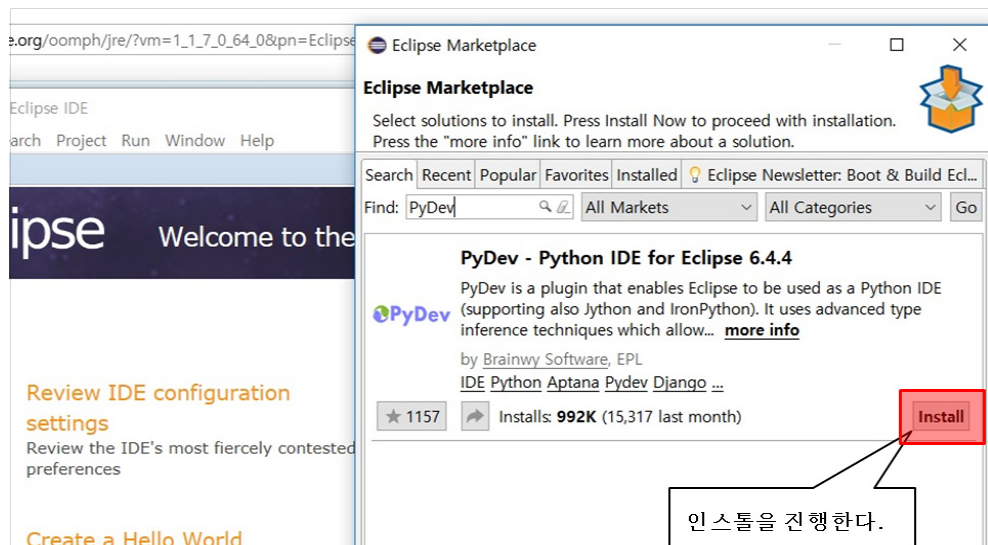


[그림 2-10] Workspace 경로 선택 사례

(나) 프로그래밍 언어를 지원할 수 있도록 인스톨을 수행한다.

eclipse의 주메뉴에서 Help > Eclipse Marketplace를 선택한 후, Find에 PyDev를 입력하여 플러그인을 검색한다.

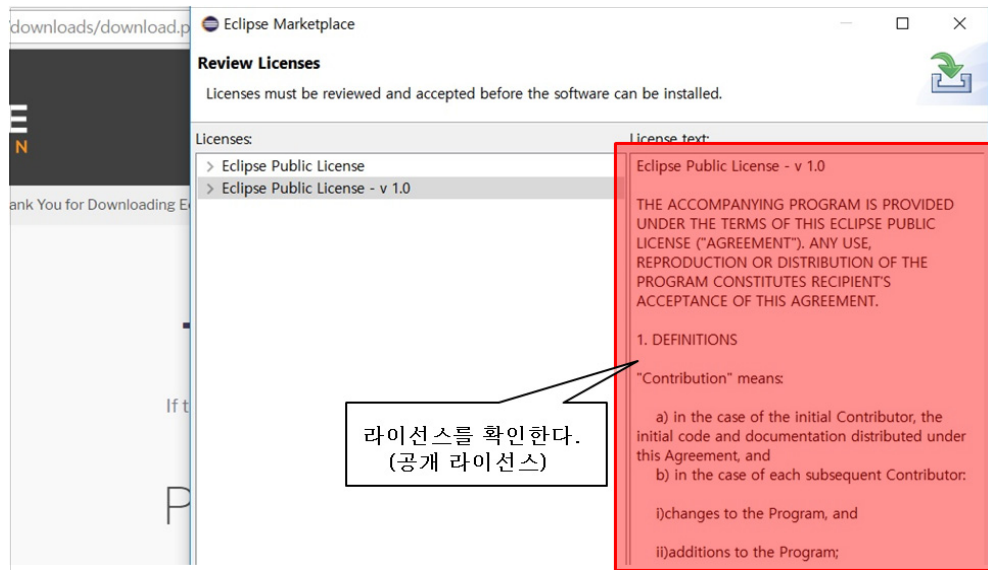
[그림 2-11]과 같이 플러그인이 검색되면 인스톨을 실행하여 파이썬을 지원할 수 있도록 설치한다.



[그림 2-11] Marketplace를 이용한 PyDev 검색 사례

(다) 라이선스에 동의하여 설치를 완료한다.

PyDev를 설치하게 되면 라이선스 동의에 대한 선택 화면이 나오고 라이선스에 동의함으로써 설치를 완료한다.



[그림 2-12] 라이선스 동의에 대한 예시

(라) 설치가 완료되었으면 개발 도구를 활용하여 프로그래밍 환경을 준비한다.

eclipse는 파이썬뿐만 아니라 자바 등의 다른 프로그래밍 언어에서도 많이 사용되고 있으며, 해당 프로그래밍 언어를 사용하기 위해서는 플러그인 설치를 통해 지원이 가능하므로, 파이썬의 사례를 참고하여 구현하고자 하는 프로그래밍 언어에 맞게 세팅한다.

2. 변수와 데이터 타입을 언어의 특성에 따라 적용한다.

변수와 데이터 타입은 프로그래밍 언어에 따라 차이가 있으므로 언어의 특성에 맞게 적용한다.

(1) 변수의 선언 방식을 확인한다.

변수의 선언 방식은 프로그래밍 언어마다 차이가 있으므로 프로그래밍 언어에 따라 어떠한 명령을 이용하여 변수를 선언하는지 파악한다.

<표 2-8> 프로그래밍 언어의 변수 선언 방식 예시

프로그래밍 언어	변수 선언 방식
JAVA	int number
C++	int number
JAVASCRIPT	var number
DELPHI	number:Integer
PASCAL	number:Integer
VISUAL BASIC	Dim number As Integer

(2) 변수명이 대소문자를 구분하는지 확인한다.

변수명의 대소문자를 구분하는 프로그래밍 언어의 경우, 대문자와 소문자를 섞어서 사용하게 되면 오류가 발생하므로 각 프로그래밍 언어가 대소문자를 구분하는지 파악한다.

(3) 데이터 타입의 유형 및 크기를 확인한다.

프로그래밍 언어가 가지는 데이터 타입의 유형을 확인하고, 해당 데이터 타입의 크기를 파악한다.

(4) 데이터 타입을 이용하여 변수를 선언한다.

프로그래밍 언어에 따른 데이터 타입을 이용하여 응용소프트웨어 개발을 위해 필요한 변수에 적용한다.

3. 언어의 특성에 따른 연산자 표기 방식 및 우선순위를 확인한다.

프로그래밍 언어에 따라 연산자 표기 방식 및 연산자의 우선순위가 다르므로 해당 프로그래밍 언어가 가지는 규칙을 파악한다.

(1) 연산자 표기 방식을 확인한다.

해당 프로그래밍 언어가 가지는 연산자 표기 방식을 파악한다.

(2) 연산자 우선순위를 확인한다.

해당 프로그래밍 언어가 가지는 연산자의 우선순위 규칙을 파악한다.

4. 명령문의 적용 방법을 확인한다.

프로그래밍 언어에 따라 명령문의 구조가 다르므로 해당 프로그래밍 언어가 가지는 명령문 구조를 파악한다.

(1) 조건문의 적용 방법을 확인한다.

해당 프로그래밍 언어에서 사용하는 조건문 작성 방법을 파악한다. 파이썬의 경우에는 [그림 2-13]과 같은 명령어 구조를 이용하여 조건문을 구현할 수 있으므로 그림을 참고하여 작성한다.

```
If x < 100 :
```

```
    print (x)
```

해설) 만약 x가 100보다 작으면 x를 출력하라.

[그림 2-13] 조건문을 이용한 구현 예시(파이썬 예시)

if를 사용할 때에는 주의하여야 할 점이 있다. 조건을 제시하는 순서에 따라 처리 순서가 바뀌기 때문인데, 파이썬의 경우에는 elif를 사용하여 조건문을 계속 이어 갈 수 있으므로 [그림 2-14]를 참고하여 작성한다.

```

if ( x > 80 ) :
    print ('x는 80보다 큼니다.')
elif ( x > 70 ) :
    print ('x는 70보다 큼니다.')
elif ( x > 60 ) :
    print ('x는 60보다 큼니다.')
elif ( x > 50 ) :
    print ('x는 50보다 큼니다.')
else :
    print ('x는 50보다 작거나 같습니다.')

```

[그림 2-14] 여러 차례 조건문을 사용하여 구현한 예시(파이썬 예시)

(2) 반복문의 적용 방법을 확인한다.

해당 프로그래밍 언어에서 사용하는 반복문 작성 방법을 파악한다. 파이썬의 경우에는 for문과 while문을 이용할 수 있는데, for문의 경우에는 range() 함수를 이용하여 반복을 수행할 수 있다, range 함수 내에 반복하고 싶은 횟수를 입력하여 작성한다.

```

for x in range(3) :
    print (x)

```

해설) 0, 1, 2로 이루어진 순서형 자료를 출력하라.

[그림 2-15] for문을 이용한 구현 예시(파이썬 예시)

while은 ‘~하는 동안’을 의미하며, 조건이 참인 경우에는 계속 반복하도록 하는 명령문이다. [그림 2-16]은 0부터 시작하여 반복이 수행될 때마다 1씩 더해 0, 1, 2를 출력하는 명령문으로 그림을 참고하여 while문을 작성한다.

```

>> x = 0
>> while ( x < 3 ) :
    print (x)
    x = x +1

```

[그림 2-16] while문을 이용한 구현 예시(파이썬 예시)

5. 변수, 데이터 타입, 연산자, 명령문을 이용하여 애플리케이션을 구현한다.

파악된 변수의 특성, 데이터 타입, 연산자, 명령문을 이용하여 프로그래밍 언어의 특성에 맞게 애플리케이션을 구현한다.

6. 문법에 오류는 없는지 코드를 검토하고 디버깅을 수행한다.

작성된 코드가 문법에 어긋나는 사례는 없는지 테스트를 통해 점검하고 디버깅을 수행한다.

③ 프로그래밍 언어의 특성을 활용하여 최적화를 수행한다.

1. 프로그래밍 언어가 가지는 패턴을 확인하여 적용한다.

프로그래밍 언어는 애플리케이션 최적화를 위해 지속적으로 발전하였으며, 그동안 문제가 되었던 부분들을 패턴화 하여 해결하고자 하였다. 사용하고자 하는 프로그래밍 언어를 최적화하기 위해 어떠한 패턴이 있는지 확인하여 적용한다.

(1) 프로그래밍 언어가 가지는 패턴을 파악한다.

상속과 구체화 등의 프로그래밍 특징을 이용한 패턴에 어떠한 것들이 있는지 파악한다. 파이썬의 경우에는 객체 지향 프로그래밍에서 주로 사용되는 디자인 패턴을 통해 최적화할 수 있으므로 <표 2-9>를 참고하여 패턴의 유형을 숙지한다.

<표 2-9> 객체 지향 프로그래밍에서 사용하는 주요 디자인 패턴의 예시

디자인 패턴	설명
싱글톤 패턴	오직 하나의 인스턴스만을 가지도록 하는 패턴
팩토리 메소드 패턴	어떤 인스턴스를 생성할지는 서브클래스에서 정의하도록 하는 패턴
옵서버 패턴	하나의 오브젝트 상태가 변할 때 관련자에게 알리고 자동 업데이트를 수행하는 패턴
퍼사드 패턴	하나의 인터페이스로 제공하는 패턴
프락시 패턴	객체의 대리자를 이용하여 다른 객체로의 접근을 통제하는 패턴
커맨드 패턴	요청 자체를 캡슐화하여 파라미터를 전달하는 패턴
템플릿 메소드 패턴	템플릿을 정의하고 구체화는 서브클래스에서 정의하는 패턴

[참고] 디자인 패턴(Design Pattern)

어떤 분야에서 반복적으로 나타나는 문제점들에 대해 전문가들의 경험을 정리하여 해결 방안을 제시한 패턴을 디자인 패턴(Design Pattern)이라고 부른다. 이 디자인 패턴을 참고하여 개발할 경우 개발의 효율성과 유지 보수성, 운용성 등의 품질이 높아지며, 프로그램의 최적화를 돕는다.

(2) 프로그래밍 언어의 특성을 활용하여 패턴을 적용한다.

프로그래밍 언어의 특성에 맞게 최적화를 위한 패턴을 적용한다. 파이썬의 경우에는 싱글톤 패턴, 팩토리 메소드 패턴, 옵서버 패턴 등이 주로 이용된다.

(가) 싱글톤 패턴을 적용한다.

싱글톤 패턴은 오직 하나의 인스턴스만을 가지도록 하는 패턴으로, 파이썬의 경우에는 기본적으로 싱글톤 패턴으로 구성되므로 [그림 2-17]을 참고하여 적용한다.

```
class DB(object) :
    def __new__(cls) :
        if not hasattr(cls, 'instance') :
            cls.instance = super(DB, cls).__new__(cls)
        return cls.instance
```

[그림 2-17] 싱글톤 패턴 소스코드 예시(파이썬 예시)

(나) 팩토리 메소드 패턴을 적용한다.

어떤 인스턴스를 생성할지 서브클래스에서 정의하도록 할 경우에는 팩토리 메소드 패턴을 적용한다.

```
import abc """ 추상클래스 구현을 위한 라이브러리 """

class DBFactory(object) :
    __metaclass__ = abc.ABCMeta

class DBFactoryImp1(imp) :
    return '100'

class DBFactoryImp2(imp) :
    return '200'
```

[그림 2-18] 팩토리 메소드 패턴 소스코드 예시(파이썬 예시)

(다) 옵서버 패턴을 적용한다.

1:M 관계의 오브젝트에 대해 감시하고 있다가 객체 상태 변화에 따라 알리고 (Notify), 자동으로 수정(Update)하고자 할 때에는 [그림 2-19]를 참고하여 옵서버 패턴을 적용한다.

```
class DBObserver(object) :
    def __init__(self) :
        ...
    def attach_observer(self, observer) :
        ...
    def remove_observer(self, observer) :
        ...
    def notify_observer(self, observer) :
        ...
```

[그림 2-19] 옵서버 패턴 소스코드 예시(파이썬 예시)

(라) 퍼사드 패턴을 적용한다.

단순화된 하나의 인터페이스로 제공하고자 하는 경우에는 퍼사드 패턴을 적용한다.

(마) 프락시 패턴을 적용한다.

객체의 대리자를 이용하여 다른 객체로의 접근을 통제하고자 하는 경우에는 프락시 패턴을 적용한다.

(바) 커맨드 패턴을 적용한다.

요청 자체를 캡슐화 하여 파라미터를 전달하고자 하는 경우에는 커맨드 패턴을 적용한다.

(사) 템플릿 메소드 패턴을 적용한다.

템플릿을 정의하고 구체화는 서브클래스에서 정의하고자 하는 경우에는 템플릿 메소드 패턴을 적용한다.

(아) 기타 응용소프트웨어 구현을 위해 사용 가능한 패턴을 적용한다.

기타 패턴을 적용하여 소스코드를 최적화한다.

[참고] GoF 디자인 패턴 예시

객체 지향 프로그래밍의 디자인 패턴 중 가장 유명한 디자인 패턴은 GoF의 23가지 디자인 패턴이 있으며, 생성 패턴과 구조 패턴, 행위 패턴으로 구분하여 패턴을 설명한다.

1. 생성 패턴

Factory Method: 서브클래스가 인스턴스를 결정하도록 하고 책임을 위임하는 패턴
Abstract Method: 생성군들을 하나에 모아놓고 팩토리 중에서 선택하게 하는 패턴
Builder: 생산 단계를 캡슐화 하여 구축 공정을 동일하게 이용하도록 하는 패턴
Prototype: 복사하여 새 개체를 생성할 수 있도록 하는 패턴
Singleton: 유일한 하나의 인스턴스를 보장하도록 하는 패턴

2. 구조 패턴

Adapter: 인터페이스로 인해 함께 사용하지 못하는 클래스를 함께 사용하도록 하는 패턴
Bridge: 추상과 구현을 분리하여 결합도를 낮춘 패턴
Composite: 개별 객체와 복합 객체를 클라이언트에서 동일하게 사용하도록 하는 패턴
Decorator: 소스를 변경하지 않고 기능을 확장하도록 하는 패턴
Facade: 하나의 인터페이스를 통해 느슨한 결합을 제공하는 패턴
Flyweight: 대량의 작은 객체들을 공유하는 패턴
Proxy: 대리인이 대신 그 일을 처리하는 패턴

3. 행위 패턴

Interpreter: 언어 규칙 클래스를 이용하는 패턴
Template: 알고리즘 골격의 구조를 정의한 패턴
Chain of Responsibility: 객체들끼리 연결 고리를 만들어 내부적으로 전달하는 패턴
Command: 요청 자체를 캡슐화 하여 파라미터로 넘기는 패턴
Iterator: 내부 표현은 보여주지 않고 순회하는 패턴
Mediator: 객체 간 상호 작용을 캡슐화한 패턴
Memento: 상태값을 미리 저장해 두었다가 복구하는 패턴
Observer: 상태가 변할 때 의존자들에게 알리고, 자동 업데이트하는 패턴
State: 객체 내부 상태에 따라서 행위를 변경하는 패턴
Strategy: 다양한 알고리즘을 캡슐화하여 알고리즘 대체가 가능하도록 한 패턴
Visitor: 오퍼레이션을 별도의 클래스에 새롭게 정의한 패턴

2. 프로그래밍 언어 특성에 따라 발생할 수 있는 보안 약점을 확인하고 최적화한다.

안전한 응용 프로그램 개발을 위해 개발 보안(Secure Coding) 가이드를 참고하거나 잘 알려진 취약점을 분석하여 보안 위협에 대비한다.

(1) 데이터 처리 측면의 보안 약점을 확인하고 최적화한다.

(가) SQL 삽입 공격에 대한 보안 약점을 제거한다.

외부 입력값에 “ OR '0'='0”와 같은 문자열이 전달되면, 모든 조건이 참이 되어 공격자가 원하는 공격을 수행할 수 있으므로, 입력값의 유효성을 검증하는 구문을 추가한다.

(나) 경로 조작 및 자원 삽입 공격에 대한 보안 약점을 제거한다.

파일 이름 등이 식별자로 사용되지 않도록 하고, 다른 경로의 파일을 접근할 수 없도록 위험 문자열(ex. “, /, \)은 제거한다.

(다) 입력 시 위협할 수 있는 특정 문자열은 replace를 이용하여 치환한다.

(라) 파일 업로드 시 파일 타입과 크기를 제한한다.

exe 파일과 같은 형식의 실행 파일은 보안 위협이 될 수 있으므로 확장자를 검증 후 업로드가 가능하도록 개선한다.

(마) 신뢰되지 않은 URL 주소로 자동 연결되지 않도록 제한한다.

(바) 동적 메모리 할당을 위한 크기를 사용하는 경우 음수가 아닌지 검사하는 구문을 추가한다.

(2) 보안 기능 측면의 보안 약점을 확인하고 최적화한다.

(가) 적절한 인증 없이 중요한 기능에 접근할 수 있지는 않은지 확인한다.

(나) 적절한 인가 없이 권한이 부여되지 않는지 확인한다.

(다) 중요 정보를 포함한 주석은 삭제한다.

(라) 하드코딩된 비밀번호는 삭제한다.

(3) 잘못된 시간 및 상태에 따라 발생할 수 있는 보안 약점을 확인하고 최적화한다.

(가) 여러 개의 프로세스가 동작할 때 각 프로세스의 시작 시간과 종료 시간을 명확히 한다.

(나) 종료되지 않는 반복문이 있는 경우 종료 구문을 추가한다.

(4) 에러 처리, 코드 오류 등에서 발생할 수 있는 보안 약점을 확인하고 최적화한다.

(가) 오류 메시지를 통해 중요 정보가 노출되지 않는지 확인한다.

(나) 예외 처리가 적절하게 수행되었는지 확인한다.

(다) 자원 해제가 적절하게 수행되었는지 확인한다.

해제된 자원을 참조하는 경우 예기치 않은 오류가 발생할 수 있으므로 자원 해제가 실행되는지 확인하여 개선한다.

(라) 변수를 초기화하여 사용하고 있는지 확인한다.

변수를 초기화하지 않고 사용하는 경우에는 예기치 않은 오류가 발생할 수 있으므로 변수를 초기화할 수 있도록 개선한다.

(마) 취약한 API를 사용하고 있지는 않은지 확인한다.

[참고] 소프트웨어 보안 약점

‘행정 기관 및 공공 기관 정보 시스템 구축·운영 지침 별표 3’에서는 개발 보안 (Secure Coding) 진행 시 참고할 수 있는 보안 약점을 제시하고 있다.

구분	설명
입력 데이터 검증 및 표현	입력값에 대하여 검증이 누락되거나 데이터 타입이 잘못 지정되어 있을 때 발생할 수 있는 보안 약점이다.
보안 기능	보안 기능(인증, 접근 제어 등)이 적절하지 않은 경우 발생할 수 있는 보안 약점이다.
시간 및 상태	여러 개의 프로세스가 동작할 때 시간 및 상태가 적절하게 관리되지 않아 발생할 수 있는 보안 약점이다.
에러 처리	에러의 처리가 미흡하거나 중요 정보가 노출될 때 발생할 수 있는 보안 약점이다.
코드 오류	프로그래밍 시 잘못된 코드 작성을 통해 오류가 발생하는 보안 약점이다.
캡슐화	캡슐화가 불충분하였을 때 발생할 수 있는 보안 약점이다.
API 오용	의도되지 않은 방법으로 API를 사용하거나, 보안에 취약한 API를 사용할 때 발생할 수 있는 보안 약점이다.

출처: 행정 기관 및 공공 기관 정보 시스템 구축·운영 지침 별표 3

[참고] 예외 처리

예외 처리는 예외가 발생할 수 있는 부분에서 예외가 발생했을 때 어떻게 처리할 것인지 지정해 주는 방법이다. 일반적으로 프로그램을 종료시키거나 로그를 남기도록 한다. 파이썬의 경우에는 try:와 except:를 이용하며, 다음과 같이 작성할 수 있다.

try :

예외가 발생할 만한 조건

except :

예외를 처리하는 방법

try: 와 except:를 이용하여 예외를 처리한다.

3. 프로그래밍 언어의 특성에 따라 발생할 수 있는 성능, 가용성 문제를 확인하고, 최적화한다.

(1) 서버 간의 데이터 통신 속도를 확인하여 최적화한다.

서버 간의 데이터 전송 시 프로그래밍 언어의 특성을 적절하지 않게 반영하였거나, 코드 작성이 잘못된 부분은 없는지 확인하여 속도에 문제가 발생하지 않도록 개선한다.

(2) 클라이언트 디스플레이 속도를 확인한다.

클라이언트 설치 파일 또는 웹 UI의 로딩 속도를 확인하여 프로그래밍 언어의 특성을 잘못 반영한 부분은 없는지 점검한다.

(3) 데이터베이스로부터 조회되는 속도를 확인한다.

데이터베이스 내에서 조회하고자 하는 내용이 적절한 시간 내에 응답하는지 확인하고, 프로그래밍 언어의 특성을 적절하지 않게 반영한 부분이 있다면 분석하여 보완한다.

(4) 가용 자원의 현황과 프로그램 구동 시의 점유율을 확인하여 최적화한다.

CPU, 메모리, 디스크 용량의 자원 상황과 점유율을 확인하여 불필요한 코드나 잘못 작성된 코드를 발견하고 수정한다.

(가) CPU 점유율을 확인한다.

CPU의 자원 상황과 점유율을 확인하여 프로그래밍 언어의 특성을 반영한다.

(나) 메모리 점유율을 확인한다.

메모리의 자원 상황과 점유율을 확인하여 프로그래밍 언어의 특성을 반영한다.

(다) 디스크의 사용 점유율을 확인한다.

디스크 용량의 자원 상황과 점유율을 확인하여 프로그래밍 언어의 특성을 반영한다.

(5) 가용성을 침해받았을 경우에 대한 대응 방안을 계획한다.

가용성이 침해될 경우에 대비하여 정상적인 운영을 위한 방안을 계획하고 문서로 작성한다.

4. 불필요하거나 중복된 코드의 제거 등을 통해 개선할 부분은 없는지 파악하여 최적화를 수행한다.

(1) 불필요, 중복 등의 문제가 있는 코드를 파악한다.

(가) 중복되는 코드를 찾아 수정한다.

중복된 코드가 있는지, 유사 데이터들의 그룹이 중복되지는 않았는지 확인하여 중복이 있을 경우 수정한다.

(나) 불필요 코드를 찾아낸다.

불필요하게 포함된 코드는 없는지 확인하여 불필요한 코드가 있다면 삭제를 수행한다.

(다) 복잡한 코드를 찾아낸다.

긴 메소드, 큰 클래스, 긴 파라미터 목록 등을 가진 소스코드를 찾는다.

(라) 표준이 지켜지지 않는 코드를 찾아낸다.

표준이 지켜지지 않았거나 지나치게 일반화된 코드를 찾는다.

(마) 비효율적인 코드를 찾아낸다.

별로 사용되지 않는 코드는 없는지 확인하여 대책을 정한다.

(2) 교체, 삭제, 병합 등을 이용하여 프로그램을 최적화한다.

(가) 불필요하거나 중복된 코드를 삭제한다.

불필요하거나 중복된 코드가 발견되었다면 발견된 중복 코드와 불필요 코드를 삭제한다.

(나) 그룹화할 수 있는 메소드 또는 모듈을 추출한다.

공통적인 특성의 그룹으로 묶을 수 있는 메소드 또는 모듈을 추출하여 그룹으로 묶는다.

(다) 메소드의 위치를 변경하거나 수식을 메소드로 전환한다.

위치 변경이나 전환이 요구되는 메소드를 찾아 위치 변경 및 전환을 수행한다.

(라) 유사 메소드는 합병한다.

유사한 메소드의 경우에는 합병을 수행하여 소스코드를 최적화한다.

[참고] 리팩토링(Refactoring)

프로그램 개발 후 기능의 변화 없이 내부 구조만 변경하는 작업을 리팩토링이라고 한다. 리팩토링은 Delete(제거), Extract(추출), Move(이동), Merge(병합), Replace(전환) 등의 방법을 통해 프로그램을 보다 최적의 상태로 만들어 주는 품질 활동이다.

(3) 테스트를 통해 최적화 과정에서 문법에 어긋나는 사항은 없는지 확인한다.

최적화 과정에서 개발자의 실수 등에 의해 오류가 발생할 수 있으므로 수정한 사항에 대해 테스트를 통한 점검을 수행한다.

(가) 작은따옴표나 큰따옴표가 닫히지 않은 문장은 없는지 확인한다.

작은따옴표나 큰따옴표가 닫히지 않은 경우 오류를 발생시킬 수 있으므로 확인하여 수정한다.

(나) for문, if문 등의 문장 끝에 세미콜론(;)을 붙였는지 확인한다.

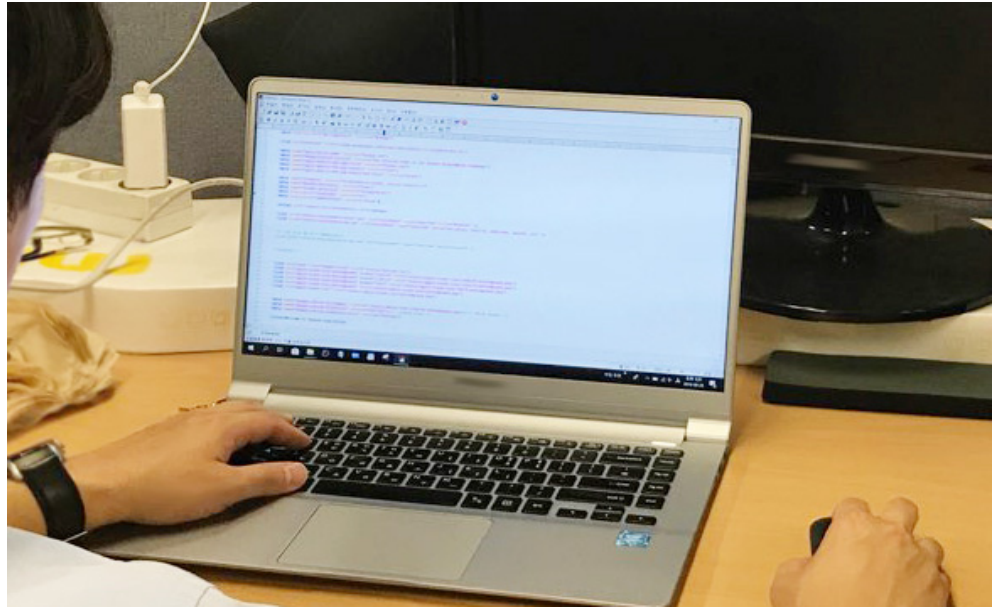
대부분의 프로그래밍 언어는 문장의 끝에 세미콜론(;)을 붙인다. 세미콜론을 붙이지 않는 경우 오류를 발생시킬 수 있으므로 확인하여 수정한다.

(다) 잘못 들어간 공백이 있는지 확인한다.

일부 프로그래밍 언어의 경우에는 들여쓰기 또는 불필요한 공백이 오류를 유발할 수 있으므로 확인하여 제거한다.

(라) 존재하지 않는 함수나 변수를 지정하지 않았는지 확인한다.

정의되지 않은 함수나 변수를 불러오는 경우에는 오류가 발생할 수 있으므로 확인하여 수정한다.



[그림 2-20] 최적화 과정에서 문법 오류를 확인하는 현장 사례

수행 tip

- 프로그래밍 언어에서 제공하는 패턴을 이용하면 효율적이고 재사용성이 높은 프로그램 개발이 가능하다.
- 최근 보안이 중시됨에 따라 프로그램 개발 시에는 잘 알려진 보안 약점에 대한 대비를 수행하여야 한다.
- 최적화 후 테스트를 수행하는 과정에서는 테스트 자동화 도구를 이용하면 효율적이다.

학습 2 교수 · 학습 방법

교수 방법

- 프로그래밍 언어의 특성을 프로그래밍 언어의 발전 과정과 연관지어 설명한다.
- 프로그래밍 언어의 분류 체계 및 주요 언어의 종류를 설명한다.
- 프로그래밍 언어의 특성을 학습자가 설명하도록 하고, 잘못된 내용을 안내한다.
- 특정 프로그래밍 언어를 이용하여 애플리케이션의 구현 실습을 유도한다.
- 프로그래밍 언어의 특성을 이용하여 최적화할 수 있는 방안을 설명한다.

학습 방법

- 프로그래밍 언어가 가지는 특성을 파악한다.
- 프로그래밍 언어의 분류 체계 및 주요 언어의 종류를 파악한다.
- 프로그래밍 언어의 특성을 설명하고, 잘못된 내용을 확인한다.
- 특정 프로그래밍 언어를 이용하여 애플리케이션의 구현을 실습한다.
- 프로그래밍 언어의 특성을 이용하여 최적화하는 방법을 실습한다.

학습 2 평 가

평가 준거

- 평가자는 학습자가 학습 목표를 성공적으로 달성하였는지를 평가해야 한다.
- 평가자는 다음 사항을 평가해야 한다.

학습 내용	학습 목표	성취수준		
		상	중	하
언어 특성 활용	- 프로그래밍 언어별 특성을 파악하고 설명할 수 있다.			
	- 파악된 프로그래밍 언어의 특성을 적용하여 애플리케이션을 구현할 수 있다.			
	- 애플리케이션을 최적화하기 위해 프로그래밍 언어의 특성을 활용할 수 있다.			

평가 방법

- 문제 해결 시나리오

학습 내용	평가 항목	성취수준		
		상	중	하
언어 특성 활용	- 프로그래밍 언어별 특성을 반영할 수 있는 능력			
	- 프로그래밍 언어의 특성을 적용하여 애플리케이션 구현에 대한 문제를 해결할 수 있는 능력			
	- 불필요한 코드를 제외하는 등의 작업을 수행하여 최적화 문제를 해결할 수 있는 능력			

• 평가자 체크리스트

학습 내용	평가 항목	성취수준		
		상	중	하
언어 특성 활용	- 프로그래밍 언어별 특성을 파악할 수 있는 능력			
	- 프로그래밍 언어의 특성을 적용하여 애플리케이션을 구현할 수 있는 능력			
	- 불필요한 코드를 제외하는 등의 작업을 수행하여 애플리케이션을 최적화할 수 있는 능력			

피드백

1. 문제 해결 시나리오

- ‘언어 특성 활용’ 학습 내용에서 제시된 언어별 특성의 적용이 가능한지 평가하고, 올바른 대처 방법과 수정이 필요한 대처 내용을 알려 준다.
- 언어 특성을 이용한 최적화 구현이 가능하도록 개선 방향을 제시한다.
- ‘언어 특성 활용’에 대한 문제 해결 시나리오 결과가 ‘상’인 경우 다른 학습자와 모범 사례를 공유하고 설명할 수 있도록 안내한다.
- ‘언어 특성 활용’에 대한 문제 해결 시나리오 결과가 ‘중’인 경우 부족한 부분을 보완할 수 있도록 설명한다.
- ‘언어 특성 활용’에 대한 문제 해결 시나리오 결과가 ‘하’인 경우 모범 사례를 참조하여 다시 학습할 수 있도록 유도한다.

2. 평가자 체크리스트

- ‘언어 특성 활용’은 프로그래밍 언어 특성을 최대한 활용하여야 하므로 언어 특성에 대한 지식을 반드시 숙지할 수 있도록 안내한다.
- 학습 내용에서 제시된 패턴의 활용이 가능하도록 안내한다.
- ‘언어 특성 활용’에 대한 평가자 체크리스트 결과가 ‘상’인 경우 다른 학습자와 모범 사례를 공유하고 설명할 수 있도록 안내한다.
- ‘언어 특성 활용’에 대한 평가자 체크리스트 결과가 ‘중’인 경우 부족한 부분을 보완할 수 있도록 설명한다.
- ‘언어 특성 활용’에 대한 평가자 체크리스트 결과가 ‘하’인 경우 모범 사례를 참조하여 다시 학습할 수 있도록 유도한다.

학습 1	기본 문법 활용하기
학습 2	언어 특성 활용하기
학습 3	라이브러리 활용하기

3-1. 라이브러리 활용

학습 목표

- 애플리케이션에 필요한 라이브러리를 검색하고 선택할 수 있다.
- 애플리케이션 구현을 위해 선택한 라이브러리를 프로그래밍 언어 특성에 맞게 구성할 수 있다.
- 선택한 라이브러리를 사용하여 애플리케이션 구현에 적용할 수 있다.

필요 지식 /

① 라이브러리(Library)

라이브러리는 효율적인 프로그램 개발을 위해 필요한 프로그램을 모아 놓은 집합체로서, 프로그래밍 언어에 따라 일반적으로 도움말, 설치 파일, 샘플 코드 등을 제공한다.

1. 라이브러리의 개념

라이브러리란 영어로 도서관을 의미하며, 필요할 때 찾아서 쓸 수 있도록 모듈화 되어 제공되는 프로그램을 말한다.

2. 라이브러리의 구성

라이브러리는 도움말, 설치 파일, 샘플 코드 등으로 구성된다.

(1) 도움말: 라이브러리를 사용할 수 있도록 하는 도움말 문서이다.

(2) 설치 파일: 라이브러리를 적용하기 위해 제공되는 설치 파일이다.

(3) 샘플 코드: 라이브러리를 이해하고 손쉽게 적용하기 위해 제공되는 샘플 소스코드이다.

② 표준 라이브러리와 외부 라이브러리

1. 표준 라이브러리

표준 라이브러리는 프로그래밍 언어가 기본적으로 가지고 있는 라이브러리를 의미한다. 각 프로그래밍 언어의 표준 라이브러리는 여러 종류의 모듈과 패키지를 가지며, 표준 라이브러리를 이용하면 별도의 파일 설치 없이 날짜와 시간 등의 기능을 이용할 수 있다.

2. 외부 라이브러리

외부 라이브러리는 표준 라이브러리와 달리 별도의 파일을 설치하여야 한다. 외부 라이브러리는 누구나 개발하여 설치할 수 있으며, 인터넷 등을 이용하여 공유할 수도 있다.

③ 모듈(Module)과 패키지(Package)

라이브러리는 모듈과 패키지를 총칭하며, 모듈이 개별 파일이라면 패키지는 파일들을 모아 놓은 폴더라고 볼 수 있다.

1. 모듈(Module)

한 개의 파일에서 기능을 제공한다.

```
import (모듈명)
```

[그림 3-1] 모듈을 불러오는 방법 예시

2. 패키지(Package)

여러 개의 모듈을 한 개의 폴더에 묶어서 기능을 제공한다. 패키지명과 모듈명을 import 하여 불러올 수 있다.

```
import (패키지명).(모듈명)
```

[그림 3-2] 패키지를 불러오는 방법 예시

수행 내용 / 라이브러리 활용하기

재료 · 자료

- 프로그래밍 언어에 대한 라이브러리 설치 파일
- 프로그래밍 언어에 대한 라이브러리 도움말
- 프로그래밍 언어에 대한 라이브러리 소스코드 예시
- 프로그래밍 언어에 대한 라이브러리 적용 샘플

기기(장비 · 공구)

- 전산 장비: 인터넷, 컴퓨터, 프린터, 복사기, 빔 프로젝터
- 지원 도구: 문서 작성 도구, 문서 발표 도구, 소프트웨어 개발 도구

안전 · 유의 사항

- 계획서를 작성하여 일정에 맞게 학습 및 실습이 가능하도록 준비한다.
- 지식 재산권이 있는 자료의 경우에는 지식 재산권을 확인할 수 있도록 유의한다.
- 개인 정보가 포함된 자료를 취급할 경우에는 보안에 유의하여 이용한다.
- 프로그래밍 언어에 맞는 라이브러리를 선택하도록 한다.

수행 순서

① 애플리케이션에 필요한 라이브러리를 검색하고 선택한다.

1. 프로그래밍 언어가 제공하는 라이브러리에는 어떠한 것들이 있는지 확인한다.

프로그래밍 언어는 개발의 노력을 덜어 주기 위해 패키지화된 라이브러리를 제공한다. 라이브러리의 개념과 역할, 종류를 파악함으로써 라이브러리에는 어떠한 것들이 있는지 확인한다.

(1) 라이브러리의 개념과 역할을 파악한다.

라이브러리란 프로그램을 보다 쉽고 효율적으로 작성하기 위해 개발자들이 사전에 만들어 놓은 양식을 의미한다. 적절한 라이브러리의 선택은 프로그램 개발을 용이하게 해 주므로 개념과 역할을 파악하도록 한다.

(2) 라이브러리의 종류를 파악한다.

라이브러리는 제공 기능에 따라 여러 종류가 있으며, 자주 사용되는 라이브러리에 대해서는 사전에 확인이 필요하다.

파이썬의 경우에는 <https://docs.python.org/3.4/library/index.html>에서 라이브러리에 대한 설명을 얻을 수 있으며, <표 3-1>에서 제시하는 라이브러리의 경우에는 기본적으로 자주 사용되는 라이브러이므로 그 기능을 파악하도록 한다.

<표 3-1> 라이브러리의 종류(파이썬 예시)

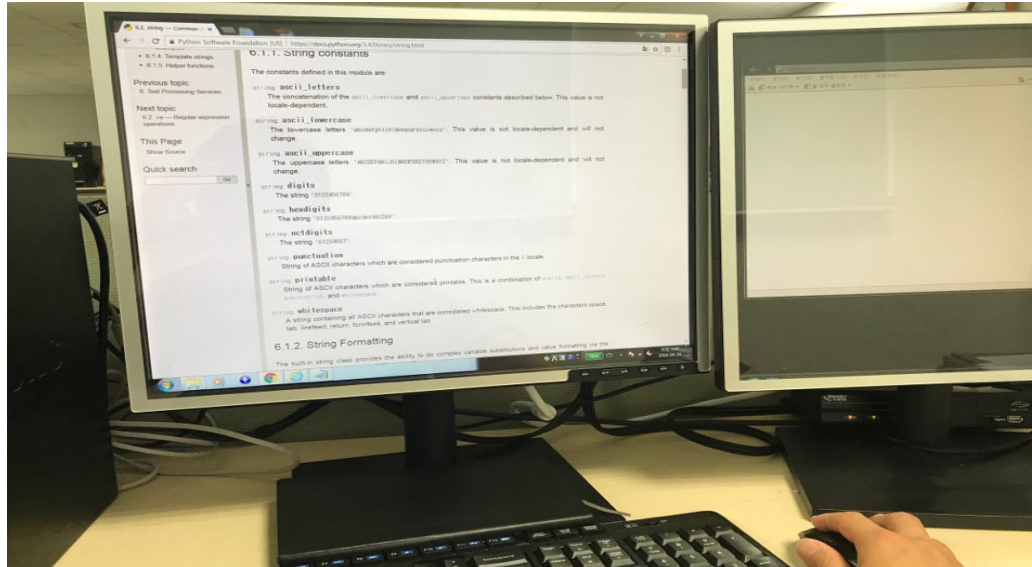
유형	설명
문자열 연산 기능	일반적인 문자열 조작을 수행하며, 파이썬의 경우 https://docs.python.org/3.4/library/string.html 를 참고한다.
문자열 탐색/치환 기능	문자열의 패턴을 정의하여 탐색, 치환 등의 작업에 사용 가능하며, 파이썬의 경우 https://docs.python.org/3.4/library/re.html 를 참고한다.
수치 계산 기능	기본적인 수치 계산을 하고자 하는 경우에는 https://docs.python.org/3.4/library/functions.html 또는 https://docs.python.org/3.4/library/math.html 를 참고한다.
난수 생성 기능	난수를 다루는 작업에 사용하며, 파이썬의 경우에는 https://docs.python.org/3.4/library/random.html 을 참고한다.
파일 경로 조작 기능	파일 경로 관련 기능을 제공하며, 파이썬의 경우에는 https://docs.python.org/3.4/library/os.path.html 을 참고한다.
디렉터리 생성 기능	디렉터리를 생성하는 기능을 제공하며, 파이썬의 경우 https://docs.python.org/3.4/library/tempfile.html 을 참고한다.
날짜 조작 기능	날짜와 시간의 일자와 관련된 작업을 하고자 하는 경우에는 https://docs.python.org/3.4/library/datetime.html 을 참고한다.
로그 출력 기능	로그 출력 기능을 제공하며, 파이썬의 경우에는 https://docs.python.org/3.4/library/logging.html 을 참고한다.

2. 라이브러리를 선택한다.

라이브러리를 선택하기 위해서는 라이브러리를 검색하고, 제약 사항이 있는지 확인하는 절차가 필요하다. 필요한 절차를 거친 후 최종적으로 라이브러리를 선택한다.

(1) 기능 요건을 구현하기에 유용한 라이브러리를 검색한다.

프로그래밍 언어는 기능 요건을 구현하기 위해 많은 유용한 라이브러리를 제공한다. 파이썬에서 제공하는 기본적인 라이브러리는 <표 3-1>을 참고하여 인터넷을 통해 검색할 수 있다.



[그림 3-3] 인터넷을 이용해 라이브러리를 검색하는 현장 사례

[참고] 표준 라이브러리와 외부 라이브러리

라이브러리는 기본적으로 설치되어 있는지, 별도의 파일 설치가 필요한지의 여부에 따라 표준 라이브러리와 외부 라이브러리로 나눌 수 있다.

표준 라이브러리는 프로그래밍 언어가 기본적으로 가지고 있는 라이브러리를 의미하며, 외부 라이브러리는 별도의 파일 설치를 필요로 하는 라이브러리를 의미한다.

(2) 라이브러리를 적용하는 데 제약 사항이 있는지 확인한다.

검색한 라이브러리를 제공하는 데 있어 프로그래밍 언어의 특성, 개발환경, 실행환경 상에서 발생하는 제약 사항은 없는지 확인한다.

(3) 라이브러리를 선택한다.

검색한 라이브러리 중 원하는 라이브러리를 선택한다. 파이썬의 경우에는 <https://docs.python.org/3.4/library/index.html>에서 라이브러리를 다운로드받을 수 있으므로 목적에 맞는 적절한 라이브러리를 선택한다.

[참고] 파이썬2, 파이썬3의 표준 라이브러리 경로

파이썬의 경우 파이썬 버전에 따라 동작하지 않는 라이브러리가 있을 수 있다. 파이썬2에서는 동작하나 파이썬3에서는 동작하지 않는 라이브러리들이 있어 파이썬2를 이용하는 경우도 많다.

다음은 파이썬2, 파이썬3의 표준 라이브러리를 제공하는 경로이다.

- 파이썬2: <https://docs.python.org/2/library/index.html>
- 파이썬3: <https://docs.python.org/3.4/library/index.html>

6.1.4. Template strings

Templates provide simpler string substitutions as described in [PEP 292](#). Instead of the normal %-based substitutions, Templates support \$-based substitutions, using the following rules:

- `$$` is an escape; it is replaced with a single `$`.
- `$identifier` names a substitution placeholder matching a mapping key of `"identifier"`. By default, `"identifier"` is restricted to any case-insensitive ASCII alphanumeric string (including underscores) that starts with an underscore or ASCII letter. The first non-identifier character after the `$` character terminates this placeholder specification.
- `${identifier}` is equivalent to `$identifier`. It is required when valid identifier characters follow the placeholder but are not part of the placeholder, such as `"${noun}ification"`.

Any other appearance of `$` in the string will result in a `ValueError` being raised.

The `string` module provides a `Template` class that implements these rules. The methods of `Template` are:

```
class string.Template(template)
```

The constructor takes a single argument which is the template string.

```
substitute(mapping, **kwds)
```

Performs the template substitution, returning a new string. *mapping* is any dictionary-like object with keys that match the placeholders in the template. Alternatively, you can provide keyword arguments, where the keywords are the placeholders. When both *mapping* and *kwds* are given and there are duplicates, the placeholders from *kwds* take precedence.

[그림 3-4] 템플릿 사용을 위해 선택된 라이브러리 정보 예시(파이썬 예시)

② 선택한 라이브러리를 프로그래밍 언어 특성에 맞게 구성한다.

1. 프로그래밍 언어의 구성이 어떻게 이루어지는지 파악한다.

프로그래밍 언어에 따라 라이브러리를 구성하기 위한 프로그램의 구조와 구성 방법을 파악한다.

(1) 프로그래밍 언어별 구조를 파악한다.

프로그래밍 언어가 가지는 구조를 파악한다. 파이썬의 경우에는 객체 지향 프로그래밍 언어의 일종으로 객체, 클래스, 메소드의 개념을 통해 프로그램을 구현한다. 따라서 파이썬으로 프로그래밍을 하고자 한다면, 객체 지향 프로그래밍의 구조적인 특징을 파악하여야 한다. 구현하고자 하는 프로그래밍 언어에 대한 구조적 특징을 파악한다.

[참고] 객체 지향 프로그래밍

객체 지향 프로그래밍은 객체와 객체와 객체 간의 통신을 통해 구현하는 프로그램 방식을 의미하며, 객체와 클래스, 메시지로 구성된다.

- ① 객체(Object): 객체는 다시 개체, 속성, 메소드로 구성되며 클래스의 인스턴스이다.
- ② 클래스(Class): 객체의 타입을 정의하고 객체를 생성하는 틀의 역할을 한다.
- ③ 메시지(Message): 객체 간 통신이 가능하도록 한다.

(2) 라이브러리 구성 방법을 파악한다.

라이브러리 구성을 위해서는 명령어를 이용하여 라이브러리를 불러오게 되며, 파이썬을 비롯한 일반적인 경우에는 import를 이용하여 라이브러리를 구성한다.



[그림 3-5] 프로그래밍 언어에 따른 라이브러리 구성 방법(파이썬 예시)

2. 프로그래밍 언어 특성에 맞게 구성한다.

프로그래밍 언어 특성에 따라 라이브러리 문서를 제공하며, 라이브러리 문서에는 도움말, 샘플 등이 포함되므로 언어 특성에 맞게 구성하기 위한 방법을 파악한다.

(1) 라이브러리 구성을 위한 도움말을 확인한다.

라이브러리 문서에서 제공하는 도움말을 확인하여 라이브러리의 기능을 파악한다.

(2) 라이브러리 구성을 위한 샘플을 확인한다.

라이브러리 문서에서 제공하는 구성을 위한 샘플 소스코드를 확인한다.

(3) 라이브러리를 언어 특성에 맞게 구성한다.

프로그래밍 언어 특성에 따른 구성 방법을 참고하여 라이브러리를 구성한다.

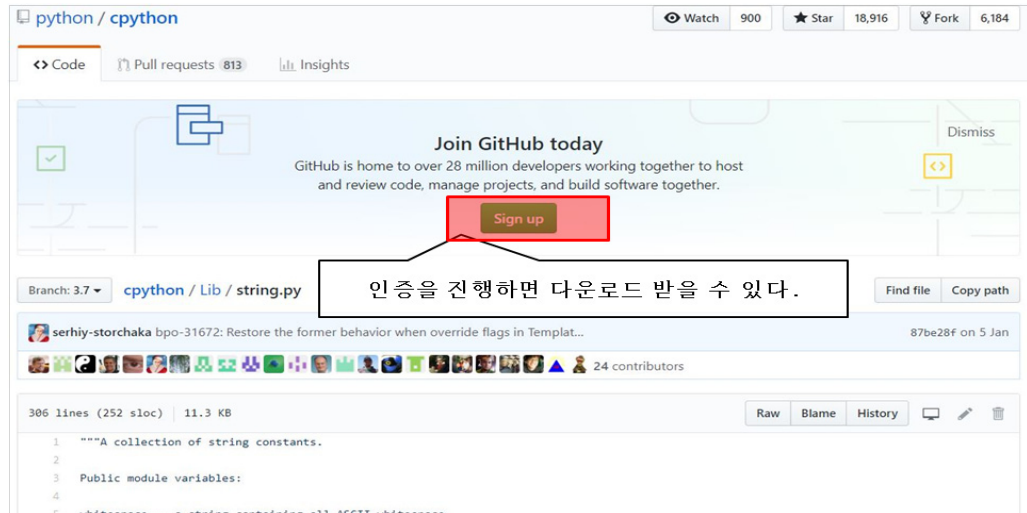
③ 선택한 라이브러리를 사용하여 애플리케이션 구현에 적용한다.

1. 라이브러리를 설치한다.

라이브러리 문서에는 라이브러리 파일이 제공되며, 라이브러리 파일을 다운로드받아 설치한다.

(1) 선택한 라이브러리를 다운로드받는다.

기능 요건을 구현하기 위해 선택한 라이브러리를 다운로드받는다. 파이썬의 경우에는 .py 파일로 되어 있으며, GitHub를 이용하여 다운로드받는다.



[그림 3-6] 파이썬 라이브러리 다운로드 사례

(2) 라이브러리를 설치한다.

다운로드받은 라이브러리를 지정된 폴더에 설치한다.

(3) 설치된 폴더를 찾아 불필요하거나 중복된 파일이 없는지 확인한다.

지정된 폴더에 들어가 불필요하거나 중복되는 파일이 존재하지 않는지 확인한다.

(4) 설치된 라이브러리가 최신 버전인지 확인한다.

라이브러리의 버전이 가장 최신 버전인지 확인하여 최신 버전이 아닌 경우에는 최신 버전으로 다시 다운로드를 받는다.

(5) 손상된 파일은 없는지 확인한다.

파일이 손상되지 않았는지 압축을 해제하여 확인하고, 파일이 손상된 경우에는 다시 다운로드를 받는다.

[참고] 파이썬의 라이브러리의 등록 방법

파이썬의 경우에는 개발된 프로그램(.py)을 라이브러리로 사용할 수 있다. 다음과 같은 방법으로 라이브러리를 등록하고 사용한다.

① 모듈로 만들고 싶은 명령문을 작성한다.

```
print('파이팅!')
```

② 모듈로 만들고 싶은 모듈명으로 명령문을 저장한다.

* 저장된 모듈은 import를 이용하여 불러올 수 있다.

③ import를 수행한다.

```
>>> import 모듈명  
파이팅!
```


2. 선택한 라이브러리를 애플리케이션 구현에 적용한다.

선택한 라이브러리를 프로그래밍 언어에 맞게 애플리케이션 구현을 위해 적용한다.

(1) 라이브러리를 적용한다.

구성된 라이브러리를 실제 프로그램 소스코드에 적용한다.

파이썬의 경우 예시 파일이 <https://docs.python.org/3.4/library/index.html>에서 제공되며, 애플리케이션의 기능 요건에 맞도록 적용한다.

```
>>> from string import Template
>>> s = Template('$who likes $what')
>>> s.substitute(who='tim', what='kung pao')
'tim likes kung pao'
>>> d = dict(who='tim')
>>> Template('Give $who $100').substitute(d)
Traceback (most recent call last):
...
ValueError: Invalid placeholder in string: line 1, col 11
>>> Template('$who likes $what').substitute(d)
Traceback (most recent call last):
...
KeyError: 'what'
>>> Template('$who likes $what').safe_substitute(d)
'tim likes $what'
```

[그림 3-7] 템플릿 사용을 위해 적용한 라이브러리 예제 코드(파이썬 예시)

(2) 라이브러리가 적용되었는지 테스트를 수행한다.

라이브러리가 적용된 후에는 테스트를 수행하여 기능에 문제가 없는지 확인하고, 테스트 케이스 작성을 통해 기능을 검증한다.

(가) 라이브러리 파일이 존재하는지 확인한다.

표준 라이브러리나 외부 라이브러리가 지정한 파일이 존재하는지 파악한다. 라이브러리가 올바르게 설치되지 않았거나, 설치가 되었으나 설정이 잘못된 경우에는 오류가 발생하므로 확인하여 수정한다.

(나) 라이브러리 버전이 정상적으로 동작하는지 확인한다.

최신 라이브러리는 발견되지 않은 오류를 가지고 있을 수 있으므로 안정된 라이브러리 버전을 이용한다.

<표 3-2> 테스트 계획과 테스트 케이스 작성 예시

구분

작성 예시 및 설명

테스트의 목적 및 방법

▪ 본 테스트의 목적은 구현된 프로세스 확인, 시스템의 정확성 확인 및 검증, 사용자의 시스템 접근성 향상을 목적으로 한다.

목적

▪ 본 테스트의 목적은 구현된 프로세스 확인, 시스템의 정확성 확인 및 검증, 사용자의 시스템 접근성 향상을 목적으로 한다.
따라서, 테스트는 각 단위 업무 및 업무 담당자 별로 분리하여 진행을 하고, 단위업무 및 전 업무의 정확성 및 연결의 정확성도 병행하여 검증한다.

테스트 계획

방법

▪ 본 테스트의 목적을 충족시키기 위하여 테스트 수행자는 테스트 시나리오에 기초한 입력 데이터 및 검증 데이터를 구성한다.
▪ 테스트 수행자는 테스트 시나리오에 의해서 테스트 진행, 테스트 결과 확인, 테스트 결과 기록을 수행하고 컨설턴트는 테스트 후 조치사항에 대한 처리 및 그 결과를 오류 조치 방침 따라 처리하고 기록한다.

모든 활동은 계획이 필요하다. 라이브러리 적용 후의 테스트 역시 계획을 필요로 하며, 테스트의 목적과 역할, 조직, 절차 등의 내용을 포함하여 테스트 계획을 수립한다.

테스트 케이스

중분류	TEST CASE	상세 설명	중요도	난이도
기능	E-mail 시스템	E-mail 시스템 - 대량 E-mail 발송	중	중
기능	회원관리기능	회원관리기능 - 통합적 회원 관리 - SMS 발송 서비스 제공 - 상담기록 관리 기능 제공	중	중
기능	통계정보메뉴	통계정보 - 국내통계(주별/월별/분기별/연별) - 해외통계(국가별, 생산, 소비) - 품목별(국내, 해외 교역, 가격)	중	하
기능	게시판 기능설정	게시판 기능설정 - 통합적 관리 기능으로 게시판 생성, 삭제, 수정, 스킴 설정, 에디터 적용 등 포괄적 관리	중	중

라이브러리 적용에 따라 테스트가 필요한 항목들의 순서를 정하고 요구 사항과 매칭하여 테스트 케이스를 도출한다.

(3) 변경된 프로그램에 대한 회귀 테스트를 통해 오류가 제거되었는지 확인한다.

(가) 확인 테스트를 수행한다.

프로그램 오류를 수정한 후에는 수정된 프로그램이 정확한 기능을 제공하는지 확인하는 테스트를 수행한다.

(나) 회귀 테스트를 수행한다.

수정하는 과정에서 다른 프로그램에 영향을 미칠 수 있으므로 기존에 테스트되었던 부분도 다시 테스트하는 회귀 테스트(Regression Test)를 수행한다.

[참고] 회귀 테스트(Regression Test)

회귀 테스트란 결함 수정 후 수정된 프로그램뿐만 아니라 관련된 프로그램도 함께 테스트하는 방법이다. 결함 수정 과정에서 다른 프로그램에 영향을 미치는 부분이 있을 수도 있다는 전제하에 테스트를 진행한다.

수행 tip

- 적절한 라이브러리의 사용은 프로그램을 보다 효율적으로 작성할 수 있도록 한다.
- 최신 버전의 라이브러리가 반드시 좋은 것만은 아니다.
- 결함이 조치되었더라도 다른 부분에 영향이 발생하였을 수 있으므로 적절한 작업량의 회귀 테스트는 필요하다.

교수 방법

- 라이브러리의 개념과 역할을 설명한다.
- 구현하고자 하는 프로그래밍 언어에 대한 라이브러리의 종류를 설명한다.
- 라이브러리를 검색하여 선택하는 방법을 설명한다.
- 라이브러리를 프로그래밍 언어에 맞게 적용하는 방법을 설명한다.
- 라이브러리 적용 후 테스트를 통해 검증을 수행하는 방법을 설명한다.

학습 방법

- 라이브러리의 개념과 역할을 파악한다.
- 구현하고자 하는 프로그래밍 언어에 대한 라이브러리의 종류를 파악한다.
- 라이브러리를 검색하여 선택하는 방법을 실습한다.
- 라이브러리를 프로그래밍 언어에 맞게 적용하는 방법을 실습한다.
- 라이브러리 적용 후 테스트를 통해 검증하는 절차를 실습한다.

학습 3 평 가

평가 준거

- 평가자는 학습자가 학습 목표를 성공적으로 달성하였는지를 평가해야 한다.
- 평가자는 다음 사항을 평가해야 한다.

학습 내용	학습 목표	성취수준		
		상	중	하
라이브러리 활용	- 애플리케이션에 필요한 라이브러리를 검색하고 선택할 수 있다.			
	- 애플리케이션 구현을 위해 선택한 라이브러리를 프로그래밍 언어 특성에 맞게 구성할 수 있다.			
	- 선택한 라이브러리를 사용하여 애플리케이션 구현에 적용할 수 있다.			

평가 방법

- 문제 해결 시나리오

학습 내용	평가 항목	성취수준		
		상	중	하
라이브러리 활용	- 애플리케이션에 필요한 라이브러리를 검색할 수 있는 능력			
	- 애플리케이션 구현을 위해 선택한 라이브러리를 프로그래밍 언어 특성에 맞게 적용할 수 있는 능력			
	- 선택한 라이브러리를 사용하여 올바르게 구현되었는지의 여부를 검증할 수 있는 능력			

• 평가자 체크리스트

학습 내용	평가 항목	성취수준		
		상	중	하
라이브러리 활용	- 애플리케이션에 필요한 라이브러리의 종류를 파악하고, 검색할 수 있는 능력			
	- 검색된 라이브러리의 제약 사항을 파악하여 선택할 수 있는 능력			
	- 선택한 라이브러리를 프로그래밍 언어 특성에 맞게 구성할 수 있는 능력			
	- 선택한 라이브러리를 사용하여 애플리케이션 구현에 적용할 수 있는 능력			

피드백

1. 문제 해결 시나리오

- ‘라이브러리 활용’ 학습 내용에서 제시된 라이브러리의 검색과 선택, 적용이 가능한지 평가하고, 올바른 대처 방법과 수정이 필요한 대처 내용을 알려 준다.
- 라이브러리 적용 후 테스트를 통한 검증이 가능하도록 개선 방향을 제시한다.
- ‘라이브러리 활용’에 대한 문제 해결 시나리오 결과가 ‘상’인 경우 다른 학습자와 모범 사례를 공유하고 설명할 수 있도록 안내한다.
- ‘라이브러리 활용’에 대한 문제 해결 시나리오 결과가 ‘중’인 경우 부족한 부분을 보완할 수 있도록 설명한다.
- ‘라이브러리 활용’에 대한 문제 해결 시나리오 결과가 ‘하’인 경우 모범 사례를 참조하여 다시 학습할 수 있도록 유도한다.

2. 평가자 체크리스트

- ‘라이브러리 활용’은 프로그래밍 언어 특성을 반영하기 위한 효율적인 방법이므로 적절한 선택이 가능하도록 라이브러리 종류와 제약 사항을 숙지할 수 있도록 안내한다.
- 선택한 라이브러리를 적용할 수 있도록 안내한다.
- ‘라이브러리 활용’에 대한 평가자 체크리스트 결과가 ‘상’인 경우 다른 학습자와 모범 사례를 공유하고 설명할 수 있도록 안내한다.
- ‘라이브러리 활용’에 대한 평가자 체크리스트 결과가 ‘중’인 경우 부족한 부분을 보완할 수 있도록 설명한다.
- ‘라이브러리 활용’에 대한 평가자 체크리스트 결과가 ‘하’인 경우 모범 사례를 참조하여 다시 학습할 수 있도록 유도한다.



- 강승준(2013). 『소프트웨어 아키텍처 설계 가이드』. 프리렉.
- 김일민·김성동(2013). 『프로그래밍 언어의 이해』. 홍릉과학출판사.
- 김종훈·김종진(2013). 『프로그래밍 언어론』. 한빛아카데미.
- 김창호(2011). 『Java Design Patterns』. 프리렉.
- 원유현(2011). 『프로그래밍 언어 개념』. 정익사.
- 장세찬(2004). 『GoF의 디자인 패턴! 이렇게 활용한다.』. 한빛미디어.
- 제나디 즐로빈(2014). 『Python Design Patterns』. 에이콘.
- 최은만(2011). 『소프트웨어 공학』. 정익사.
- Bass(2007). 『소프트웨어 아키텍처 이론과 실제』. 에이콘.
- Erich Gamma(2007). 『GoF의 디자인 패턴』. PearsonEducation.
- Kenneth C. Loudon(2004). 『프로그래밍 언어』. 사이텍미디어.
- LAN SOMMERVILLE(2008). 『소프트웨어 공학』. PearsonEducation.
- Roger S. Pressman(2001). 『소프트웨어 공학』. 한국맥그로힐.

NCS학습모듈 개발이력

발행일	2015년 12월 31일		
세분류명	응용SW엔지니어링(20010202)		
개발기관	한국소프트웨어기술진흥협회, 한국직업능력개발원		
집필진	강석진(이비스툼)*	검토진	김승현(경희대학교)
	김보운(이화여자대학교)		엄기영(우리에프아이에스)
	김홍진(LG CNS)		장온순(한국IT컨설팅)
	유은희		조상욱(세종대학교)
	장현섭((주)커리텍)		조성호(삼성카드)
	주선태(T3Q)		
	진권기(이비스툼)		
	최재준		* 표시는 대표집필자임
발행일	2017년 12월 31일		
세분류명	응용SW엔지니어링(20010202)		
개발기관	(사)한국정보통신기술사협회, 한국직업능력개발원		
집필진	박미화(동국대학교)*	검토진	권순명(주씨에이에스)
	김승환((주)캐롯아이)		김태형((사)한국정보통신기술사협회)
	김원기(LG CNS)		양승화(라이나생명보험)
	김종명(SM신용정보)		이성화(시스원)
	박현기(프리랜서)		황국인(주코스콤)
	유현주((사)한국정보통신기술사협회)		
	이구성(한국아이씨티(주))		
	이숙희(서초문화예술정보학교)		
	최창선(한빛디엔에스(주))		
	홍민표(한화S&C)		* 표시는 대표집필자임
발행일	2018년 12월 31일		
학습모듈명	프로그래밍 언어활용(LM2001020215_15v3)		
개발기관	(사)한국정보통신기술사협회, 한국직업능력개발원		
집필진	김광국(주코스콤)*	검토진	류갑상(동신대학교)
	권기학(세종대학교)		엄기영(우리에프아이에스)
	김승환(주캐롯아이)		장성수(세명컴퓨터고등학교)
	김유석(한국정보통신기술사협회)		장희수(주유알피시스템)
	문정기(교보정보통신)		
	박미화(동국대학교 융합SW교육원)		
	이주희(주유알피시스템)		
	홍민표(한화S&C)		* 표시는 대표집필자임

프로그래밍 언어활용(LM2001020215_15v3)

저작권자	교육부
연구기관	한국직업능력개발원
발행일	2018. 12. 31.

※ 이 학습모듈은 자격기본법 시행령(제8조 국가직무능력표준의 활용)에 의거하여 개발하였으며, NCS통합포털사이트(<http://www.ncs.go.kr>)에서 다운로드 할 수 있습니다.



www.ncs.go.kr